VERIQTA

GitLab CI/CD HANDBOOK

FLAGSHIP HANDBOOK
60 PAGES

From Beginner to Enterprise Production CI/CD

- COMPLETE CI/CD FUNDAMENTALS
- SECURE & SCALABLE PIPELINES
- DEPLOY TO ANY ENVIRONMENT
- BEST PRACTICES & REAL-WORLD EXAMPLES
- ENTERPRISE PRODUCTION READY

FASTER DELIVERY | SECURE BY DESIGN | AUTOMATE EVERYTHING | COLLABORATE BETTER

BUILD • TEST • SECURE • DEPLOY • MONITOR • REPEAT

1. Introduction to GitLab CI/CD

What CI/CD is

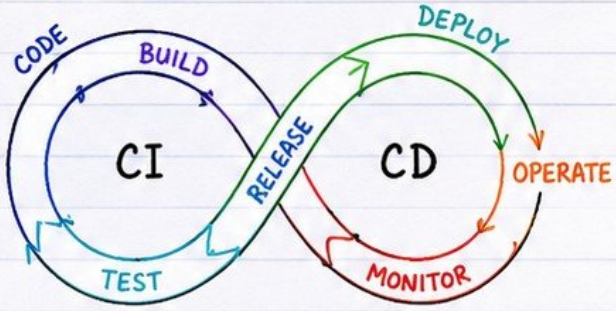
CI/CD stands for Continuous Integration and Continuous Delivery/Deployment. It is a practice that automates the process of building, testing, and delivering code to users quickly and reliably.

Goal: Deliver value to users faster with confidence.

Why GitLab CI/CD matters

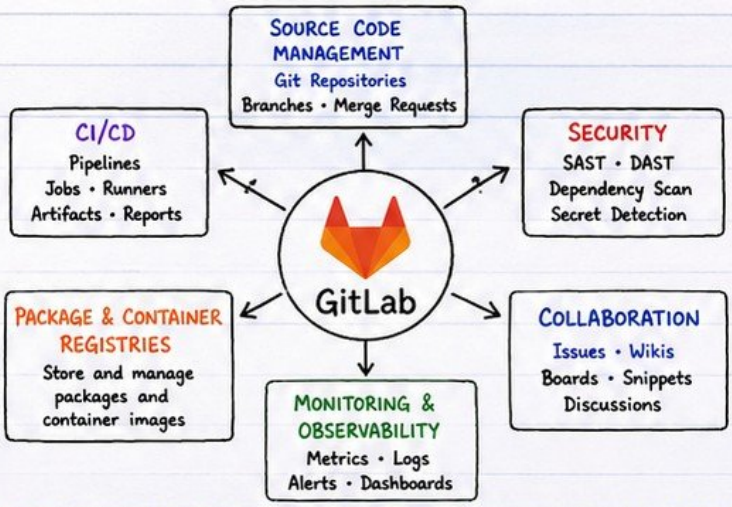
- ☑ Built-in CI/CD - no extra tools required.
- ☑ Deep integration with GitLab features.
- ☑ Secure, scalable, and highly customizable.
- ☑ Saves time through automation.
- ☑ Improve code quality and collaboration.
- ☑ Supports everything from small projects to complex enterprise systems.

DevOps Lifecycle



PLAN → CODE → BUILD → TEST → RELEASE → DEPLOY → OPERATE → MONITOR → FEEDBACK

GitLab Platform Overview



Handbook Objectives

- Understand GitLab CI/CD from basics to enterprise.
- Design scalable, secure, and reliable pipelines.
- Integrate testing, security, and quality into pipelines.
- Automate deployments with confidence.
- Implement best practices for production environments.
- Troubleshoot issues like a senior engineer.
- Build real-world CI/CD systems end-to-end.

Learning Roadmap

1	Fundamentals	Foundation
2	Pipeline Building Blocks	
3	Advanced Pipelines	Build & Secure
4	Security & Quality	
5	Deployments & Environments	Deliver
6	Integrations & Automation	
7	Enterprise & Best Practices	Scale & Operate
8	Troubleshooting & Optimization	

NOTE: This handbook takes you from beginner concepts to enterprise-grade CI/CD solutions using GitLab as the central DevOps platform.

2. GITLAB PLATFORM ARCHITECTURE

GITLAB COMPONENTS

GitLab is an all-in-one DevOps platform. Its architecture is built around modular components working together to deliver the complete CI/CD experience.

① GitLab Server	The core application.
② GitLab Runner	Executes jobs.
③ Repository	Stores your source code.
④ Registry	Stores container images and packages.
⑤ Pipeline Engine	Orchestrates pipelines and jobs.

COMPONENT DETAILS

GITLAB SERVER

Hosts the application, handles authentication, authorizes access, manages repositories, runs the CI/CD pipeline engine and provides APIs.

GITLAB RUNNER

Executes CI/CD jobs. Runners can be shared, group, project or instance level. Executors can be Docker, Kubernetes, Shell, SSH, or Virtual Machine.

REPOSITORY

Stores source code in Git. Supports branching, merging, tags, and version control.

REGISTRY

Stores Docker images and packages securely. Integrated with CI/CD for build and deploy.

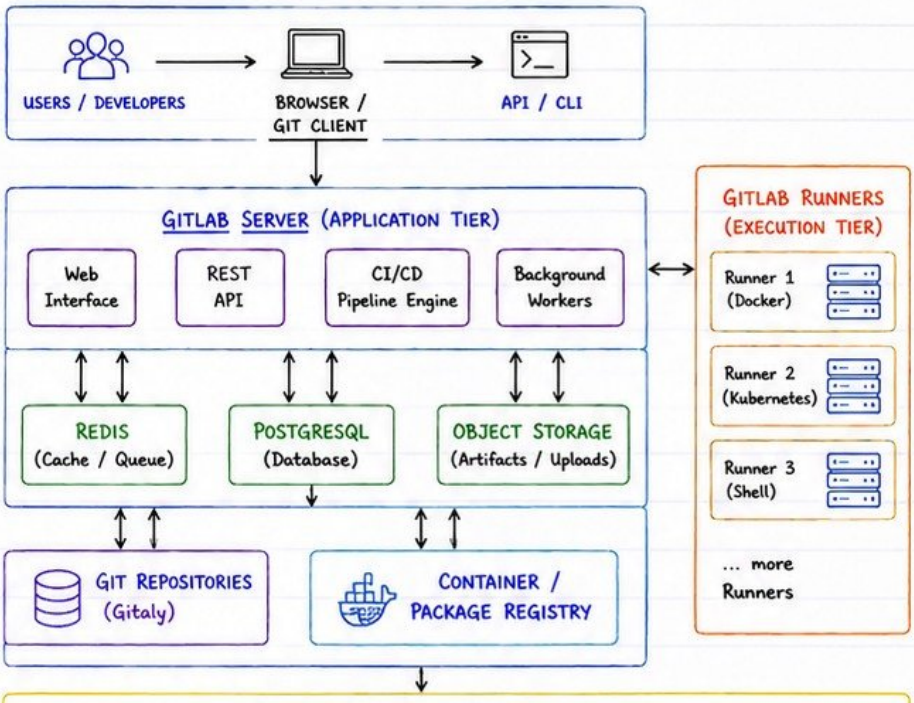
PIPELINE ENGINE

Interprets .gitlab-ci.yml, creates pipelines, schedules jobs, manages dependencies and tracks status.

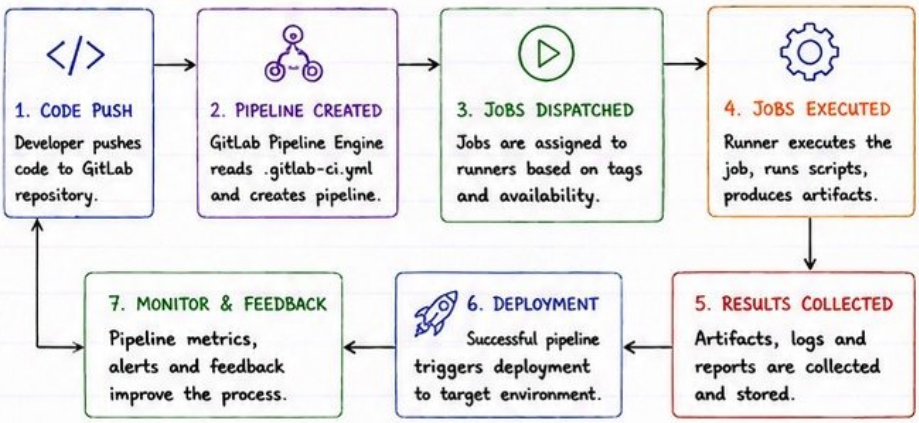
PRODUCTION CONSIDERATIONS

- ☑ Use HA GitLab architecture.
- ☑ Separate runners from server.
- ☑ Use dedicated database & Redis.
- ☑ Backup regularly.
- ☑ Monitor performance and jobs.

PRODUCTION ARCHITECTURE OVERVIEW



CI/CD FLOW OVERVIEW



KEY BENEFITS

- Single platform for the entire DevOps lifecycle.
- Tight integration between code, CI/CD, security, and deployments.
- Scalable from small teams to large enterprises.
- Secure, extensible, and cloud-native ready.

3. INSTALLING GITLAB

SELF-MANAGED vs GITLAB.COM

SELF-MANAGED GITLAB	GITLAB.COM (SaaS)
☑ Full control over data	☑ No infrastructure to manage
☑ Custom configurations	☑ Automatic updates
☑ Works behind firewall	☑ Global availability
☑ No vendor lock-in	☑ Faster to get started
☑ Requires infrastructure	☑ Limited customization
☑ You manage updates	☑ Dependent on GitLab
☑ Best for enterprises	☑ Best for teams/startups

CHOOSE SELF-MANAGED FOR:
Compliance, security, data residency, customization, or offline environments.

INSTALLATION METHODS

- **DOCKER** — Fast, portable, easy to manage.
- **LINUX PACKAGE** — Deb/RPM packages.
- **OMNIBUS** — All-in-one package (Linux).
- **CLOUD** — AWS, Azure, GCP Marketplace.
- **SOURCE** — For advanced customization.

PRODUCTION NOTE:
For production, use Omnibus (Linux) or Kubernetes Helm chart for HA deployments.

DOCKER DEPLOYMENT

1. Install Docker

```
# On Ubuntu
sudo apt update && sudo apt install -y docker.io
```

2. Run GitLab Community Edition

```
docker run -d \
  --name gitlab \
  --hostname gitlab.local \
  --publish 80:80 --publish 443:443 --publish 2222:22 \
  --restart always \
  --volume gitlab_config:/etc/gitlab \
  --volume gitlab_logs:/var/log/gitlab \
  --volume gitlab_data:/var/opt/gitlab \
  gitlab/gitlab-ce:latest
```

3. Access GitLab

URL: http://gitlab.local
(Or use your server IP / domain)

DOCKER VOLUMES
Config → /etc/gitlab
Logs → /var/log/gitlab
Data → /var/opt/gitlab
Always backup /var/opt/gitlab

LINUX INSTALLATION (OMNIBUS)

1. Add GitLab package repository

```
curl -fsSL https://packages.gitlab.com/install/repostiories/gitlab/gitlab-ce/script.deb.sh | sudo bash
```

2. Install GitLab

```
sudo apt install gitlab-ce
```

3. Reconfigure GitLab

```
sudo gitlab-ctl reconfigure
```

4. Start/Stop GitLab

```
sudo gitlab-ctl start     # start services
sudo gitlab-ctl stop      # stop services
```

OMNIBUS INCLUDES:
☑ GitLab Rails App
☑ PostgreSQL
☑ Redis
☑ Nginx
☑ Sidekiq
☑ Gitaly
☑ All pre-configured!

INITIAL CONFIGURATION

1. Set external URL

```
sudo gitlab-ctl reconfigure
```

2. Edit configuration (if needed)

```
sudo nano /etc/gitlab/gitlab.rb
external_url 'https://gitlab.yourdomain.com'
```

3. Reconfigure

```
sudo gitlab-ctl reconfigure
```

4. Verify status

```
sudo gitlab-ctl status
```

KEY SETTINGS
- external_url
- nginx['listen_port']
- nginx['listen_https']
- lets_encrypt['enable']
- gitlab_rails['time_zone']
- smtp settings
- backup settings

ENABLE HTTPS (Let's Encrypt)

```
sudo gitlab-ctl reconfigure
# Ensure ports 80 & 443 are open
```

FIRST LOGIN

1. Open your browser

```
https://gitlab.yourdomain.com
```

2. Default sign-in (first user)

Username: root
Password: Check the initial root password
at: sudo cat /etc/gitlab/initial_root_password

3. Change default password

Go to: User Settings → Edit Password

SECURITY TIP
Change the root password immediately. Create a new admin user and use it for daily operations.

PRODUCTION BEST PRACTICES

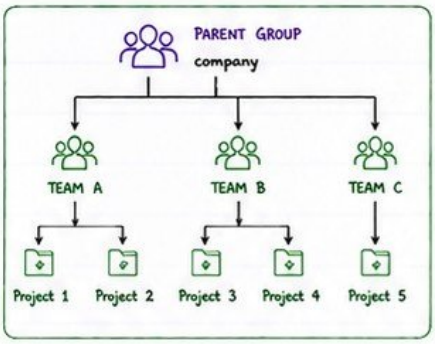
☑ Use a dedicated server.
☑ Keep backups regularly.
☑ Configure SSL/TLS.
☑ Use external PostgreSQL for scale.
☑ Monitor resources (CPU, RAM, Disk).
☑ Keep GitLab updated.

GitLab is powerful, but production success comes from planning & good operations!

4. GitLab Projects and Repositories

Groups

- Groups are containers for projects.
- Help organize teams and repositories.
- Control access, permissions and policies at the group level.
- Can be public, internal or private.



Projects

A project contains your repository, CI/CD pipelines, issues, wiki, and more.

Each project belongs to a group.

Projects can be public, internal, or private.

Project settings control visibility, permissions, branches and CI/CD.

PROJECT FEATURES

- ☑ Repository
- ☑ CI/CD Pipelines
- ☑ Issues & Boards
- ☑ Wiki & Snippets
- ☑ Container Registry
- ☑ Packages
- ☑ Deployments
- ☑ Environments
- ☑ Access Control
- ☑ Audit Events

Repository Structure

```
my-application/
├── .gitlab/                  # GitLab specific configs
├── .gitlab-ci.yml            # Pipeline definition
├── README.md                 # Project documentation
├── src/                      # Source code
│   ├── main.py
│   └── utils.py
├── tests/                    # Test files
│   └── test_main.py
├── Dockerfile                # Container build
├── docker-compose.yml        # Local env
├── requirements.txt          # Dependencies
└── docs/                     # Documentation
    └── architecture.md
```

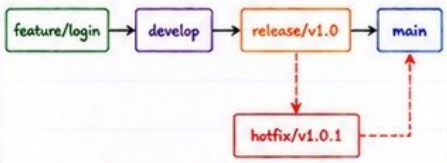
TIP:
A clean repo structure makes your code easy to navigate, test, and scale. Keep code, configs, docs and pipelines well organized.

Branches

- Branches allow you to work on features, fixes, or experiments in isolation.
- Common branch types:

BRANCH TYPE	PURPOSE
main / master	Production-ready code
develop	Integration branch
feature/*	New features
bugfix/*	Bug fixes
release/*	Release preparation
hotfix/*	Urgent production fixes

BRANCH WORKFLOW (EXAMPLE)



Protected Branches

- Protected branches prevent accidental deletions and unsafe pushes.
- They enforce code quality and approvals.

COMMON PROTECTIONS

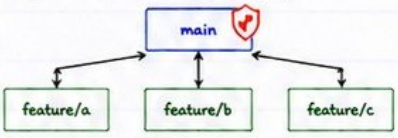
- ☑ No force push
- ☑ No branch deletion
- ☑ Require Merge Request
- ☑ Require approvals
- ☑ Require status checks (CI pipeline)
- ☑ Restrict who can push
- ☑ Signed commits (optional)

Protect your main and release branches!

Default Branch Strategy

- The default branch is the main line of development and deployment.
- Recommended strategy:

1. Use 'main' as default branch.
2. All code changes go through Merge Requests.
3. CI/CD pipelines run on every MR and branch.
4. Only merge after approvals and successful checks.
5. Keep main always deployable.



All changes flow into main via MRs

Repository Organization (Best Practices)

- ☑ Use groups to reflect your company structure.
- ☑ Keep projects focused and scoped to a single product or service.
- ☑ Use meaningful names for projects and repositories.
- ☑ Follow consistent branching strategy across teams.
- ☑ Protect critical branches and enforce code quality.
- ☑ Use README.md to document setup, build, and deploy.
- ☑ Use .gitlab-ci.yml at the root of the repository.
- ☑ Store secrets in CI/CD variables, not in code.

REMEMBER:
A well-organized repository is the foundation of a successful CI/CD pipeline and a scalable DevOps culture.

BENEFITS

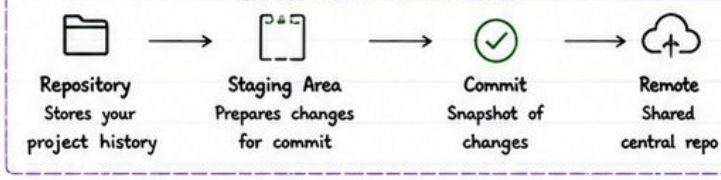
- ☑ Easier collaboration
- ☑ Better security
- ☑ Reliable automation
- ☑ Faster delivery
- ☑ Scalable workflows
- ☑ Happy teams ☺

Plan well.
Organize better.
Ship faster.

5. GIT FUNDAMENTALS FOR GITLAB

GIT is the foundation of everything in GitLab. Master these commands and workflows to collaborate like a pro and keep your codebase clean and safe.

GIT CONCEPTS IN ONE LINE



1 CLONE

Clone a remote repository to your local machine.

```
git clone https://gitlab.com/group/project.git
cd project
```

2 COMMIT

Save your changes to the local repository.

```
git add .
git commit -m "Your meaningful message"
```

3 PUSH

Push your committed changes to the remote repository.

```
git push origin main
```

4 PULL

Fetch and merge changes from the remote repository.

```
git pull origin main
```

5 FETCH

Download changes from remote, but don't merge.

```
git fetch origin
```

6 MERGE

Merge a branch into your current branch.

```
git merge feature-branch
```

7 REBASE

Reapply commits from one branch on top of another.

```
git checkout feature-branch
git rebase main
```

8 TAGS

Mark a specific point in history (release/version).

```
git tag -a v1.0.0 -m "Version 1.0.0"
git push origin v1.0.0
```

COMMIT MESSAGE CONVENTION (EXAMPLE)

```
type(scope): short description
                |
More detailed explanation (optional)
                |
Related issue: #123
```

Types:

- feat: new feature
- fix: bug fix
- docs: documentation
- refactor: code change
- test: adding tests
- chore: maintenance

9 BRANCH WORKFLOWS (COMMON MODELS)

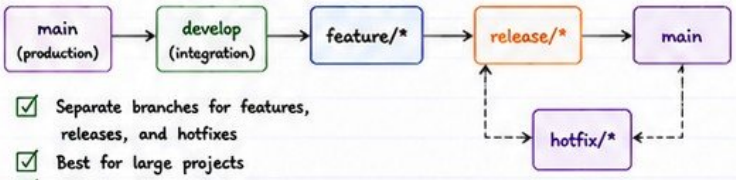
A. GITLAB FLOW (SIMPLIFIED)

- ☑ All work in feature branches
- ☑ Merge via MR with code review
- ☑ main is always stable and deployable

B. GITHUB FLOW (SIMPLE PROJECTS)

- ☑ Direct merge to main
- ☑ Good for small teams
- ☑ Keep history linear

C. GITFLOW (ENTERPRISE)



- ☑ Separate branches for features, releases, and hotfixes
- ☑ Best for large projects
- ☑ More process, more control

BEST PRACTICES

- ☑ Commit small, meaningful changes.
- ☑ Write clear commit messages.
- ☑ Pull before you push.
- ☑ Always work in feature branches.
- ☑ Use Merge Requests for collaboration.
- ☑ Keep main protected and stable.

EXAMPLE WORKFLOW (END TO END)

TIP: Master these basics and everything else in GitLab CI/CD will make perfect sense!

6. MERGE REQUESTS

Merge Requests (MRs) are the heart of collaboration in GitLab. They provide a structured way to propose changes, review code, discuss improvements, and safely merge into target branches.

MERGE REQUEST LIFECYCLE

Create MR
Propose changes

→ Review
Team reviews the changes

→ Approve
Required approvals

→ Discuss
Resolve conversations

→ Merge
Integrate safely

1 CREATING MERGE REQUESTS

- Push your branch to the remote.
- In GitLab, click "New merge request".
- Choose source branch and target branch.
- Add a clear title and description.
- Assign reviewers and labels.

EXAMPLE:

Source branch : feature/login
Target branch : main
Title : Add user login with JWT
description : Implements login endpoint and token generation.

2 REVIEWS

- Reviewers examine code for correctness, security, performance, and standards.
- Use inline comments to suggest improvements.
- Request changes if something needs to be fixed.
- Approve when the code is good to go.

REVIEW CHECKLIST

- ☑ Functionality works as expected
- ☑ Code is readable and maintainable
- ☑ Tests added or updated
- ☑ Security considerations reviewed
- ☑ No unnecessary changes
- ☑ Documentation updated

3 CODE APPROVALS

- Approvals indicate the code is ready to merge.
- Approval rules ensure quality and compliance.
- Protected branches require approvals.

APPROVAL CONDITIONS EXAMPLE

- ☑ Minimum 2 approvals from Developers group
- ☑ No pending "Request changes"
- ☑ All pipelines must pass
- ☑ Code must be up to date with target branch

BEST PRACTICES

- ☑ Keep MRs small and focused.
- ☑ Write clear titles and descriptions.
- ☑ Link related issues in the MR.
- ☑ Review early and often.
- ☑ Keep your branch updated with target branch.

4 DISCUSSIONS

- Use discussions to ask questions, share ideas, or clarify logic.
- Keep conversations respectful and productive.
- Mark as resolved when the issue is fixed.

TIPS

- ☑ Be clear and specific.
- ☑ Reference lines of code.
- ☑ Share context and examples.
- ☑ Resolve conversations after fixes.

5 DRAFT MERGE REQUESTS

- Use draft MRs when work is incomplete.
- Lets the team know it's a work in progress.
- Convert to ready when it's ready for review.

HOW TO MARK AS DRAFT

In the MR title add "[Draft]" or click "Mark as draft".
Example: [Draft] Add login endpoint

6 APPROVAL RULES

- Set rules to enforce code quality.
- Defined at project or group level.
- Ensures nothing is merged without checks.

COMMON APPROVAL RULES

- ☑ Minimum number of approvals
- ☑ Specific users or groups must approve
- ☑ All discussions must be resolved
- ☑ Pipelines must succeed
- ☑ Code owners approval (if configured)
- ☑ No approvals from author (optional)

CODE OWNERS EXAMPLE

CODEOWNERS file defines who must approve certain paths.

/src/backend/	@backend-team
/src/frontend/	@frontend-team
/infra/	@devops-team

7 MERGE STRATEGIES

STRATEGY	DESCRIPTION	BEST USED WHEN	PROS	CONS
Merge Commit (default)	Creates a merge commit to combine branches.	Preserve history of feature branches.	Easy to understand Safe, non-destructive	History can become busy with many merge commits
Fast-Forward Merge	Moves the target branch pointer to the source (branch must be up to date).	Linear history without extra merge commits.	Clean history Simple	Not possible if target has new commits
Squash and Merge	Combines all commits into one new commit.	Keep history clean and simple.	Clean history Easier to follow	Loses individual commit history
Rebase and Merge	Rebases source branch commits on top of target and merges.	Keep linear history with individual commits.	Clean & linear history Preserves commits	Rewriting history can be risky if shared

MR TITLE EXAMPLES

- ☑ Add user registration endpoint
- ☑ Fix bug in token validation
- ☑ Refactor auth service for clarity
- ☑ Update README with setup steps

TIPS

- ☑ Use labels to categorize MRs.
- ☑ Use assignees to keep ownership clear.
- ☑ Close stale MRs to reduce noise.
- ☑ Automate checks with CI pipelines.
- ☑ Communicate, review, and merge with confidence!

7. GITLAB CI/CD FUNDAMENTALS

WHAT IS GITLAB CI/CD?

GitLab CI/CD automates the build, test, package, and deploy process.
It helps teams deliver high-quality software faster and more reliably.

KEY BENEFITS

- ☑ Automate everything
- ☑ Reduce manual errors
- ☑ Faster feedback
- ☑ Consistent delivery
- ☑ Better collaboration
- ☑ Ship with confidence

CI/CD PIPELINE LIFECYCLE

A pipeline is triggered by events like push, merge request, schedule, or manual trigger.

① JOBS

- A job is the smallest unit of work in GitLab CI/CD.
- Defined in .gitlab-ci.yml.
- Runs a script on a runner.
- Example:

```
build:
  script:
    - npm install
    - npm run build
```

② STAGES

- Stages define the order of jobs.
- Jobs in the same stage run in parallel.
- Example:

```
stages:
  - build
  - test
  - package
  - deploy
```

③ RUNNERS

- Runners pick up jobs and execute them.
- Can be shared, group, or project specific.
- Supported executors: Shell, Docker, Kubernetes, SSH, Virtual Machine.

RUNNER TYPES

Shared Runner	→	Managed by GitLab
Group Runner	→	Shared within group
Project Runner	→	Dedicated to project
Specific Runner	→	Tagged for specific jobs

④ ARTIFACTS

- Artifacts are files generated by jobs and stored by GitLab.
- Useful for sharing build outputs, reports, or binaries.
- Example:

```
artifacts:
  paths:
    - dist/
    - coverage/
  expire_in: 7 days
```

USE CASES

- ☑ Share build outputs
- ☑ Publish test reports
- ☑ Deploy files
- ☑ Store logs

⑤ DEPENDENCIES

- Control job execution order and data flow.
- Use needs, dependencies, or artifacts.
- Example with needs:

```
test:
  stage: test
  needs: ["build"]
  script:
    - npm test
```

TYPES

needs	→	Defines job dependencies
dependencies	→	Download artifacts from jobs
artifacts	→	Pass files between jobs

⑥ EXECUTION FLOW (HOW IT WORKS)

BEHIND THE SCENES

- ☑ GitLab creates a pipeline based on .gitlab-ci.yml
- ☑ Jobs are added to a queue
- ☑ Runners pick jobs, execute scripts, and return results
- ☑ Artifacts and logs are stored
- ☑ Status is updated in real-time

IMPORTANT NOTES

- ☑ Jobs in the same stage run in parallel.
- ☑ Next stage starts when all jobs in previous stage succeed.
- ☑ Failed job stops the pipeline (unless allowed to continue).
- ☑ Artifacts and caches optimize performance.

EXAMPLE PIPELINE OVERVIEW

STAGE	BUILD (Parallel)		TEST (Parallel)		PACKAGE		DEPLOY	
JOBS	install	build	unit-test	lint	docker-build	publish	staging	production
DEPENDS ON	-		build		build		package	
ARTIFACTS	✓		✓		✓		(configs)	
RUNNER	Docker Runner		Docker Runner		Docker Runner		Kubernetes Runner	

QUICK REFERENCE

- Job = Do work
- Stage = Organize order
- Runner = Execute jobs
- Artifacts = Share outputs
- Dependencies = Control flow
- Pipeline = Full execution

8. UNDERSTANDING .GITLAB-CI.YML

WHAT IS .GITLAB-CI.YML?

The .gitlab-ci.yml file is the heart of your GitLab CI/CD pipeline. It defines what runs, how it runs, and in what order.

TIP:
This file lives in the root of your repository and is version-controlled like your code.

FILE STRUCTURE (HIGH LEVEL)

```
.gitlab-ci.yml
├── stages
│     - build                    # Define stage order
│     - test
│     - package
│     - deploy
├── variables                    # Default settings
├── default                      # Default settings for jobs
├── before_script / after_script
└── jobs                         # Jobs configuration
    ├── build
    ├── test
    ├── package
    └── deploy
```

GOLDEN RULE

Every valid .gitlab-ci.yml must have at least one job and a valid stage definition (or implicit stage).

EXAMPLE MINIMAL FILE

```yaml
stages:
  - build

build-job:
  script:
    - echo "Hello GitLab CI/CD!"
```

1 FILE STRUCTURE EXAMPLE

```yaml
stages:
  - build
  - test
  - package
  - deploy

variables:
  APP_NAME: my-app
  NODE_VERSION: "18"

default:
  image: node:${NODE_VERSION}
  before_script:
    - npm ci
  after_script:
    - echo "Job finished: $CI_JOB_NAME"

build:
  stage: build
  script:
    - npm run build
  artifacts:
    paths:
      - dist/

unit-test:
  stage: test
  script:
    - npm run test
  needs: ["build"]

package:
  stage: package
  script:
    - echo "Packaging..."
  artifacts:
    paths:
      - build/

deploy:
  stage: deploy
  script:
    - echo "Deploying..."
  environment: production
  only:
    - main
```

2 KEYWORDS EXPLAINED

KEYWORD	DESCRIPTION
stages	Defines the order of stages in the pipeline.
variables	Sets global variables available to all jobs.
default	Defines default settings for all jobs.
image	Docker image used to run the job.
before_script	Commands run before the main script.
script	Main commands that perform the job.
after_script	Commands run after the main script.
artifacts	Files/directories to store and pass between jobs.
needs	Defines job dependencies (runs after specific jobs).
only / except	Controls when a job runs.
environment	Deploy target environment (e.g., production).
rules	Advanced conditional job execution.
when	Controls when a job runs (on_success, manual, etc.).
tags	Assign jobs to specific runners.
cache	Reuse files/directories between jobs to speed up builds.

3 SYNTAX BASICS

- [x] YAML format (spaces, not tabs).
- [x] Indentation matters (2 spaces recommended).
- [x] Case-sensitive.
- [x] Use colons (:) to define keys.
- [x] Use dashes (-) for lists.

SYNTAX EXAMPLE

```yaml
job-name:
  stage: build
  image: node:18
  script:
    - npm install
    - npm run build
  artifacts:
    paths:
      - dist/
  only:
    - main
```

4 VALIDATION

- GitLab validates your .gitlab-ci.yml automatically.
- Errors are shown in the pipeline editor.
- Common issues:
  - ✗ Indentation mistakes
  - ✗ Missing colon
  - ✗ Invalid keyword
  - ✗ Unknown stage
  - ✗ Syntax errors

PRO TIP:
Use CI Lint to validate your YAML before committing.
Project → CI/CD → Editor → CI Lint

5 PIPELINE PARSING (HOW GITLAB READS IT)

1. Read .gitlab-ci.yml → 2. Parse YAML & Validate → 3. Build Pipeline Stages & Jobs → 4. Optimize DAG / Needs → 5. Execute Pipeline

If parsing fails → pipeline does not run.

7 COMMON FILE PATTERNS

PATTERN	USE CASE	EXAMPLE
Single stage	Simple builds	stages: [build]
Multi stage	Build, test, deploy	stages: [build, test, deploy]
DAG (needs)	Faster pipelines	needs: [job1, job2]
Manual jobs	On-demand actions	when: manual
Scheduled jobs	Nightly builds	only: [schedules]

BEST PRACTICES

- [x] Keep it simple and modular.
- [x] Reuse with YAML anchors & includes.
- [x] Use variables for values that change.
- [x] Cache dependencies to speed up builds.
- [x] Store artifacts only when necessary.
- [x] Use needs: to optimize execution time.
- [x] Keep secrets in CI/CD variables (not in code).
- [x] Document your pipeline for your team.

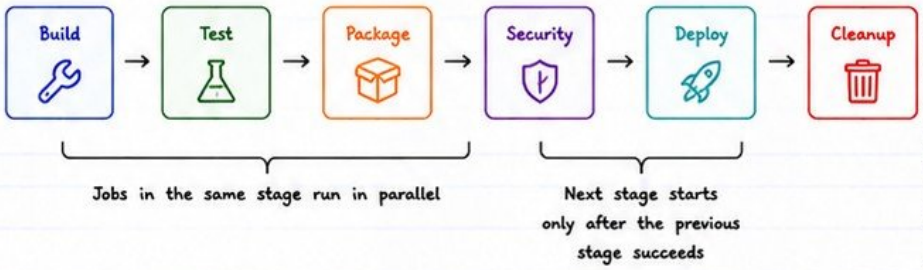
REMEMBER
A clean .gitlab-ci.yml leads to faster pipelines, low maintenance, and happier teams!

9. PIPELINE STAGES

WHAT ARE PIPELINE STAGES?

Stages define the sequential phases of your CI/CD pipeline. Jobs within the same stage run in parallel, and stages run in the defined order from left to right.

STAGE EXECUTION MODEL



BEST PRACTICES

- ☑ Keep stages simple and focused.
- ☑ Fail fast: detect issues early in the pipeline.
- ☑ Run security checks before deployment.
- ☑ Deploy to smaller env first, then promote.
- ☑ Always clean up to save resources and cost.

DEFAULT STAGE ORDER (RECOMMENDED)

1 Build → 2 Test → 3 Package → 4 Security → 5 Deploy → 6 Cleanup

STAGES EXPLAINED

STAGE	PURPOSE	WHAT HAPPENS	KEY JOBS EXAMPLES	OUTPUT / RESULT
Build	Compile source code and create build artifacts.	• Install dependencies • Compile / transpile • Run linters	• build • compile • lint	Compiled application, binaries, build artifacts
Test	Verify code quality and functionality.	• Unit tests • Integration tests • Code coverage	• unit-test • integration-test • coverage	Test reports, coverage results
Package	Create distributable artifacts.	• Package application • Version artifacts • Store for distribution	• package • docker-build • publish	Docker image, JAR, ZIP, package artifacts
Security	Find vulnerabilities and enforce policies.	• SAST / DAST scans • Dependency scan • IaC / Secret scan	• sast-scan • dependency-scan • secret-scan	Security reports, vulnerability status
Deploy	Deploy to target environment.	• Deploy to environments • Run migrations • Health checks	• deploy-staging • deploy-production • smoke-test	Running application in target environment
Cleanup	Remove temporary resources and artifacts.	• Remove preview envs • Delete old artifacts • Clean infrastructure	• cleanup-env • cleanup-artifacts • cleanup-cache	Clean environment, reduced cost

STAGE ORDERING RULES

- ☑ Stages run strictly from left to right.
- ☑ A stage starts only after the previous stage succeeds.
- ☑ Jobs within the same stage run in parallel.
- ☑ If a stage fails, downstream stages will not run.
- ☑ Use "needs" to optimize for advanced pipelines (DAG).

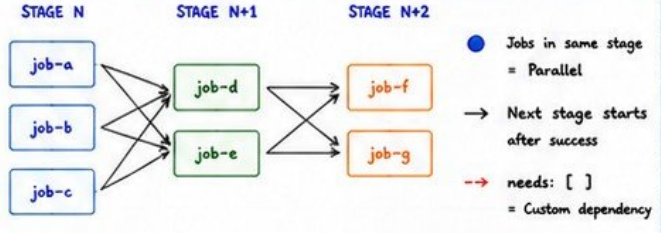
EXAMPLE .GITLAB-CI.YML

```
stages:
  - build
  - test
  - package
  - security
  - deploy
  - cleanup
# Jobs will be defined below
# for each stage
```

STAGE STATUS FLOW

- Success → Next stage runs
- Failed → Pipeline stops
- Skipped → Stage not executed (due to rules / conditions)
- Manual → Waiting for approval

STAGE DEPENDENCIES



REMEMBER

- ☑ Good stage design = Fast, reliable pipelines.
- ☑ Security before production.
- ☑ Clean up always!

COMMON ANTI-PATTERNS

- ✗ Too many or too few stages.
- ✗ Security checks after deploy.
- ✗ Deploying without automated tests.
- ✗ Not cleaning up resources.
- ✗ Sequential jobs when parallel execution is possible.

10. GITLAB JOBS

WHAT IS A JOB?

A job is a set of instructions that GitLab Runner executes in a defined environment. Jobs are the building blocks of your pipeline.

KEY POINT

Jobs run in the context of a stage. Multiple jobs in the same stage run in parallel.

1 JOB DEFINITION

- Each job is defined under the job keyword.
- It belongs to a stage.
- It contains settings and the commands to run.

2 SCRIPT (REQUIRED)

- The main commands executed by the job.
- Can be a single command or an array of commands.

3 BEFORE_SCRIPT (OPTIONAL)

- Commands executed before the script.
- Useful for setup, installing tools, logging in, etc.

4 AFTER_SCRIPT (OPTIONAL)

- Commands executed after the script, regardless of success or failure.
- Ideal for cleanup, notifications, or artifact handling.

5 RETRY

- Automatically retry a job if it fails.
- Helps reduce flaky test or network issues.

6 TIMEOUT

- Maximum time a job is allowed to run.
- Prevents stuck jobs from blocking pipelines.

7 TAGS

- Specifies which runners can pick up the job.
- Runner must have matching tags.

8 INTERRUPTIBLE JOBS

- Allows a running job to be canceled when a new
- pipeline starts for the same ref.
- Saves CI/CD minutes and resources.

JOB EXAMPLE (.GITLAB-CI.YML)

```yaml
build-job:
  stage: build
  image: node:18-alpine
  tags:
    - docker
  before_script:
   - echo "Setting up environment..."
   - npm ci
  script:
    - npm run build
    - echo "Build completed!"
  after_script:
    - echo "Cleaning up..."
    - rm -rf node_modules
  artifacts:
    paths:
      - dist/
    expire_in: 7 days
  retry:
    max: 2
    when: runner_system_failure
  timeout: 30m
  interruptible: true
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
      when: on_success
```

EXPLAANTION

- ☑ Runs in stage: build
- ☑ Uses Node 18 Alpine image
- ☑ Runs on Docker tagged runners
- ☑ Installs dependencies before script
- ☑ Runs the build command
- ☑ Cleans up after script
- ☑ Stores dist/ as artifact for 7 days
- ☑ Retries up to 2 times on runner issues
- ☑ Times out after 30 minutes
- ☑ Interruptible to save resources
- ☑ Runs only on main branch

JOB PROPERTIES QUICK REFERENCE

KEYWORD	TYPE	DESCRIPTION	EXAMPLE

script	array	Commands that define the main steps of the job.	`script:`
			`- echo "Hello"`
			`- make build`

before_script	array	Commands run before the main script.	`before_script:` `- npm ci`
after_script	array	Commands run after the job finishes.	`after_script:` `- notify`

retry	hash/integer	Retry job if it fails.	`retry:`
			`max: 2`
			`when: runner_system_failure`

timeout	string	Maximum time allowed for the job.	`timeout: 30m`

tags	array	Runner tags required to pick the job.	`tags:`
			`- docker`
			`- linux`

interruptible	boolean	Cancel job if a new pipeline is created for the same ref.	`interruptible: true`

artifacts	hash	Files/directories to save after the job.	`artifacts:`
			`paths:`
			`- dist/`

BEST PRACTICES

- ☑ Keep jobs small and focused.
- ☑ Use before_script for setup, not inside script.
- ☑ Save artifacts only when necessary.
- ☑ Use retry selectively, not for logical failures.
- ☑ Set reasonable timeouts to avoid hanging jobs.
- ☑ Use tags to route jobs to the right runners.
- ☑ Use interruptible for long-running, replaceable jobs.

TYPICAL JOB EXECUTION FLOW

Runner Picks Job → before_script (Setup) → script (Main Steps) → after_script (Cleanup) → Artifacts (Upload) → Job Finished

If job fails → retries (if configured) → if still fails → pipeline fails

REMEMBER

- ☑ Jobs run on Runners.
- ☑ Jobs in the same stage run in parallel.
- ☑ Use artifacts to pass outputs between jobs.
- ☑ Optimize jobs for speed and reliability.

11. VARIABLES AND SECRETS

WHY VARIABLES?

Variables allow you to store values that can change between environments without hardcoding them in your code or pipeline.

KEY POINT

Never hardcode secrets like passwords, API keys, or tokens in your code or pipeline. Use variables and protect them.

TYPES OF VARIABLES IN GITLAB CI/CD

- **CI Variables** — Defined in GitLab UI or .gitlab-ci.yml
- **Protected Variables** — Available only on protected branches tags
- **Masked Variables** — Values are hidden in job logs
- **Environment Variables** — Automatically available in job environment
- **Secrets (Secure) Management** — Store and manage secrets securely (avoid plain text)

① CI VARIABLES (BASIC)

- Defined at project, group, or instance level.
- Can also be defined in .gitlab-ci.yml file.

.gitlab-ci.yml example

```
variables:
  APP_ENV: "production"
  NODE_VERSION: "18"
  API_URL: "https://api.veriqta.dev"
```

② PROTECTED VARIABLES

- Available only when pipeline runs on protected branches or tags.
- Prevents secret exposure on untrusted refs.

Use Case:
Store production database password, deploy keys, tokens, etc. Only protected branches (main, stable/*) can access them.

③ MASKED VARIABLES

- Values are masked in job logs.
- GitLab hides the value even if it appears in script output.
- Masked variables must meet requirements:
  - ☑ At least 8 characters
  - ☑ No spaces
  - ☑ Allowed characters only: a-z A-Z 0-9 _@.-
  - ☑ Cannot start with any of: CI_, GITLAB_, GIT_, VERIQTA_

④ ENVIRONMENT VARIABLES

- Provided by the runner / system automatically.
- Available in every job.
- Examples:

VARIABLE	DESCRIPTION
CI_PROJECT_DIR	Path to the project directory
CI_COMMIT_SHA	The commit SHA
CI_COMMIT_BRANCH	The branch name
CI_PIPELINE_ID	The pipeline unique ID
CI_JOB_ID	The job unique ID
CI_RUNNER_TAGS	Runner tags for this job

⑤ SECURE SECRET MANAGEMENT

- Use GitLab CI/CD variables (protected + masked).
- For large secrets, use external secret stores:
  - ☑ AWS Secrets Manager
  - ☑ HashiCorp Vault
  - ☑ Azure Key Vault
  - ☑ Google Secret Manager

BEST PRACTICES

- ☑ Never commit secrets to Git.
- ☑ Use protected + masked variables.
- ☑ Limit variable scope (project/group).
- ☑ Rotate secrets periodically.
- ☑ Audit who can access variables.

VARIABLE PRECEDENCE (HIGHEST TO LOWEST)

	LEVEL	DESCRIPTION
1	Pipeline Trigger Variables	Variables passed when triggering pipeline manually (API, UI, trigger token)
2	Job-level Variables	Defined inside a specific job in .gitlab-ci.yml
3	Pipeline-level Variables	Defined at the top level of .gitlab-ci.yml
4	Project-level Variables	Defined in project settings
5	Group-level Variables	Defined in group settings
6	Instance-level Variables	Defined in GitLab instance (settings > CI/CD)
7	Predefined Variables	Built-in GitLab variables (CI_*, GITLAB_*, etc.)

EXAMPLE PRECEDENCE

If APP_ENV exists in multiple places: Job-level value will be used.

Job-level (highest) ↑ Pipeline-level ↑ Project-level ↑ Group-level ↑ Instance-level ↑ Predefined (lowest)

VARIABLES IN ACTION (EXAMPLE)

```yaml
variables:
  APP_ENV: "production"          # Pipeline-level
  API_URL: "https://api.veriqta.dev"

deploy-prod:
  stage: deploy
  image: alpine:latest
  variables:                     # Job-level overrides
    APP_ENV: "staging"           # This value wins
  script:
    - echo "Environment: $APP_ENV"
    - echo "API: $API_URL"
```

TIPS

- ☑ Use variables for all environment-specific values.
- ☑ Keep secrets out of your repository.
- ☑ Use protected + masked for sensitive data.
- ☑ Document variables for your team.
- ☑ Test pipelines with safe dummy values before using real secrets.

COMMON MISTAKES

- ✗ Hardcoding secrets in .gitlab-ci.yml
- ✗ Using unprotected variables for production secrets
- ✗ Printing secrets in logs
- ✗ Not masking tokens or passwords
- ✗ Granting variable access to everyone

QUICK CHECKLIST

- ☑ Are all secrets stored as variables?
- ☑ Are sensitive variables protected and masked?
- ☑ Is variable scope limited?
- ☑ Are variables documented?
- ☑ Are variable values reviewed periodically?

12. GITLAB RUNNERS

WHAT IS A GITLAB RUNNER?

A GitLab Runner is an application that picks up jobs from GitLab CI/CD and executes them in an isolated environment. Runners connect to GitLab, wait for jobs, run them, and report results.

TYPES OF RUNNERS

SHARED RUNNERS

- Provided and managed by GitLab
- Available to all projects
- Easy to get started
- Limited customization
- No access to internal resources

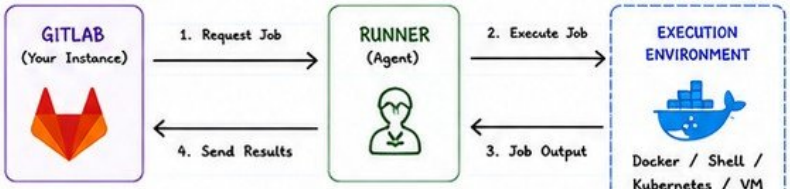
GROUP RUNNERS

- Shared within a group
- Managed by group owners
- Access to group resources
- More control and security
- Ideal for teams/organizations

PROJECT RUNNERS

- Dedicated to a single project
- Full control by project owners
- Access to private resources
- Highest security and isolation
- Best for critical workloads

RUNNER ARCHITECTURE



1. The Runner polls GitLab for new jobs.
2. When a job is available, the Runner prepares the environment.
3. The job script is executed in the configured executor.
4. Results, logs, and status are sent back to GitLab.

KEY TAKEAWAY

Runners are the workers.
GitLab is the brain.
Jobs are the tasks.

RUNNER REGISTRATION

1. Get Registration Token

Go to: Settings > CI/CD > Runners
Copy the registration token.

2. Install GitLab Runner

Linux example (Ubuntu):

```
curl -L https://gitlab-runner-downloads.s3.amazonaws.com/
latest/binaries/gitlab-runner-linux-amd64 -o gitlab-runner
chmod +x gitlab-runner
sudo mv gitlab-runner /usr/local/bin/
```

3. Register the Runner

```
sudo gitlab-runner register
--url https://gitlab.com/ # or your GitLab URL
--registration-token YOUR_TOKEN
--description "project-runner-01"
--executor docker
--tag-list "docker,linux,prod"
--run-untagged=false
--locked=false
```

4. Start the Runner

```
sudo gitlab-runner run
# or use systemd service
sudo systemctl enable --now gitlab-runner
```

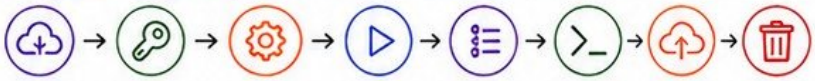
RUNNER SCOPE COMPARISON

TYPE	SCOPE	WHO CAN USE	MANAGED BY	BEST FOR	ACCESS LEVEL
Shared Runners	Instance-wide	All projects (on GitLab.com)	GitLab	Public projects, open source, learning	Limited No internal access
Group Runners	Group level	Projects in the same group	Group owners	Teams, departments, internal projects	Medium Access to group resources
Project Runners	Project level	Only that project	Project owners	Sensitive projects, private workloads, production	High Full control & isolation

RUNNER EXECUTORS (COMMON)

docker	Runs each job in a Docker container. Most popular and isolated.
shell	Runs jobs directly on the host machine. Fast but less isolated.
kubernetes	Runs jobs as pods in a Kubernetes cluster. Scalable and powerful.
virtualbox / docker+machine	Creates ephemeral VMs for each job. Good isolation, higher overhead.

RUNNER LIFECYCLE



1. **Install** — Install GitLab Runner on a server or container.
2. **Register** — Register the runner with GitLab using the token.
3. **Configure** — Set executor, tags, and other settings.
4. **Start** — Start the runner service and connect.
5. **Pick Jobs** — Runner polls GitLab and picks available jobs.
6. **Execute** — Job runs in the configured executor.
7. **Report** — Results and logs are sent back to GitLab.
8. **Cleanup** — Artifacts stored, environment cleaned up.

Runners are stateless workers. If a runner fails, another runner can pick up the job.

RUNNER CONFIGURATION FILE

```
[runners]
  name = "project-runner-01"
  url = "https://gitlab.com/"
  token = "******"
  executor = "docker"
[runners.docker]
  image = "alpine:latest"
  privileged = false
  volumes = ["/cache"]
  shm_size = 0
```

TIPS

- ☑ Use tags to route jobs to the right runners.
- ☑ Keep runners updated.
- ☑ Use autoscaling runners for dynamic workloads.
- ☑ Secure runners and restrict network access.
- ☑ Monitor runner health and performance.

BEST PRACTICES

- ☑ Use specific runners for production workloads.
- ☑ Tag runners clearly (e.g., docker, linux, prod).
- ☑ Protect runner machines and limit access.
- ☑ Use autoscaling for high availability.
- ☑ Clean runners regularly to avoid disk space issues.
- ☑ Avoid using shared runners for sensitive jobs.

COMMON MISTAKES

- ✗ Using shared runners for private data.
- ✗ Not using tags, causing jobs to run anywhere.
- ✗ Leaving outdated runners without updates.
- ✗ Allowing untrusted code on privileged runners.

13. RUNNER EXECUTORS

WHAT IS AN EXECUTOR?

An executor defines HOW the GitLab Runner runs your jobs. Different executors provide different isolation, performance, and use cases.

COMMON RUNNER EXECUTORS

1. SHELL

- Runs jobs directly on the host machine.
- Fast and lightweight.
- No isolation between jobs.
- Best for trusted, single-tenant environments.

Use Cases:
Simple jobs, build scripts, internal automation.

2. DOCKER

- Runs each job in a Docker container.
- Good isolation.
- Easy to use.
- Requires Docker installed on the runner host.

Use Cases:
Most CI/CD workloads, microservices, builds, testing.

3. KUBERNETES

- Runs jobs as pods in a Kubernetes cluster.
- Highly scalable.
- Strong isolation.
- Best for dynamic and large-scale workloads.

Use Cases:
Enterprise CI/CD, large teams, cloud-native platforms.

4. SSH

- Executes jobs on remote machines over SSH.
- No containers by default.
- Good for legacy systems.
- Requires SSH access configuration.

Use Cases:
Deployments to bare metal, legacy servers, remote environments.

5. VIRTUAL MACHINE

- Runs jobs in full virtual machines.
- Maximum isolation.
- Heavier and slower than containers.
- Supports snapshots.

Use Cases:
Deployments, compliance requirements, Windows workloads.

6. AUTOSCALING EXECUTORS

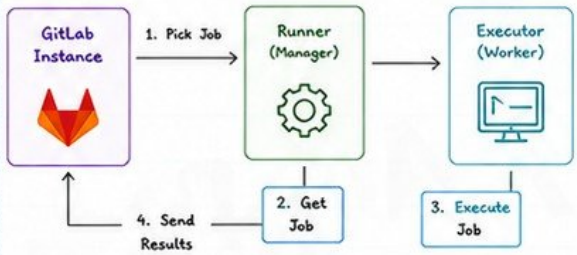
- Runners scale automatically based on job load.
- Can use Docker, Kubernetes, or VM under the hood.
- Reduces idle cost.
- Ideal for bursty workloads.

Use Cases:
Dynamic workloads, cost optimization, cloud environments.

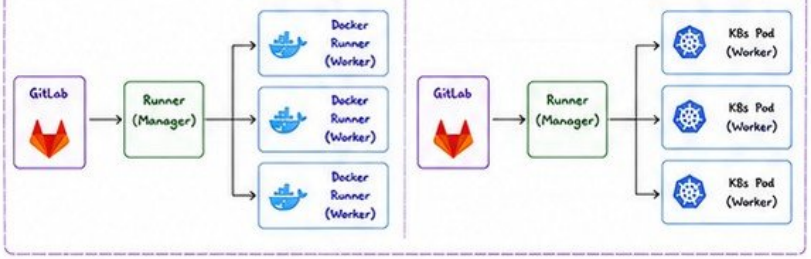
EXECUTOR COMPARISON

EXECUTOR	ISOLATION	PERFORMANCE	SCALABILITY	SETUP COMPLEXITY	BEST FOR
Shell	Low	Very Fast	Low	Very Easy	Trusted environments, simple jobs
Docker	Medium	Fast	Medium	Easy	Most CI/CD pipelines
Kubernestes	High	Fast	Very High	Medium/Hard	Enterprise, large scale, cloud-native
SSH	Low	Medium	Low	Medium	Legacy systems, remote deployments
Virtual Machine	Very High	Slow	Medium	Hard	High security, compliance, Windows builds
Autoscaling (Any)	High	Fast	Very High	Medium/Hard	Dynamic scaling, burst workloads, cloud

RUNNER ARCHITECTURE (WITH EXECUTOR)



AUTOSCALING EXECUTOR EXAMPLES



EXECUTOR SELECTION GUIDE

HOW TO CHOOSE THE RIGHT EXECUTOR?

- ☑ Need maximum speed and trust the machine? → Shell
- ☑ Using containers and want isolation? → Docker
- ☑ Running in Kubernetes already? → Kubernetes
- ☑ Deploying to remote servers? → SSH
- ☑ Need full OS isolation or compliance? → VM
- ☑ Need automatic scaling and cost optimization? → Autoscaling

KEY POINTS

- ☑ Executors define the environment where jobs run.
- ☑ Choose based on isolation, performance, scalability, and cost.
- ☑ Docker is the default choice for most teams.
- ☑ Kubernetes executor is best for cloud-native and scale.
- ☑ Autoscaling executors save cost by scaling on demand.
- ☑ Always monitor runner health and resource usage.
- ☑ Use tags to route jobs to the right executor.

EXECUTOR SELECTION IN .GITLAB-CI.YML

```yaml
job-build:
  stage: build
  script:
    - echo "Building application"
  tags:
    - docker
    - linux
```

- tags: [docker, linux] → Job will run on a runner with BOTH tags.
- Use tags to target specific executors.
- Each runner can have multiple tags.

BEST PRACTICES

- ☑ Use specific executors for different types of workloads.
- ☑ Keep runners up to date with the latest versions.
- ☑ Isolate sensitive workloads on dedicated runners.
- ☑ Use autoscaling to handle peak loads efficiently.
- ☑ Monitor, log, and alert on runner performance.
- ☑ Secure your runners and limit who can register them.

14. Pipeline Execution Flow

Overview

A GitLab CI/CD pipeline goes through a well-defined flow from creation to completion. Understanding this flow helps you troubleshoot faster and design reliable pipelines.

KEY TAKEAWAY

Pipelines are created by events.
Jobs run on Runners.
Status changes show real-time state.
Monitoring + logs = faster debugging.

Pipeline Execution Flow (High Level)

1. **Pipeline Creation** — Pipeline is created by an event (push, MR, tag, schedule, API).
2. **Job Scheduling** — Jobs are added to the pipeline and scheduled for execution.
3. **Runner Assignment** — A suitable runner is picked based on tags, executor, and availability.
4. **Queue Processing** — Job waits in queue until the runner is ready to pick it up.
5. **Job Execution** — Runner executes the job script in isolated environment.
6. **Status & Completion** — Job finishes and reports status. Pipeline updates in real-time.

Detailed Flow Explanation

1 Pipeline Creation

- Triggered by: push, merge request, tag, schedule, manual run, API.
- .gitlab-ci.yml is read and validated.
- Pipeline object is created in GitLab.

2 Job Scheduling

- Stages are read from .gitlab-ci.yml.
- Jobs are created and added to their respective stages.
- DAG (needs) are evaluated.
- Jobs with dependencies are ordered.

3 Runner Assignment

- GitLab looks for available runners.
- Runner must match job tags and be online.
- Runner capabilities (executor, arch) are validated.
- Job is assigned to the selected runner.

4 Queue Processing

- If no runner is available, job goes to queue.
- Job waits until a runner is free.
- Queue is FIFO per runner.
- Timeouts and concurrency limits apply.

5 Job Execution

- Runner picks job from queue.
- Environment is prepared (executor starts container/VM/etc).
- before_script → script → after_script run.
- Artifacts and cache are uploaded.
- Exit code determines job status.

6 Status Lifecycle

- Job status is reported back to GitLab.
- Stages update in real-time.
- Pipeline status = worst status of all jobs.
- Notifications, webhooks, and approvals triggered.

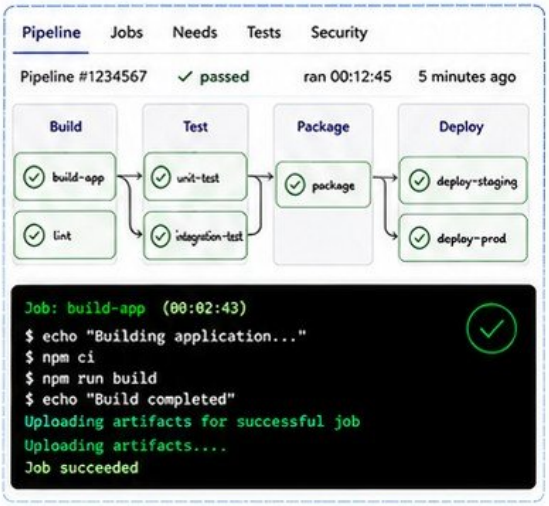
Status Lifecycle

- **created** — Job created
- **pending** — Waiting for runner
- **running** — Runner executing job
- **success** — Job completed successfully
- **failed** — Job failed (exit code ≠ 0)
- **canceled** — Job canceled by user/system
- **skipped** — Job skipped (by rules/only)
- **manual** — Waiting for manual play
- **scheduled** — Scheduled for future time
- **blocked** — Blocked by approval
- **stuck** — Runner lost or no heartbeat

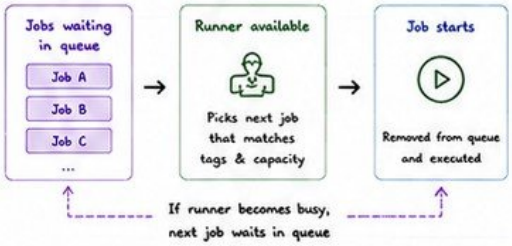
NOTE: A pipeline is successful only when all required jobs succeed.
If any job fails, the pipeline status becomes failed.

Execution Monitoring

- **Pipeline Graph** — Visual view of stages, jobs, and dependencies.
- **Job Logs** — Real-time logs from runner (stdout/stderr).
- **Job Trace** — Detailed step-by-step command output.
- **Metrics** — Track duration, success rate, runner usage, queue time.
- **Alerts & Notifications** — Slack, Email, Webhooks, PagerDuty integrations.



Queuing Behavior



TIPS

- ☑ Use tags to route jobs to the right runners.
- ☑ Keep runner concurrency balanced.
- ☑ Use needs to reduce pipeline duration.
- ☑ Monitor queue time regularly.
- ☑ Use retries for flaky jobs.

BEST PRACTICES

- ☑ Design small, focused jobs.
- ☑ Use caching and artifacts to optimize performance.
- ☑ Set timeouts and retry policies.
- ☑ Monitor runner health and capacity.
- ☑ Alert on long queue times or failing pipelines.

15. ARTIFACTS

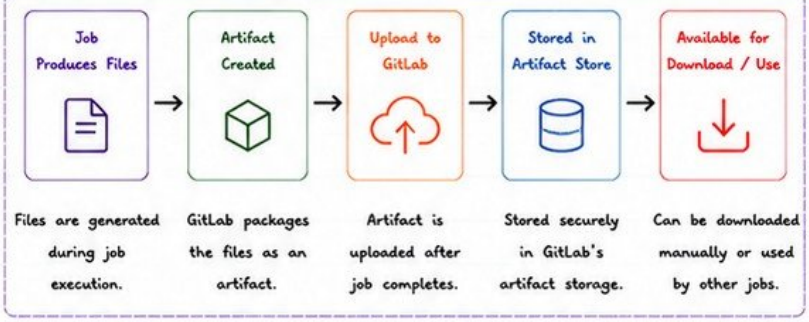
KEY TAKEAWAY

- ☑ Artifacts persist data between jobs and stages.
- ☑ They enable traceability, sharing, and reusability.
- ☑ Use retention policies to control storage and cost.
- ☑ Keep artifacts small, relevant, and secure.

WHAT ARE ARTIFACTS?

Artifacts are files and directories generated during a job that are uploaded to GitLab and made available for download or use in downstream jobs and stages.

ARTIFACT UPLOAD FLOW



ARTIFACT CREATION (EXAMPLE)

.gitlab-ci.yml

```yaml
build:
  stage: build
  script:
    - npm ci
    - npm run build
  artifacts:
    name: "build-$CI_COMMIT_SHORT_SHA"
    paths:
      - dist/
      - package.json
      - package-lock.json
    expire_in: 7 days
    when: always
```

Define files/ folders to keep

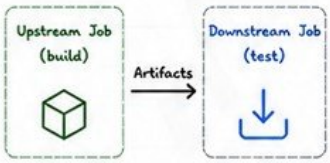
How long to keep artifact

Upload even if job fails (optional)

ARTIFACT DOWNLOAD & USAGE

Using artifacts in downstream jobs:

```yaml
test:
  stage: test
  needs: ["build"]
  script:
    - npm test
dependencies:
  - build    # downloads artifacts from build job
```



Artifacts are automatically downloaded before the job starts.

ARTIFACT RETENTION

- Controlled using **expire_in** in .gitlab-ci.yml
- Default is 30 days if not specified (instance setting)
- Can be kept forever with: **expire_in: never**
- Retention applies after successful upload

EXAMPLES	
expire_in	Retention Period
1 day	Artifacts deleted after 1 day
7 days	Artifacts deleted after 7 days
30 days	Artifacts deleted after 30 days
never	Artifacts kept forever (manual cleanup)

REPORTS (SPECIAL ARTIFACTS)

GitLab understands certain reports and shows them in the UI.

REPORT TYPE	PURPOSE	EXAMPLE
JUnit	Test results	junit: report.xml
Coverage	Code coverage	coverage: coverage.xml
Code Quality	Code quality issues	codequality: gl-code-quality.json
SAST	Security scan results	sast: gl-sast-report.json
Dependency Scanning	Vulnerability in deps	dependency_scanning: gl-dependency-scanning-report.json
Container Scanning	Container image scan	container_scanning: gl-container-scanning-report.json
License Compliance	License checks	license_scanning: gl-license-scanning-report.json

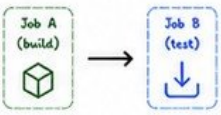
TIP

Reports are artifacts but displayed beautifully in GitLab UI with insights and trends.

SHARING ARTIFACTS

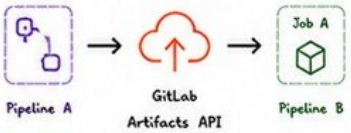
Between Jobs (Same Pipeline)

- Use dependencies or needs
- Ensures correct order
- Only required artifacts are downloaded



Across Pipelines

- Use the GitLab API to download artifacts
- Or use the "Download" button in UI
- Use project access tokens for automation



BEST PRACTICES

- ☑ Only store what you need.
- ☑ Keep artifacts small to save storage and bandwidth.
- ☑ Set appropriate expire_in values.
- ☑ Use artifacts for build outputs, not source code.
- ☑ Never store secrets in artifacts.
- ☑ Use reports for better visibility and insights.
- ☑ Clean up expired/unused artifacts regularly.
- ☑ Use needs to reduce unnecessary downloads.

COMMON MISTAKES

- ✗ Storing large or unnecessary files.
- ✗ Setting expire_in: never without a reason.
- ✗ Forgetting to protect sensitive data.
- ✗ Not using needs/dependencies (downloads everything).
- ✗ Not cleaning up old artifacts (storage overflow).

16. PIPELINE CACHING

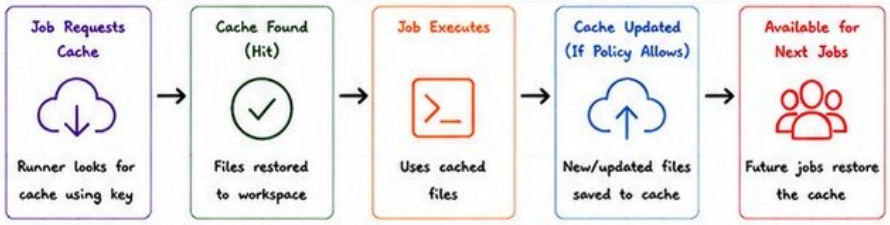
KEY TAKEAWAY

- ☑ Caching avoids re-downloading and rebuilding dependencies.
- ☑ Good cache strategy = faster pipelines + lower CI/CD cost.
- ☑ Cache smartly, invalidate intentionally.

WHAT IS PIPELINE CACHING?

Pipeline caching stores files and directories between jobs and pipelines so they can be reused. It reduces network usage, build time, and improves overall pipeline performance.

CACHE FLOW



CACHE STRATEGY

- Cache dependencies that don't change often.
- Use specific and stable keys.
- Restore cache early in the job.
- Update cache only when needed.
- Avoid caching large or unnecessary files.

EXAMPLE: NODE.JS DEPENDENCY CACHE

.gitlab-ci.yml

```yaml
install:
  stage: install
  image: node:20
  cache:
    key: "$CI_PROJECT_NAME-node-$CI_COMMIT_REF_SLUG"
    paths:
      - node_modules/
    policy: pull-push
  script:
    - npm ci
```

- Unique key per branch
- Cache the node_modules folder
- Pull cache before job, push after job
- Install using cached deps

CACHE KEYS

KEY TYPE	EXAMPLE	WHEN TO USE	DESCRIPTION
Static Key	"node-cache"	Simple projects	Same key for all pipelines (not recommended)
Branch Key	"node-$CI_COMMIT_REF_SLUG"	Per branch	Isolates cache per branch
File Based	"node-$CI_PROJECT_ID-$CI_COMMIT_REF_SLUG" (with files: [package-lock.json])	Dependencies change	Changes when lock file changes (best practice)
Fallback Key	key: "$CI_COMMIT_REF_SLUG" fallback_keys: - "main-node-cache"	Shared fallback	Use default cache if branch cache missing
Composite Key Key	"${CI_PROJECT_NAME-${CI_JOB_NAME--${CI_COMMIT_REF_SLUG}"	Complex projects	Unique per job and branch

CACHE POLICIES

POLICY	BEHAVIOR	USE CASE
pull	Download cache if available (never upload)	Read-only caches, speed up without updating
push	Upload cache (ignore existing)	Publish cache for later jobs
pull-push	Download if exists, then upload after job	Most common policy
pull-if-not-exists	Download only if cache exists, do not upload	Prevent overwriting existing cache

DEPENDENCY CACHING EXAMPLES

Node.js	Python	Java (Maven)	Java (Gradle)	Go	.NET
Cache node_modules/ Key based on package-lock.json	Cache .cache/pip/ Key based on requirements.txt	Cache .m2/repository/ Key based on pom.xml	Cache .gradle/ Key based on build.gradle	Cache $GOPATH/pkg/mod Key based on go.sum	Cache packages/ .nuget/ Key based on *.csproj

PERFORMANCE IMPROVEMENTS

- ☑ Reduces dependency download time by 60% - 90%.
- ☑ Speeds up builds, tests, and deployments.
- ☑ Less network traffic and external service load.
- ☑ Lower CI/CD minutes and infrastructure cost.
- ☑ Consistent and faster developer feedback loop.

CACHE TROUBLESHOOTING

ISSUE	POSSIBLE CAUSE	HOW TO FIX
Cache not found	Wrong key, different branch, cache expired	Check key, fallback_keys, and retention
Cache not uploaded	Policy set to pull or push skipped	Use pull-push or push policy
Large cache size	Too many files cached	Cache only necessary paths, exclude big folders
Stale / incorrect cache	Cache not invalidated	Use file-based keys, clear cache
Pipeline still slow	Cache path wrong or not restored early	Verify paths, restore in earlier stage

BEST PRACTICES

- ☑ Cache only what is necessary.
- ☑ Use file-based keys for automatic invalidation.
- ☑ Keep cache size small and relevant.
- ☑ Use pull-push for most use cases.
- ☑ Avoid caching build outputs—cache dependencies only.
- ☑ Regularly review and clean unused caches.

TIP

Use lock files (like package-lock.json, pom.xml, go.sum, requirements.txt) in your cache key for best results.

COMMON MISTAKES

- ✗ Using the same static key for all pipelines.
- ✗ Caching too much (build artifacts, logs, binaries).
- ✗ Not using fallback keys.
- ✗ Forgetting to update cache when dependencies change.
- ✗ Relying on cache without validating correctness.

17. PIPELINE RULES

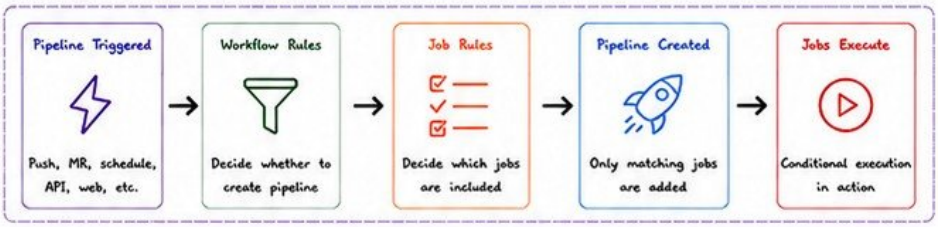
KEY TAKEAWAY

- ☑ Rules control WHEN jobs and pipelines run.
- ☑ Use rules for flexibility and future-proofing.
- ☑ only/except are legacy but still useful.
- ☑ Good rules = faster pipelines + lower cost.

☆ WHAT ARE PIPELINE RULES?

Rules define when a job is included in a pipeline. They allow conditional execution based on variables, branches, events, file changes, and more.

☆ PIPELINE EVALUATION FLOW



☆ RULE TYPES

1. rules (RECOMMENDED)

- Most powerful and flexible.
- Supports conditions, variables, changes, when, allow_failure, etc.
- Evaluated top-to-bottom.

2. only (LEGACY)

- Includes a job only in specific refs or events.
- Limited conditions.
- Avoid mixing with rules.

3. except (LEGACY)

- Excludes a job from specific refs or events.
- Limited conditions.
- Avoid mixing with rules.

☆ EXAMPLES

A. Using rules (Recommended)

```yaml
build-job:
  stage: build
  script: echo "Building..."
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
      when: on_success
    - if: '$CI_COMMIT_TAG'
      when: on_success
    - when: never
```

Run on main branch
Run on tags
Else never run

B. Using only (Legacy)

```yaml
test-job:
  stage: test
  script: npm test
  only:
    - main
    - merge_requests
    - tags
```

Run only on main, MR and tags

C. Using except (Legacy)

```yaml
deploy-job:
  stage: deploy
  script: ./deploy.sh
  except:
    - branches
    - tags
```

Do not run on branches or tags

☆ WORKFLOW RULES

Control whether a pipeline is created at all.

```yaml
workflow:
  rules:
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
      when: always
    - if: '$CI_COMMIT_BRANCH == "main"'
      when: always
    - when: never
```

If none of the workflow rules match, the pipeline will NOT be created.

☆ CONDITIONAL EXECUTION EXAMPLES

CONDITION	DESCRIPTION
if: '$CI_COMMIT_BRANCH == "main"'	Run on main branch
if: '$CI_COMMIT_TAG'	Run on tags
if: '$CI_PIPELINE_SOURCE == "schedule"'	Run on scheduled pipelines
if: '$CI_MERGE_REQUEST_ID'	Run in Merge Request pipelines
changes: - "src/**/*"	Run only if files changed
exists: - Dockerfile	Run only if file exists
when: manual	Manual trigger
when: never	Never run

☆ DYNAMIC PIPELINES

Rules + variables allow dynamic behavior and multi-pipeline strategies.

1. Child Pipelines

Trigger child pipeline based on rules. Great for microservices or mono-repos.

2. Matrix Pipelines

Use variables + rules to create multiple job variations.

3. Environment Based

Run jobs only for specific environments (dev, staging, prod).

☆ PRACTICAL EXAMPLE: CONDITIONAL DEPLOY

```yaml
deploy:
  stage: deploy
  script: ./deploy.sh
  environment:
    name: production
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
      when: manual
      allow_failure: false
    - if: '$CI_COMMIT_TAG'
      when: manual
    - when: never
```

Run on main (manual)
Run on tags (manual)
Never in other cases

☆ RULES EVALUATION ORDER

1. Rules are evaluated top to bottom.
2. First matching rule is applied.
3. If no rule matches → job is excluded.
4. Avoid overlapping or conflicting rules.

TIP

Always end with
- when: never
to prevent accidental execution.

☆ BEST PRACTICES

- ☑ Use rules instead of only/except.
- ☑ Keep rules simple and readable.
- ☑ Use variables for flexibility.
- ☑ Avoid duplicate pipelines with workflow rules.
- ☑ Test rules using CI Lint.
- ☑ Document complex rules.

☆ COMMON MISTAKES

- ✕ Mixing rules with only/except.
- ✕ Forgetting the default "when".
- ✕ Overlapping rules causing unexpected runs.
- ✕ Not testing rules with CI Lint.
- ✕ Creating duplicate pipelines.

18. BRANCH-BASED PIPELINES

KEY TAKEAWAY

- ☑ Different branches have different purposes.
- ☑ Pipelines should adapt to branch type.
- ☑ Protect what matters. Automate the rest.
- ☑ Fast feedback on feature branches.
- ☑ Safe, reliable deployments from main.

WHAT IS BRANCH-BASED PIPELINES?

Branch-based pipelines run automatically when code is pushed to specific branches. Each branch type has a defined workflow, rules, and deployment strategy.

BRANCH OVERVIEW

FEATURE BRANCHES

Used for new features and changes.
Short-lived.
Created from main.
Merged back via MR.

RELEASE BRANCHES

Stabilization branch for a release.
Testing, QA and bug fixes before release.

MAIN BRANCH

Production-ready code.
Always deployable.
Source of truth.
Protected and stable.

HOTFIX BRANCHES

Urgent fixes for production issues.
Created from main.
Protected and quickly.

PROTECTED BRANCHES

Branches with rules that prevent direct pushes and enforce quality and approvals.

PIPELINE BEHAVIOR BY BRANCH TYPE

BRANCH TYPE	TRIGGER	PIPELINE TYPE	DEPLOYMENT	PURPOSE
Feature Branches	Push / MR	Branch Pipeline (Fast feedback)	No deployment (or Review App)	Build, Test, Lint, Security
Release Branches	Push	Branch Pipeline (Full validation)	Staging / Pre-Prod	Integration testing, QA, UAT
Main Branch	Push / Merge MR	Branch Pipeline (Production)	Production	Reliable production deployments
Hotfix Branches	Push	Branch Pipeline (Fast track)	Production (Fast deploy)	Quick fixes, minimal delay
Protected Branches	Push blocked	N/A	N/A	Enforce process and safety

PIPELINE FLOW BY BRANCH TYPE

PROTECTED BRANCH SETTINGS (EXAMPLE)

- ☑ Require Merge Requests
- ☑ Require code owner approval
- ☑ Require status checks to pass
- ☑ Require up-to-date branch
- ☑ Restrict who can push
- ☑ Prevent force pushes
- ☑ Prevent branch deletion

EXAMPLE PROTECTED BRANCHES

main
release/*
production
critical/*

EXAMPLE: .GITLAB-CI.YML RULES BY BRANCH

```yaml
# Feature branches
feature-pipeline:
  stage: test
  rules:
    - if: '$CI_COMMIT_BRANCH == /^feature\//'
      when: always
    - when: never

# Release branches
release-pipeline:
  stage: test
  rules:
    - if: '$CI_COMMIT_BRANCH == /^release\//'
      when: always

# Main branch
production-pipeline:
  stage: deploy
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
      when: always

# Hotfix branches
hotfix-pipeline:
  stage: deploy
  rules:
    - if: '$CI_COMMIT_BRANCH == /^hotfix\//'
      when: always
```

Run jobs only on matching branch types

BEST PRACTICES

- ☑ Keep feature branches short-lived.
- ☑ Use Merge Requests for all changes.
- ☑ Protect main and release branches.
- ☑ Run fast pipelines on feature branches.
- ☑ Deploy only from main or release.
- ☑ Use environments for deployments.
- ☑ Automate checks: test, lint, security.
- ☑ Monitor pipeline success and failures.

COMMON MISTAKES

- ✗ Pushing directly to main.
- ✗ Long-lived feature branches.
- ✗ No branch protection rules.
- ✗ Deploying from feature branches.
- ✗ Skipping tests for hotfixes.
- ✗ Not cleaning up old branches.

TIP

A good branching model + smart pipelines = Faster delivery, higher quality, and fewer production issues.

PIPELINE BEHAVIOR SUMMARY

19. MERGE REQUEST PIPELINES

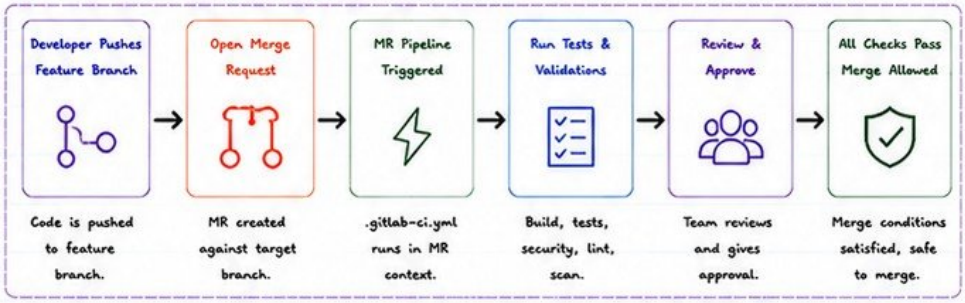
KEY TAKEAWAY

- ☑ MR pipelines run automatically for every Merge Request.
- ☑ They validate changes before merging.
- ☑ Catch issues early and improve code quality.
- ☑ Integrate reviews, approvals, and checks.
- ☑ Safe merges lead to stable production.

WHAT ARE MERGE REQUEST PIPELINES?

Merge Request (MR) pipelines run when a developer opens or updates a merge request. They test and validate the proposed changes without affecting the target branch until the MR is merged.

MERGE REQUEST PIPELINE FLOW FLOW



MR PIPELINES (CI CONTEXT)

- Runs in isolated environment.
- Uses source branch code.
- Tests changes before they enter the target branch.
- Can access target branch for comparison.
- Does not deploy to production.

VALIDATION IN MR PIPELINES

- ☑ Code compilation / build
- ☑ Unit tests
- ☑ Integration tests
- ☑ Static code analysis (SAST)
- ☑ Security scans & dependency checks
- ☑ Code quality & linting
- ☑ Infrastructure validation (IaC)
- ☑ Docker image build (optional)
- ☑ Preview environment deployment (optional)

TIP

Fail fast, give clear feedback. MRs should be small, tested, and reviewable.

EXAMPLE: .GITLAB-CI.YML (MR PIPELINE)

```yaml
stages:
  - build
  - test
  - security
  - review

variables:
  GIT_DEPTH: "50"

workflow:
  rules:
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
      when: always
    - when: never

build:
  stage: build
  script:
    - echo "Building application..."
    - npm ci
    - npm run build

test:
  stage: test
  script:
    - npm test -- --coverage

lint:
  stage: test
  script:
    - npm run lint

sast:
  stage: security
  script:
    - echo "Running SAST scan..."
  allow_failure: false

review:
  stage: review
  script:
    - echo "MR pipeline completed successfully"
  rules:
    - if: '$CI_MERGE_REQUEST_ID'
      when: always
```

Pipeline runs ONLY for Merge Requests

CODE REVIEW AUTOMATION

- **Automated Test Results** — Post pipeline results as MR widgets and comments.
- **Code Quality Reports** — SonarQube, CodeClimate, or built-in reports.
- **Security Scan Results** — SAST/DAST findings right inside the MR.
- **Inline Code Discussion** — Leave comments on specific lines of code.

APPROVAL INTEGRATION

- Require approvals before merge.
- CODEOWNERS auto-assign reviewers.
- Protect critical paths with approval rules.
- Integrate with Slack / Email / Teams notifications.

MERGE CHECKS (MUST PASS BEFORE MERGE)

CHECK TYPE	DESCRIPTION
✓ Pipeline Success	All required jobs passed.
✓ Required Approvals	All mandatory approvals granted.
✓ Status Checks	All status checks are successful.
✓ Code Quality Gate	Quality gate passed (SonarQube, etc).
✓ Security Gate	No critical/high vulnerabilities.
✓ Merge Conflicts	No merge conflicts with target branch.
✓ Branch Rules	Branch protection rules satisfied.

BEST PRACTICES

- ☑ Keep MRs small and focused.
- ☑ Run full test suite in MR pipelines.
- ☑ Use meaningful MR titles and descriptions.
- ☑ Automate as many checks as possible.
- ☑ Provide fast feedback to developers.
- ☑ Don't allow merge on failing pipelines.

COMMON MISTAKES

- ✗ Skipping tests in MR pipelines.
- ✗ Allowing merge without approvals.
- ✗ Ignoring flaky tests.
- ✗ MRs that are too large and hard to review.
- ✗ Not using branch protection rules.

MR PIPELINE BEHAVIOR SUMMARY

Trigger on MR + Run Validations + Review & Approve + All Checks Pass = Merge Allowed

20. PARENT AND CHILD PIPELINES

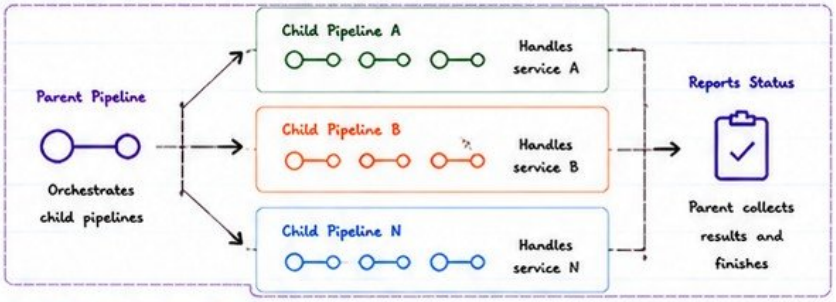
KEY TAKEAWAY

- ☑ Parent pipelines orchestrate high-level flow.
- ☑ Child pipelines handle detailed workloads.
- ☑ Dynamic child pipelines create pipelines at runtime.
- ☑ Perfect for monorepos and large organizations.
- ☑ Improves scalability, maintainability and speed.

WHAT ARE PARENT AND CHILD PIPELINES?

Parent-child pipelines allow you to break a large workflow into smaller pipelines. The parent pipeline triggers child pipelines and waits for them to complete. This enables modular, scalable and reusable CI/CD.

PIPELINE HIERARCHY OVERVIEW



1. PARENT PIPELINES

- Define the high-level stages and flow.
- Trigger one or more child pipelines.
- Collect results from children.
- Control overall success or failure.
- Best for orchestration and governance.

2. CHILD PIPELINES

- Execute specific components or services.
- Have their own .gitlab-ci.yml.
- Run independently but are controlled by the parent.
- Can be reused across projects.

3. DYNAMIC CHILD PIPELINES

- Child pipelines are generated at runtime.
- Configuration is created dynamically.
- Ideal when you don't know the exact pipelines until execution.
- Great for monorepos and scalable systems.

PIPELINE EXECUTION FLOW

EXAMPLE: PARENT PIPELINE

```yaml
# .gitlab-ci.yml (Parent)
stages:
  - validate
  - build
  - child_pipelines
  - deploy

validate:
  stage: validate
  script:
  - echo "Validating repository..."

build:
  stage: build
  script:
  - echo "Building shared artifacts..."

child_pipelines:
  stage: child_pipelines
  trigger:
    include:
      - local: services/service-a/.gitlab-ci.yml
      - local: services/service-b/.gitlab-ci.yml
    strategy: depend

deploy:
  stage: deploy
  script:
  - echo "Deploying after all children succeed..."
```

EXAMPLE: CHILD PIPELINE (service-a/.gitlab-ci.yml)

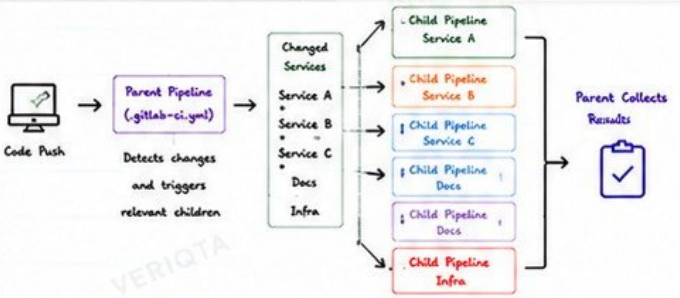
```yaml
stages:
  - test
  - build
  - package

test:
  stage: test
  script:
    - npm test

build:
  stage: build
  script:
    - npm run build

package:
  stage: package
  script:
    - echo "Packaging Service A..."
```

TIP

Use strategy: depend so the parent waits for all child pipelines to finish and uses their success or failure.

ℹ If any child pipeline fails, the parent pipeline will also fail (when using strategy: depend).

MONOREPO WORKFLOW EXAMPLE



ENTERPRISE USE CASES

- **Large Monorepos** — Run pipelines only for changed components.
- **Microservices Architecture** — Each service has its own child pipeline.
- **Multi-Environment Deployments** — Different environments triggered as children.
- **Compliance & Security** — Run security scans, QA gates and approvals in isolated child pipelines.
- **Team Isolation** — Teams own their pipelines, parent orchestrates.

BENEFITS

- ☑ Scalable and modular
- ☑ Faster feedback
- ☑ Better parallelization
- ☑ Reusable components
- ☑ Simpler maintenance
- ☑ Clear separation of concerns
- ☑ Enterprise-ready governance

BEST PRACTICES

- ☑ Keep parent pipeline lightweight.
- ☑ Use includes to organize child configs.
- ☑ Use strategy: depend for result aggregation.
- ☑ Pass variables explicitly to children.
- ☑ Use rules to trigger only when needed.
- ☑ Version control child pipeline configs.

COMMON MISTAKES

- ✗ Triggering all child pipelines when not needed.
- ✗ Not using strategy: depend (losing visibility).
- ✗ Deep nesting (more than 2-3 levels).
- ✗ Hardcoding paths (breaks portability).
- ✗ Not handling failures in child pipelines.

QUICK REFERENCE

trigger: include	→ Create child pipelines
strategy: depend	→ Wait for child results
rules	→ Control when to trigger
variables	→ Pass data to children
needs:pipeline	→ Use artifacts from children

21. Multi-Project Pipelines

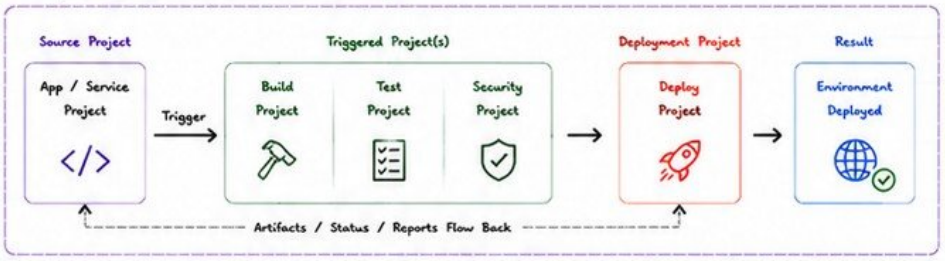
KEY TAKEAWAY

- ☑ Multi-project pipelines connect projects together.
- ☑ Pipelines can trigger or depend on other projects.
- ☑ Enables enterprise-wide automation and reuse.
- ☑ Promotes loose coupling and team autonomy.
- ☑ Essential for microservices and platform teams.

WHAT ARE MULTI-PROJECT PIPELINES?

Multi-project pipelines allow a pipeline in one project to trigger, depend on, or consume artifacts from pipelines in other projects.
This enables cross-team workflows, shared CI/CD, and enterprise automation.

MULTI-PROJECT PIPELINE OVERVIEW



MULTI-PROJECT CAPABILITIES

Trigger Pipelines
Start a pipeline in another project from your pipeline.

Pipeline Dependencies
Wait for another project's pipeline to complete.

Consume Artifacts
Download artifacts produced by other project pipelines.

Status Propagation
Upstream pipeline reflects downstream success or failure.

Cross-Project Variables
Pass variables, context and parameters between projects.

CROSS-PROJECT PIPELINE FLOW

1. **Code Push in Source Project**
   Developer pushes code to service repository.
2. **Trigger Downstream Pipeline**
   Source pipeline triggers build/test project.
3. **Build / Test / Scan**
   Downstream project builds, tests, scans.
4. **Artifacts / Reports**
   Artifacts and reports are stored and returned.
5. **Deployment Pipeline**
   Deployment project consumes artifacts and deploys.
6. **Status Returned**
   Results are propagated back to source project.

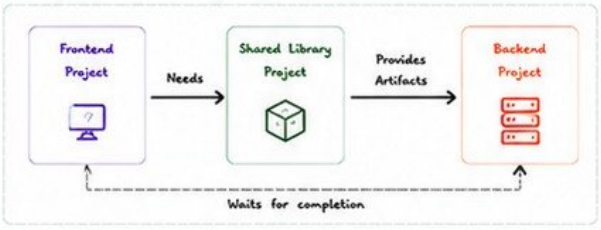
EXAMPLE: TRIGGER ANOTHER PROJECT

```yaml
# .gitlab-ci.yml (in app-service project)
stages:
  - build
  - trigger-tests
  - deploy

build:
  stage: build
  script:
  - echo "Building application..."
  artifacts:
    paths:
      - dist/

trigger-tests:
  stage: trigger-tests
  trigger:
    project: my-group/qa-tests      # project to trigger
    branch: main
    strategy: depend                # wait for completion

trigger-deploy:
  stage: deploy
  trigger:
    project: my-group/deployments   # deployment project
    branch: main
    strategy: depend
```

TIP

strategy: depend makes the upstream pipeline wait and reflect the downstream pipeline status.

PIPELINE DEPENDENCY EXAMPLE

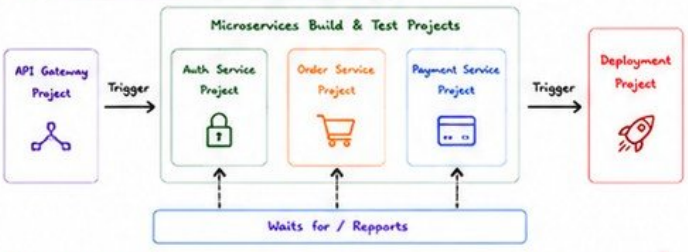


EXAMPLE: USE ARTIFACTS FROM ANOTHER PROJECT

```yaml
use-shared-library:
  stage: build
  needs:
    - project: my-group/shared-library
      job: build-lib
      ref: main
      artifacts: true       # download artifacts
  script:
    - echo "Using library artifacts..."
    - ls -la lib/
  dependencies:
    - my-group/shared-library
```

needs + artifacts: true allows downloading artifacts from another project's job.

MICROSERVICES ORCHESTRATION EXAMPLE



ENTERPRISE USE CASES

- **Centralized Build Project** — Standardise build environment and tooling.
- **Security Scanning Project** — Run security scans once, share results.
- **Environment Promotion Project** — Promote artifacts across environments.
- **Shared Libraries & Components** — Build once, consume everywhere.
- **Compliance & Audit Project** — Generate reports and compliance evidence.

BEST PRACTICES

- ☑ Keep projects loosely coupled.
- ☑ Use strategy: depend for reliability.
- ☑ Pass only required variables.
- ☑ Secure cross-project access with least privilege.
- ☑ Version artifacts and libraries.
- ☑ Monitor pipeline duration and failure points.

COMMON MISTAKES

- ✗ Creating circular triggers (A triggers B, B triggers A).
- ✗ Not using strategy: depend and missing failures.
- ✗ Over-sharing artifacts unnecessarily.
- ✗ Passing too many variables.
- ✗ Hardcoding project names instead of using variables.

TIP

Use CI/CD variables for project paths and branches:
$PROJECT_PATH
$TARGET_BRANCH

QUICK REFERENCE COMMANDS

```
Trigger pipeline via API:
 curl --request POST --form token=TOKEN \
    --form ref=main \
   https://gitlab.com/api/v4/projects/PROJECT_ID/trigger/pipeline
Get pipeline status:
 curl --header "PRIVATE-TOKEN: TOKEN" \
    https://gitlab.com/api/v4/projects/PROJECT_ID/pipelines/PIPELINE_ID
```

22. Docker Integration

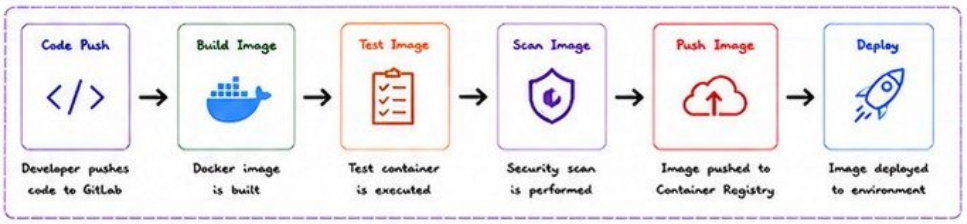
KEY TAKEAWAY

- ☑ Docker integration enables reproducible builds and consistent deployments.
- ☑ Use Multi-stage builds to create small, secure and fast images.
- ☑ BuildKit dramatically improves build speed and caching.
- ☑ Follow production practices for secure, optimized and reliable containers.

WHAT IS DOCKER INTEGRATION?

GitLab CI/CD can build, test, scan and push Docker images as part of your pipelines. Docker ensures consistency from development to production.

DOCKER PIPELINE FLOW



DOCKER BUILDS

- Build images inside CI/CD pipelines.
- Tag images with commit SHA, branch or version.
- Push images to GitLab Container Registry or external registries (ECR, GCR, ACR, Docker Hub).
- Enable caching to speed up builds.

EXAMPLE: DOCKER BUILD IN GITLAB CI

```yaml
build-image:
  stage: build
  image: docker:24
  services:
    - docker:24-dind
  variables:
    DOCKER_TLS_CERTDIR: ""
  script:
    - echo "Logging in..."
    - echo "$CI_REGISTRY_PASSWORD" | docker login
      -u "$CI_REGISTRY_USER" --password-stdin $CI_REGISTRY
    - echo "Building image..."
    - docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA .
    - docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
  rules:
    - if: '$CI_COMMIT_BRANCH'
      when: on_success
```

TIP

Always tag images with immutable identifiers like commit SHA for traceability.

DOCKER-IN-DOCKER (DIND)

- Run Docker daemon inside a container.
- Required for building images in CI/CD.
- Use privileged mode or rootless mode for better security.
- Disable TLS for simplicity inside CI (or configure securely).

SECURITY NOTE

DIND with privileged mode has security risks. Use dedicated runners or rootless mode when possible.

BUILDKIT

- Modern build engine for Docker.
- Faster builds, parallel execution and better caching.
- Supports advanced features like mount cache, secrets and inline cache.
- Enable using DOCKER_BUILDKIT=1.

ENABLE BUILDKIT IN CI

```yaml
variables:
  DOCKER_BUILDKIT: "1"
```

BENEFITS

- ☑ Up to 50% faster builds
- ☑ Better caching
- ☑ Reduced network usage
- ☑ Smaller build contexts
- ☑ Supports advanced features

MULTI-STAGE BUILDS

- Use multiple stages in Dockerfile.
- Keep build tools in build stage, only runtime files in final stage.
- Results in smaller, secure and fast images.

Dockerfile (Multi-stage example)

```dockerfile
# Stage 1: Build
FROM node:20 AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Stage 2: Runtime
FROM node:20-alpine
WORKDIR /app
COPY --from=build /app/dist ./dist
COPY package*.json ./
RUN npm ci --omit=dev
CMD ["node", "dist=server.js"]
```

OUTCOME

- ☑ Smaller image size
- ☑ No build tools in production image
- ☑ Faster pull & deploy
- ☑ Better security posture

CONTAINER OPTIMIZATION

- Use minimal base images (alpine, distroless, etc.)
- Use .dockerignore to exclude unnecessary files.
- Leverage layer caching (order Dockerfile instructions from least to most changing).
- Clean up packages and cache in same layer.
- Run as non-root user.
- Use multi-stage builds.
- Scan images for vulnerabilities.

PRODUCTION PRACTICES

- ☑ Tag images with version + SHA (e.g., 1.2.3, latest, $CI_COMMIT_SHA)
- ☑ Use immutable tags in deployments.
- ☑ Enable image scanning (Trivy, Snyk, etc.).
- ☑ Sign images (Cosign / Notary v2).
- ☑ Store images in private registry.
- ☑ Set resource limits (CPU/Memory).
- ☑ Use HEALTHCHECK in Dockerfile.
- ☑ Keep base images up to date.
- ☑ Follow least privilege principle.

EXAMPLE PIPELINE: BUILD, TEST, PUSH

```yaml
stages:
  - build
  - test
  - push

variables:
  DOCKER_BUILDKIT: "1"
  DOCKER_TLS_CERTDIR: ""

build:
  stage: build
  image: docker:24
  services:
    - docker:24-dind
  script:
    - docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA .
  artifacts:
    reports:
      dotenv: build.env     # contains IMAGE_TAG
  after_script:
    - echo "IMAGE_TAG=$CI_REGISTRY_IMAGE:$CI_COMMIT_SHA" > build.env
```

```yaml
test:
  stage: test
  image: $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
  script:
    - npm test
    - npm run lint
```

```yaml
push:
  stage: push
  image: docker:24
  services:
    - docker:24-dind
  needs: ["build"]
  script:
    - echo "$CI_REGISTRY_PASSWORD" | docker login
      -u "$CI_REGISTRY_USER" --password-stdin $CI_REGISTRY
    - docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
      when: on_success
```

COMMON MISTAKES

- ✗ Using latest tag in production deploys.
- ✗ Building huge images with unnecessary files.
- ✗ Not using .dockerignore.
- ✗ Running containers as root.
- ✗ Pushing images without scanning.
- ✗ Storing secrets in Docker images.

23. GitLab Container Registry

KEY TAKEAWAY

- [x] GitLab Container Registry stores and manages Docker images for your projects.
- [x] Built-in, secure, and deeply integrated with GitLab CI/CD.
- [x] Supports versioning, access control, scanning, and cleanup.
- [x] Essential for reproducible, secure and scalable container delivery.

WHAT IS GITLAB CONTAINER REGISTRY?

GitLab Container Registry is a built-in, private registry that stores your Docker container images. It is tightly integrated with projects, CI/CD pipelines, permissions, and security features.

REGISTRY ARCHITECTURE

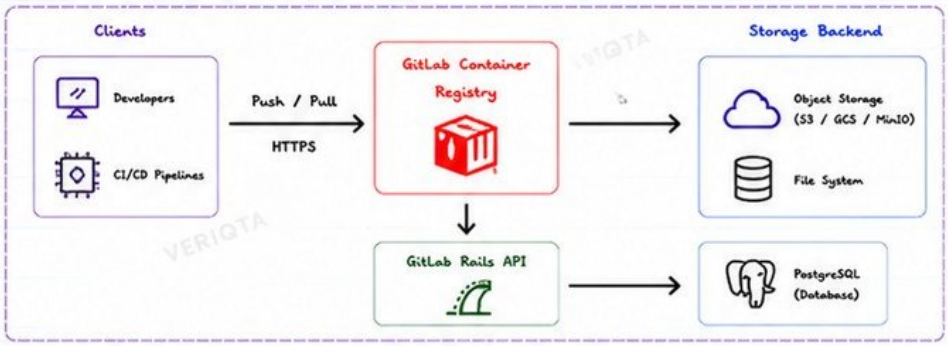


IMAGE STORAGE

	Images are stored as layers (blobs) using the OCI/Docker image format.
	Layers are deduplicated to save storage across images.
	Manifests reference layers and metadata.
	Storage can be local disk or external object storage.
	Highly available when configured with object storage.

AUTHENTICATION

	Uses GitLab user credentials or Deploy Tokens.
	Supports Personal Access Tokens (PAT) and Project Access Tokens.
	Access is controlled via GitLab permissions and groups.
	Supports 2FA, SSO, LDAP and SAML for enterprise.
	All connections use HTTPS/TLS for secure communication.

VERSIONING

	Images are versioned using tags (e.g., v1.0.0, latest).
	Can also use branch-based or commit SHA tags.
	Immutable tags ensure reproducible deployments.
	Easy rollback to any previous image version.
	Track image history and metadata in GitLab UI.

CLEANUP

	Remove untagged images to free storage.
	Use retention policies to keep images for a defined period.
	Delete old or unused tags safely.
	Monitor registry storage usage regularly.
	Automate cleanup with CI jobs or scheduled maintenance.

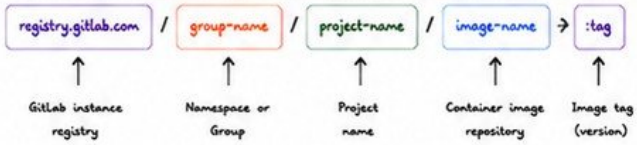
SECURITY SCANNING

	Built-in Vulnerability Scanning for container images.
	Scans OS packages and application dependencies.
	Shows severity levels: Critical, High, Medium, Low, Unknown.
	Scan results appear in GitLab UI and CI pipelines.
	Fail pipeline if vulnerabilities exceed defined threshold.

EXAMPLE: PUSH IMAGE TO GITLAB REGISTRY

```
# Login to GitLab Registry
$ echo "$CI_REGISTRY_PASSWORD" | \
  docker login $CI_REGISTRY -u "$CI_REGISTRY_USER" --password-stdin

# Build image
$ docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA .

# Tag image
$ docker tag $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA \
  $CI_REGISTRY_IMAGE:latest

# Push image
$ docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
$ docker push $CI_REGISTRY_IMAGE:latest
```

REGISTRY URL STRUCTURE



BEST PRACTICES

- [x] Use semantic version tags.
- [x] Always scan images in pipelines.
- [x] Enforce least-privilege access.
- [x] Use immutable tags in production.
- [x] Clean up old images regularly.
- [x] Store images in external object storage for scalability.

COMMON MISTAKES

- [x] Using only 'latest' tag.
- [x] Not cleaning up old images.
- [x] Storing secrets in images.
- [x] Pushing large unnecessary layers.
- [x] Not scanning images.
- [x] Over-permissive registry access.

TYPICAL REGISTRY WORKFLOW

REGISTRY STORAGE CONSIDERATIONS

- [x] Use object storage for HA and scalability.
- [x] Enable lifecycle rules for cost optimization.
- [x] Monitor storage growth and plan capacity.
- [x] Enable deduplication to reduce costs.

24. PACKAGE REGISTRY

KEY TAKEAWAY

- [x] GitLab Package Registry stores and manages any type of package.
- [x] Supports many ecosystems and tools.
- [x] Keeps dependencies close to your code, secure and versioned.
- [x] Improves build reliability and speeds up development.

WHAT IS PACKAGE REGISTRY?

GitLab Package Registry is a feature that allows you to publish, store, and manage packages for different package types and ecosystems. It integrates with CI/CD, permissions, and dependency management.

PACKAGE REGISTRY ARCHITECTURE

SUPPORTED PACKAGE TYPES

MAVEN	Java libraries and dependencies.	maven/gradle projects.
NPM	JavaScript packages and frontend dependencies.	Node.js applications.
PYPI	Python packages and libraries.	Python applications and tools.
NUGET	.NET packages and libraries.	.NET applications.
GENERIC PACKAGES	Any package type (archives, binaries, etc).	Zip, tar, Debian, RPM, Helm charts, CLI tools.

PACKAGE REGISTRY URL PATTERN

```
https://gitlab.com/api/v4/projects/<project-id>/packages/<package-type>/
```

Base URL · Project ID · Package Type

TIP

Use CI_JOB_TOKEN or Deploy Tokens for secure access from pipelines.

AUTHENTICATION

- CI_JOB_TOKEN (recommended for CI/CD).
- Personal Access Token (PAT) with api scope.
- Deploy Tokens for cross-project access.
- Read / Write permissions controlled per project or group.

BENEFITS

- [x] Private and secure storage.
- [x] Integrated with GitLab CI/CD.
- [x] Versioned and immutable packages.
- [x] Faster builds and reliable deploys.
- [x] Centralized dependency management.

PACKAGE MANAGER CONFIGURATION EXAMPLES

MAVEN (settings.xml)

```
<servers>
  <server>
    <id>gitlab-maven</id>
    <username>${CI_JOB_USER}</username>
    <password>${CI_JOB_TOKEN}</password>
  </server>
</servers>

<mirrors>
  <mirror>
    <id>gitlab</id>
    <mirrorOf>*</mirrorOf>
    <url>https://gitlab.com/api/v4/
    projects/${CI_PROJECT_ID}/packages/maven</url>
  </mirror>
</mirrors>
```

NPM (.npmrc)

```
//gitlab.com/api/v4/projects/
${CI_PROJECT_ID}/packages/npm/:_authToken=${CI_JOB_TOKEN}

registry=https://gitlab.com/api/v4/projects/
${CI_PROJECT_ID}/packages/npm/
```

Install

```
$ npm install my-package
```

Publish

```
$ npm publish --registry=https://gitlab.com/api/v4/
  projects/${CI_PROJECT_ID}/packages/npm/
```

PYPI (pip.conf)

```
[global]
index-url = https://gitlab.com/api/v4/
projects/${CI_PROJECT_ID}/packages/pypi/simple

[install]
trusted-host = gitlab.com
```

Install

```
$ pip install my-package
```

Publish

```
$ twine upload --repository gitlab
  dist/*
```

NUGET (nuget.config)

```
<configuration>
  <packageSources>
    <add key="GitLab"
      value="https://gitlab.com/api/v4/projects/
      ${CI_PROJECT_ID}/packages/nuget/index.json" />
  </packageSources>
</configuration>
```

Install

```
$ dotnet add package MyPackage --source GitLab
```

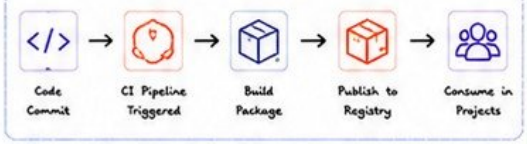
GENERIC PACKAGE (curl)

```
# Upload
curl --header "JOB-TOKEN: ${CI_JOB_TOKEN}" \
  --upload-file my-tool.tar.gz \
  "https://gitlab.com/api/v4/projects/
  ${CI_PROJECT_ID}/packages/generic/
  my-tool/1.0.0/my-tool.tar.gz"

# Download
curl --header "JOB-TOKEN: ${CI_JOB_TOKEN}" \
  --output my-tool.tar.gz \
  "https://gitlab.com/api/v4/projects/
  ${CI_PROJECT_ID}/packages/generic/
  my-tool/1.0.0/my-tool.tar.gz"
```

DEPENDENCY MANAGEMENT

- Centralize internal libraries and tools.
- Version control for all dependencies.
- Use immutable versions (no moving tags).
- Promote packages across environments.
- Automate cleanup of old versions.

EXAMPLE: CI PUBLISH FLOW



Example .gitlab-ci.yml (Publish Maven Package)

```
stages:
  - build
  - publish

publish-maven:
  stage: publish
  image: maven:3.9-eclipse-temurin-17
  script:
    - mvn -B clean deploy \
      -DaltDeploymentRepository=gitlab::default::
      https://gitlab.com/api/v4/projects/${CI_PROJECT_ID}/packages/maven
  only:
    - main
  variables:
    MAVEN_OPTS: "-Dmaven.repo.local=.m2/repository"
```

CLEANUP POLICIES

ACTION	DESCRIPTION
Keep Latest Versions	Keep only the latest N versions of each package.
Age Based Cleanup	Remove packages older than X days.
Remove Untagged	Delete Untagged without versions or tags.
Automate via CI	Use scheduled pipelines or API for cleanup.

COMMON MISTAKES

- ✗ Pushing without versioning.
- ✗ Using latest tag (mutable).
- ✗ Not authenticating in CI.
- ✗ Over-permissive access.
- ✗ Not cleaning up old packages.

SECURITY SCANNING

GitLab scans packages for known vulnerabilities.
Works with Dependency Scanning and Container Scanning.
View results in GitLab UI under Security > Packages.
Block vulnerable dependencies using policies.

25. Testing Pipelines

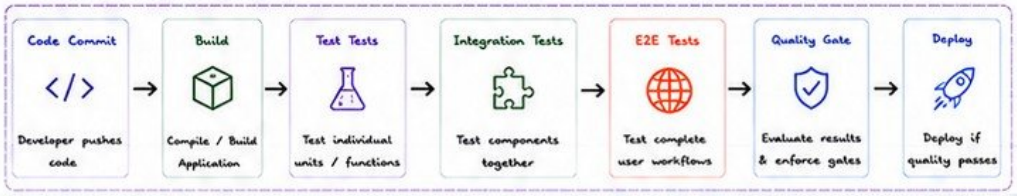
KEY TAKEAWAY

- ☑ Automated testing in pipelines ensures code quality, reliability and safety.
- ☑ Test early, test often, test at every stage.
- ☑ Fast feedback = happy developers.
- ☑ Quality gates stop bad code from reaching production.

☆ WHAT IS TESTING IN PIPELINES?

Testing pipelines automate the execution of different types of tests at every stage of CI/CD. They validate functionality, performance and security before code is merged or deployed.

☆ TESTING PIPELINE FLOW



1. UNIT TESTING

- Test individual functions or classes in isolation.
- Fast execution and easy to debug.
- Run on every commit.

Example (Jest - JavaScript)

```
npm test -- --ci --reporters=default --reporters=jest-junit
# Output: junit.xml
```

2. INTEGRATION TESTING

- Validate interactions between modules, services or databases.
- Ensures components work correctly together.

Example (Integration Test)

```
npm run test:integration -- --reporters=jest-junit
# Output: junit-integration.xml
```

3. END-TO-END TESTING

- Simulate real user scenarios across the system.
- Runs against deployed or ephemeral environments.

Example (Playwright - E2E)

```
npx playwright test --reporter=junit
# Output: junit-e2e.xml
```

4. TEST REPORTS

- Publish test results for visibility and tracking.
- Use standard formats (JUnit, XML) for consistency.
- Store reports as pipeline artifacts.

Example Report Summary

Stage	Total	Passed	Failed	Skipped	Success %
Unit Tests	125	123	2	0	98.4%
Integration	45	44	1	0	97.8%
E2E Tests	32	30	2	0	93.8%
Overall	202	197	5	0	97.5%

Reports available in GitLab UI and artifacts.

5. PARALLEL TESTING

- Run tests in parallel to reduce pipeline time.
- Split tests by files, modules or suites.
- Use tags and parallel matrix.

Example (.gitlab-ci.yml)

```
unit_tests:
  stage: test
  parallel:
    matrix:
      - NODE_VERSION: ["18", "20"]
      - TEST_SUITE: ["unit-a", "unit-b"]
  script:
    - npm ci
    - npm run test -- --suite $TEST_SUITE
```

6. QUALITY GATES

- Define rules that code must pass before progressing.
- Fail the pipeline if quality thresholds are not met.
- Common gates: coverage, test pass rate, code quality.

Example Quality Gates

Gate	Metric	Threshold	Action
Test Pass Rate	% Passed	>= 95%	Fail Pipeline
Code Coverage	% Coverage	>= 80%	Fail Pipeline
Critical Issues	Security	0	Fail Pipeline
Performance	Avg Response	<= 200ms	Fail Pipeline

TIP

Keep unit tests fast, integration tests reliable, and E2E tests focused. Good tests + good pipelines = production confidence.

☆ EXAMPLE: .GITLAB-CI.YML (TESTING PIPELINE)

```
stages:
  - build
  - test
  - quality
  - deploy

variables:
  NODE_ENV: test

build:
  stage: build
  image: node:20-alpine
  script:
    - npm ci
    - npm run build

unit_test:
  stage: test
  image: node:20-alpine
  script:
    - npm ci
    - npm test -- --ci --reporters=default
        --reporters=jest-junit
  artifacts:
    reports:
      junit: junit-unit.xml
    expire_in: 1 week

integration_test:
  stage: test
  image: node:20-alpine
  services:
    - name: postgres:15
      alias: db
  script:
    - npm run test:integration -- --reporters=jest-junit
  artifacts:
    reports:
      junit: junit-integration.xml
    expire_in: 1 week

e2e_test:
  stage: test
  image: mcr.microsoft.com/playwright:v1.45-focal
  script:
    - npx playwright install --with-deps
    - npx playwright test --reporter=junit
  artifacts:
    reports:
      junit: junit-e2e.xml
    expire_in: 1 week

quality_gate:
  stage: quality
  image: registry.gitlab.com/gitlab-org/security-products/analyzers:latest
  - echo "Evaluating quality gates..."
  - |
    passed=true
    # Example checks (replace with real tools)
    # 1. Test Pass Rate
    pass_rate=$(./scripts/calc-pass-rate.sh)
    if [ "$pass_rate" -lt 95 ]; then
      echo "Test pass rate $pass_rate% is below 95%"
      passed=false
    # 2. Coverage
    if [ "$coverage" -lt 80 ]; then
      echo "Coverage $coverage% is below 80%"
      passed=false
    fi
    if [ "$passed" != true ]; then
      exit 1
    fi
needs:
  - stage: deploy
  - ./scripts/deploy.sh
needs:
needs: - quality_gate
  - if: '$CI_COMMIT_BRANCH == "main"'
```

☆ BEST PRACTICES

- ☑ Write small, fast, reliable unit tests.
- ☑ Use meaningful test data and isolated environments.
- ☑ Run tests in parallel where possible.
- ☑ Fail fast and provide clear feedback.
- ☑ Publish and review test reports regularly.
- ☑ Keep E2E tests minimal and critical.

☆ COMMON MISTAKES

- ✕ Skipping tests to save time.
- ✕ Slow or flaky tests causing false failures.
- ✕ No quality gates or weak thresholds.
- ✕ Not publishing test results.
- ✕ E2E tests covering everything (too many).

☆ TESTING TOOLS EXAMPLES

Jest / Mocha	Unit Testing (JS)
JUnit / TestNG	Unit Testing (Java)
PyTest	Unit & Integration (Python)
Playwright / Cypress	End-to-End Testing
SonarQube	Code Quality & Coverage
OWASP ZAP	Security Testing
Snyk	Dependency Scanning

26. Code Quality

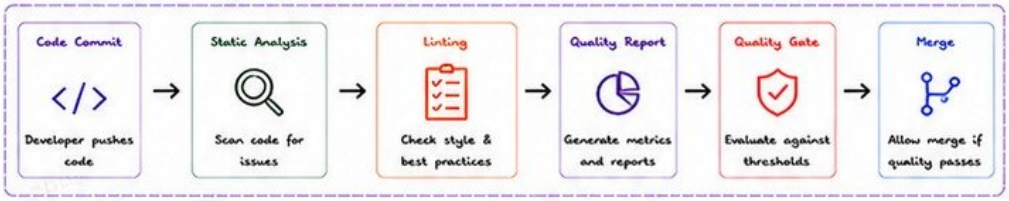
KEY TAKEAWAY

- [x] Code quality is a non-functional requirement.
- [x] Static analysis and linting catch issues early.
- [x] Quality reports give visibility and drive improvement.
- [x] Reduce technical debt to lower future risk.
- [x] Merge quality gates ensure only high-quality code reaches main.

WHAT IS CODE QUALITY?

Code quality ensures that code is maintainable, reliable, secure and consistent with standards. GitLab CI/CD integrates quality tools to analyze code, measure issues and enforce quality gates.

CODE QUALITY PIPELINE OVERVIEW



1. STATIC ANALYSIS

- Analyzes code without executing it.
- Finds bugs, vulnerabilities, code smells, security issues and complexity problems.
- Examples: SonarQube, CodeClimate, Semgrep, Bandit.

Example: SonarQube scan

```
sonarqube-check:
  stage: quality
  image: sonarsource/sonar-scanner-cli:latest
  script:
    - sonar-scanner \
      -Dsonar.projectKey=$CI_PROJECT_PATH_SLUG \
      -Dsonar.sources=. \
      -Dsonar.host.url=$SONAR_HOST_URL \
      -Dsonar.login=$SONAR_TOKEN
  only:
    - merge_requests
    - branches
```

2. LINTING

- Enforces coding standards and style consistency.
- Catches formatting issues and anti-patterns.
- Examples: ESLint, Pylint, Flake8, RuboCop, ShellCheck.

Example: ESLint (Node.js)

```
eslint:
  stage: lint
  image: node:20-alpine
  before_script:
    - npm ci
  script:
    - npx eslint . --format junit -o eslint-report.xml
  artifacts:
    when: always
    reports:
      junit: eslint-report.xml
      codequality: gl-code-quality-report.json
    paths:
      - eslint-report.xml
      - gl-code-quality-report.json
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
```

3. QUALITY REPORTS

- Unified view of issues, coverage, code smells, vulnerabilities and duplications.
- Reports available in GitLab MR widgets and dashboards.
- Formats: JUnit, Cobertura, CodeClimate, SonarQube, Code Quality JSON.

Example: Coverage Report

Metric	Value	Target	Status
Coverage	87.3%	>= 80%	✓
Bugs	12	<= 20	✓
Vulnerabilities	3	<= 5	✓
Code Smells	45	<= 50	✓
Duplications	2.1%	<= 3%	✓

4. TECHNICAL DEBT

- Measures the cost of future rework caused by poor code quality.
- Tracked in minutes/hours/days to fix.
- Reduce technical debt continuously.

Technical Debt Indicators

- Code Smells
- Duplicated Code
- Complexity
- Long Methods / Classes
- Outdated Dependencies
- Poor Test Coverage

5. MERGE QUALITY GATES

- Prevent low-quality code from being merged.
- Fail the pipeline if thresholds are not met.
- Gates can block merge in Merge Requests.

Example Quality Gate Policy

Metric	Threshold	Action on Fail
Coverage	>= 80%	Block Merge
Bugs	= 0 Critical, < 10 Major	Block Merge
Vulnerabilities	= 0 Critical, < 5 High	Block Merge
Code Smells	< 100	Warn / Block
Duplications	< 3%	Block Merge
Technical Debt	< 1 Day	Warn
Lint Errors	= 0	Block Merge

TIP

Start with warn mode, then move to block mode for critical metrics. Improve quality gradually and keep the bar high.

BEST PRACTICES

- [x] Run static analysis and linting on every MR.
- [x] Fail fast: catch issues early.
- [x] Set realistic but strict quality thresholds.
- [x] Monitor trends, not just snapshots.
- [x] Fix root causes, not just symptoms.
- [x] Keep dependencies updated and secure.

COMMON MISTAKES

- [x] Ignoring warnings and code smells.
- [x] Setting thresholds too low.
- [x] Allowing critical issues in production.
- [x] No visibility into quality trends.
- [x] Not treating technical debt seriously.

EXAMPLE: COMPLETE QUALITY PIPELINE (.gitlab-ci.yml)

```
stages:
  - lint
  - test
  - static_analysis
  - quality_gate
lint:
  stage: lint
  image: node:20-alpine
  script:
    - npm ci
    - npx eslint . --format junit -o eslint.xml
  artifacts:
    reports:
      junit: eslint.xml
      codequality: gl-code-quality-report.json
test:
  stage: test
  image: node:20-alpine
  script:
    - npm ci
    - npm test -- --coverage --ci
  artifacts:
    reports:
      junit: junit.xml
      coverage_report:
        coverage_format: cobertura
        path: coverage/cobertura-coverage.xml
static_analysis:
  stage: static_analysis
  image: sonarsource/sonar-scanner-cli:latest
  script:
    - sonar-scanner -Dsonar.login=$SONAR_TOKEN
  only:
    - merge_requests
    - branches
quality_gate:
  stage: quality_gate
  image: curlimages/curl:8.6.0
  script:
    - echo "Checking Quality Gate..."
    - curl -s -u $SONAR_TOKEN: "${SONAR_HOST_URL}/api/qualitygates/
      project_status?projectKey=${CI_PROJECT_PATH_SLUG}"
      -o result.json
    - status=$(grep -o '"status":"[^"]*"' result.json | cut -d'"' -f4)
    - if [ "$status" != "OK" ]; then echo "Quality Gate Failed"; exit 1; fi
    - echo "Quality Gate Passed"
  only:
    - merge_requests
    - branches
```

TOOLS YOU CAN USE

- SonarQube
- CodeClimate
- Semgrep
- ESLint
- Pylint
- Bandit
- Gitleaks
- ShellCheck
- Stylelint
- Depcheck / Snyk

27. SECURITY SCANNING

KEY TAKEAWAY

- [x] Security scanning finds issues before attackers do.
- [x] Shift security left and automate scans in every pipeline.
- [x] Combine multiple scan types for complete protection.
- [x] Block risky code and dependencies before they reach production.
- [x] Security is everyone's responsibility.

WHAT IS SECURITY SCANNING?

GitLab integrates multiple security scanners into CI/CD to identify vulnerabilities, misconfigurations, secrets, and compliance issues early in the development lifecycle.

SECURITY SCANNING PIPELINE OVERVIEW

1. SAST (STATIC APPLICATION SECURITY TESTING)

- Analyzes source code for security vulnerabilities.
- Finds issues like SQLi, XSS, insecure API usage, buffer overflows, etc.

Example (SAST - GitLab Semgrep)

```yaml
sast:
  stage: test
  image: registry.gitlab.com/gitlab-org/security-products/sast:latest
  variables:
    SAST_EXCLUDED_PATHS: "node_modules,vendor"
  artifacts:
    reports:
      sast: gl-sast-report.json
```

2. DAST (DYNAMIC APPLICATION SECURITY TESTING)

- Tests running applications for vulnerabilities.
- Finds issues like auth bypass, SQLi, XSS, CSRF, insecure configurations.

Example (DAST)

```yaml
dast:
  stage: test
  image: registry.gitlab.com/gitlab-org/security-products/sast:latest
  variables:
    DAST_FULL_SCAN_ENABLED: "true"
    DAST_WEBSITE: "https://veriqta-app-staging.veriqta.com"
  artifacts:
    reports:
      dast: gl-dast-report.json
```

3. DEPENDENCY SCANNING

- Scans project dependencies and lock files.
- Finds known vulnerabilities (CVEs) in libraries and packages.

Example (Dependency Scanning)

```yaml
dependency_scanning:
  stage: test
  image: registry.gitlab.com/gitlab-org/security-products/
         dependency-scanning:latest
artifacts:
  reports:
    dependency_scanning: gl-dependency-scanning-report.json
```

4. SECRET DETECTION

- Detects hardcoded secrets, tokens, keys, passwords in code.
- Works on commits and merge requests.

Example (Secret Detection)

```yaml
secret_detection:
  stage: test
  image: registry.gitlab.com/gitlab-org/security-products/
    secret-detection:latest
  variables:
    GIT_DEPTH: "0"
  artifacts:
    reports:
      secret_detection: gl-secret-detection-report.json
```

5. CONTAINER SCANNING

- Scans container images for OS packages and known vulnerabilities.
- Supports Trivy based scanning.

Example (Container Scanning)

```yaml
container_scanning:
  stage: test
  image: aquasec/trivy:latest
  variables:
    CS_IMAGE: "$CI_REGISTRY_IMAGE:$CI_COMMIT_SHA"
  script:
    - trivy image --format template --template
      "@contrib/gitlab.tpl" -o gl-container-scanning-report.json
      $CS_IMAGE
  artifacts:
    reports:
      container_scanning: gl-container-scanning-report.json
```

6. LICENSE COMPLIANCE

- Checks open source licenses for compliance with policy.
- Ensures allowed and approved licenses only.

Example (License Compliance)

```yaml
license_scanning:
  stage: test
  image: registry.gitlab.com/gitlab-org/security-products/
    license-scanning:latest
  artifacts:
    reports:
      license_scanning: gl-license-scanning-report.json
```

SECURITY REPORTS IN GITLAB

Report	Purpose	Output	Location
SAST	Source code vulnerabilities	gl-sast-report.json	Security > SAST
DAST	Running app vulnerabilities	gl-dast-report.json	Security > DAST
Dependency Scanning	Vulnerable dependencies	gl-dependency-scanning-report.json	Security > Dependency List
Secret Detection	Exposed secrets & tokens	gl-secret-detection-report.json	Security > Secret Detection
Container Scanning	Vulnerabilities in images	gl-container-scanning-report.json	Security > Container Scanning
License Compliance	License policy violations	gl-license-scanning-report.json	Security > License Compliance

QUALITY GATE EXAMPLE

Gate	Condition	Action
Critical Vulnerabilities	> 0	Fail Pipeline
High Vulnerabilities	> 0	Fail Pipeline
License Violations	> 0	Fail Pipeline
Secrets Detected	> 0	Fail Pipeline
Coverage	< 80%	Fail Pipeline
Code Quality Score	< 70	Fail Pipeline

TIP

Start with visibility, then enforce gates. Fix critical issues first, then raise the bar gradually.

EXAMPLE: COMPLETE SECURITY STAGE (.gitlab-ci.yml)

```yaml
stages:
  - build
  - test
  - security
  - deploy

security:
  stage: security
  needs: ["build"]
  parallel:
    matrix:
      - SCAN: [sast, dast, dependency_scanning, secret_detection,
               container_scanning, license_scanning]
  script:
    - echo "Running $SCAN"
  rules:
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event" ||
      $CI_COMMIT_BRANCH == "main"'
```

```yaml
include:
  - template: Security/SAST.gitlab-ci.yml
  - template: Security/DAST.gitlab-ci.yml
  - template: Security/Dependency-Scanning.gitlab-ci.yml
  - template: Security/Secret-Detection.gitlab-ci.yml
  - template: Security/Container-Scanning.gitlab-ci.yml
  - template: Security/License-Scanning.gitlab-ci.yml

# Quality Gate
security_quality_gate:
  stage: security
  needs: ["sast", "dast", "dependency_scanning", "container_scanning",
          "secret_detection", "license_scanning"]
  image: alpine:3.19
  script:
    - apk add --no-cache curl jq
    - echo "Evaluating security reports..."
    - ./scripts/quality-gate.sh
  rules:
    - when: always
```

BEST PRACTICES

- [x] Run security scans on every MR and main branch.
- [x] Fail fast on critical and high severity issues.
- [x] Keep dependencies updated regularly.
- [x] Store secrets in Vault or CI/CD variables.
- [x] Review reports and fix root causes.

COMMON MISTAKES

- [x] Ignoring security scan results.
- [x] Allowing vulnerabilities to merge.
- [x] Storing secrets in repository.
- [x] Not updating dependencies.
- [x] Disabling security gates.

TOOLS POWERED BY GITLAB

SAST	Semgrep, GoSec, Brakeman
DAST	OWASP ZAP
Dependencies	Gemnasium, npm audit, pip-audit
Secrets	Gitleaks
Containers	Trivy
Licenses	FOSSA, License-Scanning

28. COMPLIANCE PIPELINES

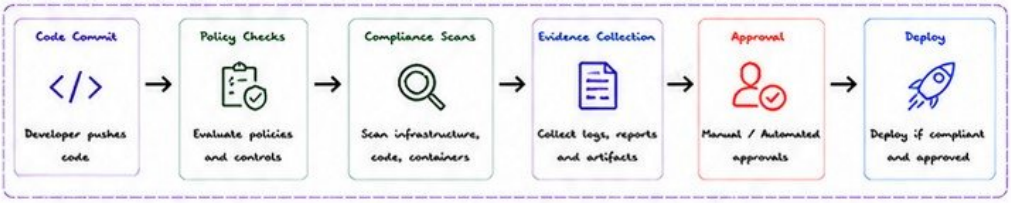
KEY TAKEAWAY

- ☑ Compliance pipelines ensure we meet industry, legal and organizational standards automatically.
- ☑ Automate policy checks, evidence collection and approvals.
- ☑ Build audit-ready systems with traceability and governance.
- ☑ Compliance is built in, not bolted on.

WHAT IS COMPLIANCE PIPELINE?

Compliance pipelines automate and enforce compliance requirements throughout the DevOps lifecycle. They ensure that code, infrastructure, containers and deployments meet regulatory, security and organizational policies before reaching production.

COMPLIANCE PIPELINE FLOW



1. COMPLIANCE FRAMEWORKS

SOC 2	Security, availability, processing integrity, confidentiality, privacy.
ISO 27001	Information security management system (ISMS).
PCI DSS	Payment card industry data security standard.
HIPAA	Health data privacy and security requirements.
GDPR	Data protection and privacy regulation.
NIST CSF	Identify, Protect, Detect, Respond, Recover framework.

2. POLICY ENFORCEMENT

- Enforce policies as code using OPA, Conftest, or GitLab Policies.
- Block deployments that violate security or compliance rules.
- Use predefined or custom compliance rulesets.
- Fail fast to prevent non-compliant changes.

Example: OPA Policy Check

```
opa_policy_check:
  stage: policy
  image: openpolicyagent/opa:latest
  script:
    - opa eval --input build/plan.json \
      --data policies/ \
      "data.compliance.deny == {}"
  artifacts:
    reports:
      sast: opa-report.json
  allow_failure: false
```

3. AUDIT READINESS

- Collect evidence automatically at every stage.
- Store artifacts securely for audit trails.
- Maintain traceability from commit to deployment.
- Generate compliance reports on demand.

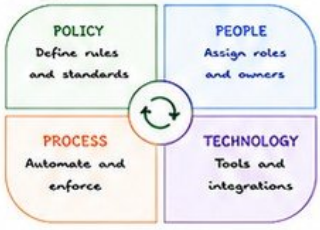
Example Evidence Collected

- Pipeline logs and job outputs
- Security scan reports
- Infrastructure as Code (IaC) plans
- Approval records and comments
- Container image digests and SBOMs
- Policy evaluation results

4. GOVERNANCE

- Define compliance policies and standards.
- Assign responsibilities and ownership.
- Maintain risk register and exceptions.
- Review and update controls regularly.

Governance Model



5. APPROVAL WORKFLOWS

- Use manual approvals for high-risk changes.
- Implement role-based approvals.
- Require multiple approvers for sensitive environments.
- Record all approvals for audit trails.

Example Approval Workflow

Stage	Who Approves	Condition	Action
Policy Check	CI System	Policy must pass	Auto Approve
Security Scan	Security Team	No Critical/High	Auto Approve
Compliance Gate	Compliance Owner	All controls pass	Manual Approve
Production Deploy	Release Manager	All approvals done	Deploy

TIP

Define clear approval rules and use Least Privilege for approvers. Always record approvals and reasons.

COMPLIANCE GATES EXAMPLES

Gate	Description	On Fail
Policy Gate	All policies must pass	Fail Pipeline
Security Gate	No Critical/High vulnerabilities	Fail Pipeline
License Gate	Only approved licenses	Fail Pipeline
Secrets Gate	No secrets in code or configs	Fail Pipeline
Compliance Score	Score >= 80%	Fail Pipeline

BEST PRACTICES

- ☑ Shift compliance left - catch issues early.
- ☑ Automate as much as possible.
- ☑ Keep policies version controlled.
- ☑ Maintain clean audit trails.
- ☑ Review and test compliance controls regularly.

COMMON MISTAKES

- ✕ Treating compliance as a manual afterthought.
- ✕ Skipping policy checks for speed.
- ✕ Not storing evidence or artifacts.
- ✕ Overly broad approvals.
- ✕ Not updating controls with changing regulations.

EXAMPLE: COMPLIANCE PIPELINE (.gitlab-ci.yml)

```
stages:
  - build
  - security
  - compliance
  - approve
  - deploy

build:
  stage: build
  image: docker:24
  script:
    - echo "Building application..."
    - docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA .
  artifacts:
    paths:
      - build/
    expire_in: 1 week

security_scan:
  stage: security
  image: registry.gitlab.com/gitlab-org/security-products/analyzers:latest
  script:
    - /analyzer run
  artifacts:
    reports:
      dependency_scanning: gl-dependency-scanning-report.json
      sast: gl-sast-report.json
      container_scanning: gl-container-scanning-report.json

compliance:
  stage: compliance
  image: alpine:3.19
  script:
    - ./scripts/aggregate_reports.sh
    - ./scripts/check
  artifacts:
    reports:
      junit: compliance-report.xml
  when: always

approval:
  stage: approve
  image: alpine:3.19
  environment:
    - /scripts/deploy.sh
    name: production
    url: https://app.example.com
  only: main
```

TOOLS YOU CAN USE

- GitLab Compliance Frameworks
- OPA / Conftest
- Trivy / Grype
- Snyk
- Checkov
- Anchore
- CycloneDX / Syft
- License Finder

29. PIPELINE SECURITY

KEY TAKEAWAY

- [x] Secure pipelines protect your code, infrastructure and data.
- [x] Apply least privilege everywhere.
- [x] Protect your runners, branches, tokens and secrets.
- [x] Security in CI/CD is everyone's responsibility.

WHAT IS PIPELINE SECURITY?

Pipeline security is about preventing unauthorized access, protecting secrets, securing build environments and ensuring that only trusted code goes from commit to production.

PIPELINE SECURITY LAYERS

Code & Repos	Runners	Secrets & Tokens	Jobs & Pipelines	Deployments	Monitoring & Audit
Protected branches, code review	Secure, isolated and trusted	Store, protect and rotate	Least privilege, safe execution	Controlled releases	Detect, alert and audit

1. LEAST PRIVILEGE

- Grant minimal permissions required to users, jobs and services.
- Avoid using personal access tokens.
- Use scoped CI_JOB_TOKEN.

Good Practice

- [x] Use project access tokens with least scope
- [x] Limit protected variables to protected refs
- [x] Restrict job permissions (GITHUB_TOKEN / CI_JOB_TOKEN)
- [x] Avoid 'sudo' in jobs

2. SECURE RUNNERS

- Use dedicated, isolated runners for production.
- Keep runners updated and hardened.
- Avoid shared runners for sensitive jobs.
- Use autoscaling runners when possible.

Good Practice

- [x] Use runners in private subnets
- [x] Enable runner authentication token rotation
- [x] Disable privileged mode unless required
- [x] Use ephemeral runners for untrusted code

EXAMPLE: PROTECTED BRANCH CONFIGURATION

```yaml
# .gitlab-ci.yml
stages:
  - build
  - test
  - deploy

deploy_prod:
  stage: deploy
  image: alpine:3.19
  script:
    - echo "Deploying to production..."
  only:
    - main
  when: manual
  allow_failure: false

# Protected branch settings (Settings > Repository)
# - Require merge request
# - Require pipeline success
# - Require approvals (2)
# - Prevent  force push
# - Prevent  deletion
```

4. TOKEN SECURITY

- Use short-lived tokens when possible.
- Avoid storing tokens in code.
- Limit token scopes to the minimum.
- Rotate tokens regularly.

Good Practice

- [x] Use project access tokens
- [x] Use CI_JOB_TOKEN for API calls
- [x] Disable unused tokens
- [x] Set token expiration

5. CREDENTIAL MANAGEMENT

- Store secrets in GitLab CI/CD variables.
- Use masked and protected variables.
- Use external secret managers for sensitive data.
- Never print secrets in logs.

Good Practice

- [x] Use masked + protected variables
- [x] Use environment scoped variables
- [x] Integrate with HashiCorp Vault / AWS Secrets Manager
- [x] Audit access to secrets

SECURE VARIABLE EXAMPLE

Type	Key	Value	Settings
Variable	DB_PASSWORD	••••••••	Masked, Protected
Variable	AWS_ACCESS_KEY_ID	••••••••	Masked, Protected
Variable	KUBE_CONFIG	••••••••	File, Protected
Variable	SLACK_WEBHOOK	••••••••	Masked

DO NOT DO THIS

- [x] Hardcode secrets in .gitlab-ci.yml
- [x] Use personal tokens in pipelines
- [x] Expose secrets in job logs
- [x] Use unprotected variables for prod
- [x] Run privileged containers by default

TIP

Assume any job can be compromised. Minimize what it can access. Secure everything, and verify continuously.

6. HARDENING BEST PRACTICES

- [x] Keep GitLab and Runners up to date
- [x] Disable unnecessary features
- [x] Enforce HTTPS and secure certificates
- [x] Use network isolation for runners
- [x] Limit outbound network access
- [x] Scan pipeline images for vulnerabilities
- [x] Enable audit events and monitoring
- [x] Review permissions regularly

PIPELINE SECURITY CHECKLIST

- [x] Least privilege applied
- [x] Runners are isolated and hardened
- [x] Protected branches configured
- [x] Secrets are masked and protected
- [x] No hardcoded credentials
- [x] Tokens are scoped and rotated
- [x] Images are scanned
- [x] Audit logs are reviewed

EXAMPLE: SECURE PIPELINE (.gitlab-ci.yml)

```yaml
default:
  image: alpine:3.19
  before_script:
    - apk add --no-cache curl
  tags: [secure-runner]
variables:
  GIT_DEPTH: "20"
stages:
  - build
  - scan
  - test
  - deploy
build:
  stage: build
  script:
   - echo "Building application..."
  only: [merge_requests, main]
sast:
  stage: scan
  include:
    - template: Security/SAST.gitlab-ci.yml
deploy_prod:
  stage: deploy
  script:
    - echo "Deploy to production"
  environment: production
  only: [main]
  when:  manual
  allow_failure: false
```

COMMON PIPELINE THREATS & MITIGATIONS

Threat	Mitigation
Malicious code in MR	Code review, SAST, protected branches
Secret leakage in logs	Masked variables, no echo, log review
Runner compromise	Isolated runners, updates, least privilege
Unauthorized deployments	Protected environments, approvals
Token theft	Scoped tokens, rotation, short-lived
Dependency vulnerabilities	Dependency scanning, updates
Supply chain attacks	Pin versions, verify checksums, scanning

30. Deployment Strategies

KEY TAKEAWAY

- ☑ Deployment strategy defines how new versions reach production.
- ☑ Choose the right strategy for your app, risk level and team maturity.
- ☑ Automate deployments with GitLab CI/CD and environment promotions.
- ☑ Measure, monitor and rollback fast when issues occur.

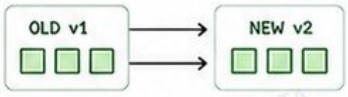
☆ WHAT IS A DEPLOYMENT STRATEGY?

A deployment strategy is a planned method for releasing application changes to production with minimal risk and maximum control. The goal is to deliver value safely and rollback quickly if needed.

☆ TYPICAL DEPLOYMENT PIPELINE FLOW

1. ROLLING DEPLOYMENT

- Gradually replace old instances with new ones.
- Zero downtime for most applications.
- Uses Kubernetes RollingUpdate by default.

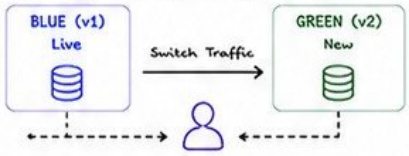


Example (Kubernetes)

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 25%
    maxUnavailable: 25%
```

2. BLUE-GREEN DEPLOYMENT

- Two identical environments: Blue (live) and Green (new).
- Switch traffic to new environment instantly.
- Easy rollback by switching back.

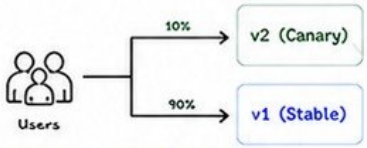


Example (GitLab CI/CD)

```
deploy_blue:
  stage: deploy
  environment: { name: blue }
  script: ./deploy.sh blue

deploy_green:
  stage: deploy
  environment: { name: green }
  script: ./deploy.sh green

switch_traffic:
  stage: deploy
  script: ./switch_traffic.sh green
```

3. CANARY DEPLOYMENT

- Release new version to a small subset of users.
- Monitor metrics before increasing traffic.
- Automatically rollback if error rate increases.

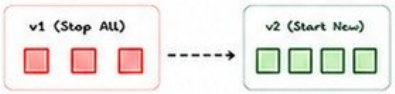


Example (Kubernetes + Istio/NGINX)

```
# Route 10% traffic to v2
gateway:
  http:
  - route:
    - destination:
        host: app
        subset: v2
      weight: 10
    - destination:
        host: app
        subset: v1
      weight: 90
```

4. RECREATE DEPLOYMENT

- Terminate all old instances before starting new.
- Simple but causes downtime.
- Use only when downtime is acceptable.



Example (Kubernetes)

```
strategy:
  type: Recreate
```

5. PROGRESSIVE DELIVERY (GRADUAL ROLLOUT)

- Automatically increase traffic to new version over time.
- Uses metrics and automated analysis.
- Combines canary + automation for safety.

Example (Argo Rollouts)

```
strategy:
  canary:
    steps:
    - setWeight: 10
    - pause: { duration: 5m}
    - setWeight: 25
    - pause: { duration: 10m}
                              - pause: { duration: 10m}
                              - pause: { duration: 10m}
```

6. STRATEGY SELECTION GUIDE

Strategy	Downtime	Risk Level	Best For
Rolling	Low	Low	Most web apps (Default)
Blue-Green	None	Low	Mission critical apps
Canary	Very Low	Low	User facing features
Recreate	High	High	Batch jobs, simple apps
Progressive	Very Low	Lowest	Complex systems, critical releases

☆ DEPLOYMENT BEST PRACTICES

- ☑ Automate everything (build, test, deploy, rollback).
- ☑ Use health checks and readiness probes.
- ☑ Monitor metrics, logs and user experience.
- ☑ Have an instant rollback plan.
- ☑ Promote through environments (dev → staging → prod).
- ☑ Use feature flags for safer releases.
- ☑ Document and review deployment runbooks.

☆ COMMON MISTAKES

- ✗ Skipping testing before production.
- ✗ Deploying large changes without validation.
- ✗ No monitoring after deployment.
- ✗ No rollback plan or untested rollback.
- ✗ Ignoring infrastructure capacity.
- ✗ Not using feature flags when needed.

TIP

Start with Rolling Deployments. Move to Blue-Green or Canary as your automation, testing and monitoring maturity grows.

☆ STRATEGY COMPARISON AT A GLANCE

Strategy	Downtime	Rollback Speed	Infrastructure Cost	Complexity	Risk	Ideal Use Case
Rolling	Low	Medium	Low	Low	Low	Most applications
Blue-Green	None	Fast	High (2x env)	Medium	Low	Critical applications
Canary	Very Low	Fast	Low	Medium	Low	New features, updates
Recreate	High	Medium	Low	Low	High	Batch jobs, non-critical
Progressive	Very Low	Fast	Low	High	Very Low	Complex, mission critical

☆ TOOLS YOU CAN USE

GitLab CI/CD	Pipelines
Kubernetes	Deployments
Argo Rollouts	Progressive Delivery
Istio / NGINX	Traffic Management
Prometheus / Grafana	Monitoring

31. GitLab Environments

KEY TAKEAWAY

- ☑ Environments map your delivery process from dev to production.
- ☑ Each environment has a purpose, URL, and lifecycle.
- ☑ Use Review Apps for feature validation.
- ☑ Promote code through environments with controlled releases.
- ☑ Clean up idle environments automatically.

WHAT ARE GITLAB ENVIRONMENTS?

GitLab Environments represent the different stages where your application lives. They help you track deployments, monitor health, manage releases, and control promotion across the delivery lifecycle.

ENVIRONMENT LIFECYCLE OVERVIEW

Create	Deploy	Monitor	Promote	Expire / Stop	Delete
Environment is created	Application deployed	Health, metrics and logs	Move to next stage	Stop environment when idle	Remove environment

ENVIRONMENT HIERARCHY

Development
Purpose: Daily development and integration.
Auto Deploy: Yes
URL Example: dev.example.com

Testing
Purpose: Automated testing and QA validation.
Auto Deploy: Yes
URL Example: test.example.com

Staging
Purpose: Pre-production validation and UAT.
Auto Deploy: Manual / Scheduled
URL Example: staging.example.com

Production
Purpose: Live environment for real users.
Auto Deploy: Manual (Protected)
URL Example: app.example.com

Review Apps
Purpose: Per-merge request isolated environments.
Auto Deploy: Yes (On MR)
URL Example: review-$CI_COMMIT_REF_SLUG.example.com

GITLAB ENVIRONMENT CONFIGURATION (.gitlab-ci.yml)

```yaml
deploy_dev:
  stage: deploy
  image: alpine:3.19
  environment:
    name: development
    url: https://dev.example.com
    on_stop: stop_dev
  script:
    - ./deploy.sh dev

stop_dev:
  stage: deploy
  when: manual
  environment:
    name: development
    action: stop
  script:
    - ./stop.sh dev

# Review App for Merge Requests
deploy_review:
  stage: deploy
  image: alpine:3.19
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    url: https://review-$CI_COMMIT_REF_SLUG.example.com
    on_stop: stop_review
    auto_stop_in: 3 days
  only:
    - merge_requests
  script:
    - ./deploy.sh review

stop_review:
  stage: deploy
  when: manual
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    action: stop
  script:
    - ./stop.sh review
```

REVIEW APP FLOW

1. Developer opens Merge Request
2. GitLab creates Review App
3. QA tests the changes
4. MR Approved
5. Environment stopped automatically or manually
6. Environment deleted after expiry

TIP

Use auto_stop_in for Review Apps to control cost and clutter. Keep Production protected and deploy only through approved pipelines.

BEST PRACTICES

- ☑ Use meaningful environment names.
- ☑ Protect Production from direct pushes.
- ☑ Use Review Apps for every MR.
- ☑ Monitor environment health continuously.
- ☑ Expire idle environments automatically.

ENVIRONMENT DASHBOARD (WHAT YOU SEE)

Environment	Status	Last Deployed	Deployed By	URL	Auto Stop
development	Available	2 min ago	ann.felix	dev.example.com	Never
testing	Available	10 min ago	ann.felix	test.example.com	Never
staging	Available	1 hour ago	ann.felix	staging.example.com	Never
production	Available	2 days ago	ann.felix	app.example.com	Never
review/feature-login	Stopping in 1 day	just now	ann.felix	review-feature-login.example.com	1 day

COMMON ACTIONS

- Deploy to Environment
- Stop Environment
- Edit Environment
- Delete Environment
- View Logs & Metrics

COMMON MISTAKES

- ✗ Deploying to Production automatically.
- ✗ Not using protected environments.
- ✗ Leaving Review Apps running too long.
- ✗ No environment health monitoring.
- ✗ Hardcoding URLs in pipeline scripts.

ENVIRONMENT PROTECTION (PRODUCTION EXAMPLE)

Protected Environment: production

Allowed to deploy:
- ✓ Maintainers
- ✓ Release Managers
- ✓ CI/CD with approvals

Blocked: All other users

Rules Example (.gitlab-ci.yml)

```yaml
deploy_prod:
  stage: deploy
  script: ./deploy.sh prod
  environment:
    name: production
    url: https://app.example.com
  only:
    - main
  when: manual
  allow_failure: false
```

Approval Flow

Deploy Job Triggered → Approval Required → Deploy to Production

ENVIRONMENT VARIABLES EXAMPLE

Environment	Key	Value (Example)
development	API_URL	https://dev.api.example.com
testing	API_URL	https://test.api.example.com
staging	API_URL	https://staging.api.example.com
production	API_URL	https://api.example.com

32. Review Apps

KEY TAKEAWAY

- ☑ Review Apps create a unique environment for every change.
- ☑ Developers get instant previews before merge.
- ☑ They are isolated, on-demand, and automatically cleaned up.
- ☑ Improve quality, speed, and collaboration across teams.

☆ WHAT ARE REVIEW APPS?

Review Apps are temporary environments created for each Merge Request. They allow developers, QA, and stakeholders to preview changes, test features, and provide feedback before code is merged.

☆ REVIEW APP LIFECYCLE

1. DYNAMIC ENVIRONMENTS

- Each MR gets a unique environment.
- Names are generated automatically.
- Fully isolated and repeatable.

Naming Convention Example

`review/<project>-<mr-id>-<slug>`

Example: `review/shop-125-add-login`

2. FEATURE PREVIEWS

- See real UI/UX of your changes.
- Test integrations, APIs, and flows.
- Share link with team or clients.

3. AUTOMATIC CLEANUP

- Environments stop when MR is closed.
- Prevents resource waste and cost.
- Ensures clean and secure systems.

Auto Stop Triggers

- ☑ MR Merged
- ☑ MR Closed
- ☑ Branch Deleted
- ☑ Inactivity Timeout (e.g. 3 days)

4. DEVELOPER WORKFLOWS

1. Create feature branch
   `git checkout -b feature/login`
2. Push branch to GitLab
   `git push -u origin feature/login`
3. Open Merge Request
4. Review App is deployed automatically
5. Test, iterate, get feedback
6. Merge → Review App cleaned up

GITLAB CI/CD EXAMPLE (.gitlab-ci.yml)

```yaml
stages:
  - deploy_review      - stop_review

deploy_review:
  stage: deploy_review
  image: alpine:3.19
  environment:
    name: review/$CI_COMMIT_REF_SLUG-$CI_MERGE_REQUEST_IID
    url: https://$CI_ENVIRONMENT_SLUG.$REVIEW_DOMAIN
    on_stop: stop_review
    auto_stop_in: 3 days
  script:
    - echo "Deploying Review App for MR $CI_MERGE_REQUEST_IID"
    - ./deploy.sh review
  rules:
    - if: $CI_MERGE_REQUEST_IID
      when: on_success
stop_review:
  stage: stop_review
  image: alpine:3.19
  environment:
    name: review/$CI_COMMIT_REF_SLUG-$CI_MERGE_REQUEST_IID
    action: stop
  script:
    - echo "Stopping Review App for MR $CI_MERGE_REQUEST_IID"
    - ./cleanup.sh review
  rules:
    - if: $CI_MERGE_REQUEST_EVENT_TYPE == "merged" || $CI_MERGE_REQUEST_EVENT_TYPE == "closed"
      when: on_success
```

5. ENTERPRISE COLLABORATION

- Share secure preview links.
- Collect feedback directly on MR.
- Integrate with Slack, Teams, Jira.
- Role-based access and approvals.

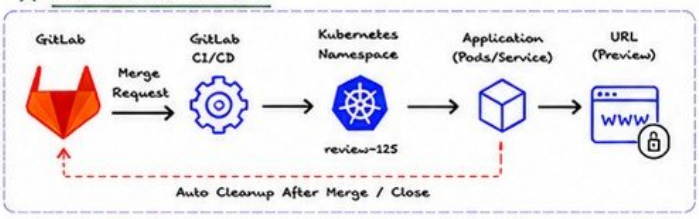
Who Benefits?

- Developers – Build & test faster
- QA Teams – Validate features early
- Product Owners – See real changes
- Clients – Provide feedback
- Operations – Monitor & secure

TIP

Use review apps for every MR. Keep them lightweight, fast, and automatically cleaned up. This builds trust and speeds up delivery.

☆ REVIEW APP ARCHITECTURE



☆ ENVIRONMENT EXAMPLES

Environment	Purpose	URL Example	Auto Stop
development	Daily dev & integration	dev.example.com	Never
review/123	MR #123 preview	review-123.example.com	3 days
testing	Automated testing	test.example.com	Never
staging	Pre-production validation	staging.example.com	Never
production	Live users	app.example.com	Never

☆ BEST PRACTICES

- ☑ Use meaningful environment names.
- ☑ Limit resource usage per Review App.
- ☑ Set auto_stop_in to avoid orphaned apps.
- ☑ Protect sensitive data in Review Apps.
- ☑ Monitor Review Apps (logs, metrics, uptime).
- ☑ Clean up namespaces, DBs, and storage.

☆ COMMON MISTAKES

- ✗ Not using auto cleanup → resource waste.
- ✗ Exposing Review Apps to public unintentionally.
- ✗ Using production data in Review Apps.
- ✗ Heavy apps causing long startup times.
- ✗ Not monitoring Review App health.

☆ TOOLS YOU CAN USE

- GitLab CI/CD
- Kubernetes
- Helm
- Ingress / NGINX
- Prometheus / Grafana
- Slack / Teams
- Jira / Linear
- Sentry
- PostgreSQL / MySQL
- Redis

33. DEPLOYMENTS

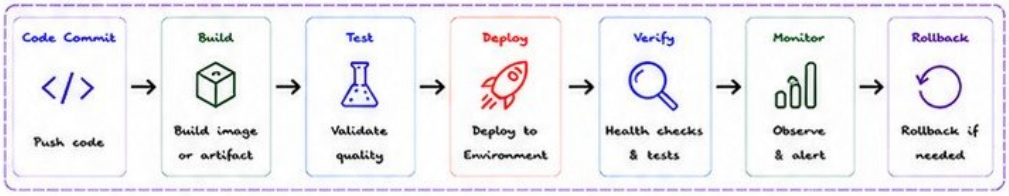
KEY TAKEAWAY

- ☑ Deployments move your application from one environment to another.
- ☑ Every deployment should be traceable, repeatable, and reversible.
- ☑ Use approvals and protected environments for production releases.
- ☑ Always validate, monitor, and have a rollback plan.

WHAT ARE DEPLOYMENTS?

Deployments in GitLab CI/CD move a built artifact to a target environment.
They are tracked, versioned, and can be rolled back.
GitLab keeps a full deployment history for audit and troubleshooting.

DEPLOYMENT FLOW



1. DEPLOYMENT JOBS

- Deploy using GitLab CI/CD jobs.
- Use environments to track where it runs.
- Mark jobs with environment, url, and action.

Example (.gitlab-ci.yml)

```yaml
deploy_staging:
  stage: deploy
  image: alpine:3.19
  environment:
    name: staging
    url: https://staging.example.com
    action: start
  script:
    - echo "Deploying to Staging..."
    - ./deploy.sh staging
  only:
    - main
deploy_production:
  stage: deploy
  image: alpine:3.19
  environment:
    name: production
    url: https://app.example.com
    action: start
  script:
    - echo "Deploying to Production..."
    - ./deploy.sh production
  only:
    - tags
```

2. DEPLOYMENT HISTORY

- View all deployments in a timeline.
- See who deployed what, when, and where.
- Track status: success, failed, canceled.

Deployment History - Production

ID	Status	Environment	Deployed By	Time	Commit / Version	
#125	Success	production	ann.felix	2h ago	v1.4.0	✓
#124	Success	production	ann.felix	1 day ago	v1.3.9	✓
#123	Failed	production	ann.felix	2 days ago	v1.3.8	✗
#122	Success	production	ann.felix	3 days ago	v1.3.7	✓
#121	Success	staging	ann.felix	3 days ago	-	✓

You can filter by environment, user, status or time range.

3. ROLLBACK

- Rollback to any previous successful deployment.
- One click rollback from the UI or CLI.

Rollback Options

UI	Go to Deployments → Select deployment → Click Rollback
CLI	glab rollout undo <environment> --to <deploy-id>
Kubernetes	kubectl rollout undo deployment/<app> --namespace=<ns>

Example: Rollback to deployment #124

```
$ glab deployment rollback production --to 124
✓ Rollback started
✓ Deployment #126 created
✓ Environment: production is now at v1.3.9
```

4. DEPLOYMENT APPROVALS

- Protect critical environments with manual approvals.
- Ensure only authorized users can deploy.
- Use protected environments in GitLab settings.

Example: Manual Approval Job

```yaml
deploy_production:
  stage: deploy
  image: alpine:3.19
  environment:
    name: production
    url: https://app.example.com
    action: start
  when: manual
  allow_failure: false
  only:
    - tags
  script:
    - ./deploy.sh production
```

TIP

Use protected environments + manual jobs + approvals to keep production safe.

5. PRODUCTION DEPLOYMENTS

- Use protected environments.
- Require approvals and successful pipelines.
- Monitor closely and have rollback ready.

Production Deployment Checklist

- ☑ Protected environment in place
- ☑ Only tags can deploy
- ☑ Manual approval enabled
- ☑ Health checks pass
- ☑ Monitoring & alerts active
- ☑ Rollback plan tested

DEPLOYMENT STATES

State	Meaning
created	Deployment created but not started
running	Deployment is in progress
success	Deployment completed successfully
failed	Deployment failed
canceled	Deployment was canceled
skipped	Deployment skipped by rules

DEPLOYMENT STRATEGIES SUPPORT

Rolling Update	Blue-Green	Canary	Recreate	Progressive Delivery
Update pods gradually with zero downtime.	Two identical environments. Switch traffic instantly.	Deploy to small % of users first, then scale up.	Stop all, then start new version.	Automate progression based on metrics.
Best for most apps.	Best for critical apps.	Best for risk reduction.	Best for simple apps.	Best for advanced teams.

BEST PRACTICES

- ☑ Use meaningful environment names.
- ☑ Deploy only from protected branches or tags.
- ☑ Require approvals for production.
- ☑ Always run tests before deployment.
- ☑ Monitor after deployment.
- ☑ Keep rollback simple and tested.

COMMON MISTAKES

- ✗ Deploying to production without approval.
- ✗ Not protecting environments.
- ✗ No rollback plan.
- ✗ Deploying untested code.
- ✗ Skipping health checks.

USEFUL COMMANDS (gitab CLI)

```
List deployments:
  glab deployment list
View deployment details:
  glab deployment viewww <id>
Rollback deployment:
  glab deployment rollback <environment> --to <id>
Cancel deployment:
  glab deployment cancel <id>
```

TOOLS YOU CAN USE

- GitLab CI/CD
- Kubernetes
- Helm
- Istio / Linkerd
- Prometheus / Grafana
- Argo Rollouts
- Slack / Teams
- Sentry

34. KUBERNETES INTEGRATION

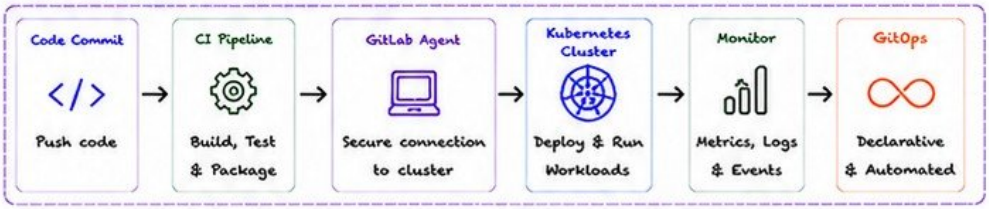
KEY TAKEAWAY

- ☑ GitLab integrates deeply with Kubernetes for reliable, scalable deployments.
- ☑ Use the GitLab Agent to connect securely to your clusters.
- ☑ Automate deployments with CI/CD jobs, Helm charts, and GitOps principles.
- ☑ Enable GitOps for declarative, auditable, and self-healing operations.
- ☑ Always follow production-grade practices for security, reliability, and observability.

WHAT IS KUBERNETES INTEGRATION?

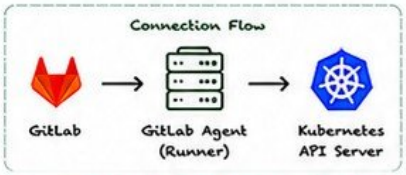
GitLab allows you to connect to Kubernetes clusters, deploy applications, manage environments, and use GitOps workflows for continuous delivery. It combines CI/CD automation with Kubernetes-native capabilities.

KUBERNETES INTEGRATION OVERVIEW



1. CLUSTER CONNECTION

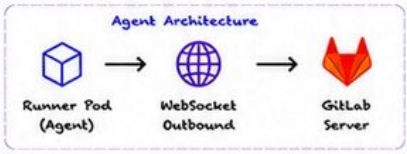
- Connect GitLab to your Kubernetes cluster using the GitLab Agent.
- Agent runs inside your network.
- No inbound firewall rules required.




```
# Register Kubernetes cluster using GitLab Agent
gitlab-agent register \
  --url https://gitlab.com \
  --token glrt-XXXXXXXXXXXXXXXXXXXX \
  --name k8s-prod-agent \
  --kubernetas-context production
```

2. GITLAB AGENT

- Lightweight agent that connects to GitLab.
- Handles CI/CD workloads and deployments.
- Supports multiple clusters and projects.
- Secure, outbound-only connection.



3. DEPLOYMENTS TO KUBERNETES

- Use Kubernetes executor or GitLab Agent for deployments.
- Apply manifests, Helm charts, or Kustomize.
- Track deployments in GitLab Environments.

Example: CI Job to Deploy to Kubernetes

```
deploy:
  stage: deploy
  image: bitnami/kubectl:latest
  environment:
    name: staging
    url: https://staging.example.com
  script:
    - echo "Deploying to Kubernetes..."
    - kubectl config use-context $K8S_CONTEXT
    - kubectl apply -f k8s/namespace.yaml
    - kubectl apply -f k8s/deployment.yaml
    - kubectl rollout status deployment/myapp -n staging
  only:
    - main
```

Kubernetes Executor Configuration

```
runners:
  kubernetes:
    image: alpine:3.19
    namespace: gitlab-runner
    service_account_overwrite_allowed: "k8s-runner"
    privileged: false
    poll_timeout: 300
```

4. HELM INTEGRATION

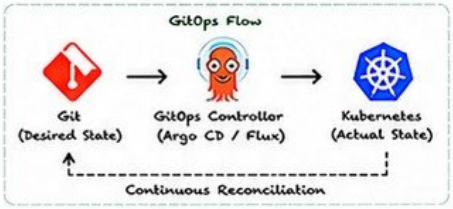
- Package your app with Helm charts.
- Version, configure, and deploy easily.
- Promote the same chart across environments.

Example: Helm Deploy Job

```
deploy_helm:
  stage: deploy
  image: alpine/helm:3.14.0
  script:
    - helm repo add myrepo https://charts.example.com
    - helm repo update
    - helm upgrade --install myapp myrepo/myapp \
        --namespace staging --create-namespace \
        -f values-staging.yaml --atomic --wait
  environment:
    name: staging
    url: https://staging.example.com
```

5. GITOPS CONCEPTS

- Git is the single source of truth.
- Changes via Merge Requests.
- Automated reconciliation in the cluster.
- Tools: Argo CD, Flux, or GitLab Deploy Boards.



6. PRODUCTION INTEGRATION BEST PRACTICES

- ☑ Use protected branches and environments.
- ☑ Enforce approvals for production deployments.
- ☑ Use namespaces per environment or team.
- ☑ Enable RBAC and least-privilege ServiceAccounts.
- ☑ Store kubeconfig or tokens as protected variables.
- ☑ Use Helm values per environment (dev/stage/prod).
- ☑ Enable health checks and rollout monitoring.
- ☑ Use auto-rollback on failed deployments.
- ☑ Enable audit logging and notifications.

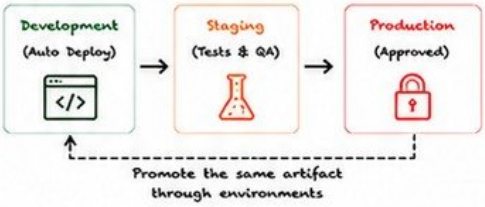
DEPLOYMENT HISTORY & ROLLBACK

- View all deployments in Environments.
- See status, commit, duration, and user.
- Rollback to any previous successful version.

Example: Rollback with Helm

```
# Rollback to previous revision
helm rollback myapp 5 --namespace production

# Rollback to specific revision
helm rollback myapp 3 --namespace production
```

ENVIRONMENT PROMOTION FLOW



COMMON MISTAKES

- ✕ Hardcoding kubeconfig or tokens in CI/CD.
- ✕ Deploying to production without approvals.
- ✕ Using latest image tags in production.
- ✕ Skipping health checks and monitoring.
- ✕ No rollback strategy defined.

USEFUL COMMANDS (kubectl)

```
kubectl get pods -n <namespace>
kubectl get deployments -n <namespace>
kubectl rollout status deployment/<name> -n <ns>
kubectl rollout history deployment/<name> -n <ns>
kubectl rollout undo deployment/<name> -n <ns>
```

TOOLS & INTEGRATIONS

GitLab Agent	- Secure cluster connectivity
Helm	- Package & deploy apps
Argo CD / Flux	- GitOps controllers
Kubernetes	- Orchestrate workloads
Prometheus/Grafana	- Observe & monitor

35. Infrastructure as Code

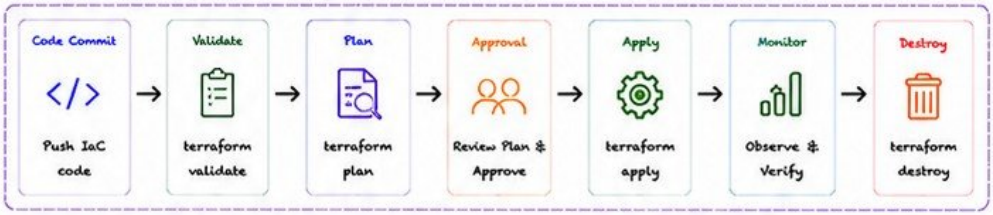
KEY TAKEAWAY

- ☑ Infrastructure as Code (IaC) brings consistency, repeatability, and speed.
- ☑ Terraform pipelines automate validation, planning, and provisioning.
- ☑ Always validate and review the plan before applying.
- ☑ Use approvals and protected environments for production safety.
- ☑ Destroy with care. Infrastructure is code, but impact is real.

WHAT IS INFRASTRUCTURE AS CODE?

IaC means you define, version, and manage infrastructure using code.
Terraform is a popular IaC tool that supports multi-cloud resources.
GitLab CI/CD automates the full lifecycle: validate, plan, apply, and destroy.

TERRAFORM PIPELINE OVERVIEW



1. TERRAFORM PIPELINE STAGES

Validate Check Terraform code syntax and configuration.
Command: `terraform validate`

Phon execution plan. No changes are made to infrastructure.
Command: `terraform plan -out=tfplan`

Approval
Human review of the plan output. Ensure changes are safe, intended, and compliant.

Apply
Apply the approved plan to provision or update infrastructure.
Command: `terraform apply tfplan`

Destroy
Tear down infrastructure when no longer needed.
Command: `terraform destroy`

2. GITLAB CI/CD PIPELINE EXAMPLE (.gitlab-ci.yml)

```yaml
stages:
  - validate
  - plan
  - approve
  - apply
  - destroy

variables:
  TF_ROOT: "infra/terraform"

validate:
  stage: validate
  image: hashicorp/terraform:1.6.6
  script:
    - cd $TF_ROOT
    - terraform init -backend-config="backend.hcl"
    - terraform validate
  rules:
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event" || $CI_COMMIT_BRANCH'

plan:
  stage: plan
  image: hashicorp/terraform:1.6.6
  script:
    - cd $TF_ROOT
    - terraform init -backend-config="backend.hcl"
    - terraform plan -out=tfplan
    - terraform show -no-color tfplan > plan.txt
  artifacts:
    paths:
      - $TF_ROOT/tfplan
      - $TF_ROOT/plan.txt
    expire_in: 7 days
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'

approve:
  stage: approve
  image: alpine:3.19
  script:
    - echo "Review the plan artifact (plan.txt) before approving."
    - echo "Approve this job to continue."
  when: manual
  allow_failure: false
  needs: ["plan"]
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'

apply:
  stage: apply
  image: hashicorp/terraform:1.6.6
  script:
    - cd $TF_ROOT
    - terraform init -backend-config="backend.hcl"
    - terraform apply -input=false tfplan
  environment:
    name: production
    url: https://infra.example.com
  needs: ["approve"]
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
```

3. STAGE DETAILS

VALIDATION
- Catches syntax and configuration errors early.
- Runs on every commit and MR.

PLAN
- Generates execution plan.
- Shows what will be created, updated, or destroyed.

APPROVAL
- Human gate before changes are applied.
- Use protected environments and code owners.

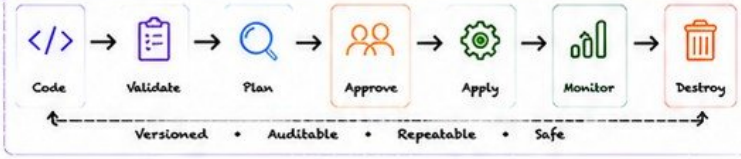
APPLY
- Applies the exact plan that was approved.
- Records state in remote backend.

DESTROY
- Destroys infrastructure safely when required.
- Highly restricted and manual.

4. TERRAFORM BACKEND (REMOTE STATE)

- Stores state securely and centrally
- Enables team collaboration
- Prevents state corruption with locking

5. INFRASTRUCTURE LIFECYCLE FLOW



6. APPROVAL WORKFLOWS

TIP

Always review the plan.
Infrastructure changes are powerful.
A small mistake in code can create big impact in production.

BEST PRACTICES

- ☑ Keep Terraform code modular and reusable.
- ☑ Use meaningful names and tagging.
- ☑ Store state in secure remote backend.
- ☑ Enable state locking.
- ☑ Protect production environments.
- ☑ Require approvals for plan and apply.
- ☑ Log and monitor all infrastructure changes.

COMMON MISTAKES

- ✗ Applying without reviewing the plan.
- ✗ Using local state instead of remote backend.
- ✗ Hardcoding secrets in Terraform code.
- ✗ Destroying production without approvals.
- ✗ Not using workspaces or environment separation.

USEFUL TERRAFORM COMMANDS

```
terraform init                  # Initialize backend and providers
terraform validate              # Validate configuration
terraform fmt -check            # Check code formatting
terraform plan -out=tfplan      # Generate and save plan
terraform show tfplan           # Readable plan output
terraform apply tfplan          # Apply the saved plan
terraform destroy -auto-approve # Destroy resources
```

36. Cloud Deployments

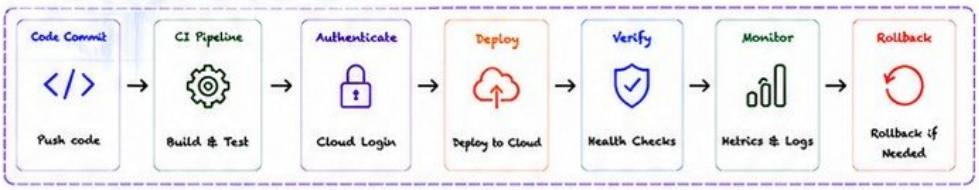
KEY TAKEAWAY

- [x] Deploy securely to AWS, Azure, and Google Cloud using GitLab CI/CD.
- [x] Use strong authentication and least privilege access for all cloud operations.
- [x] Automate deployments, monitoring, and rollback with confidence.
- [x] Follow enterprise practices for security, governance, and cost control.
- [x] Always validate, scan, and protect your pipeline and infrastructure.

WHAT ARE CLOUD DEPLOYMENTS?

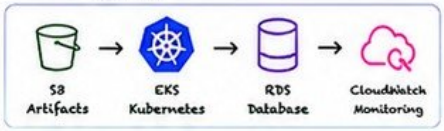
Cloud deployments allow you to deliver applications and infrastructure to public cloud providers securely and reliably.
GitLab CI/CD helps you automate and standardize deployments across AWS, Azure, and Google Cloud Platform.

CLOUD DEPLOYMENT OVERVIEW



1. AMAZON WEB SERVICES (AWS)

- Use IAM roles and OIDC for authentication.
- Deploy to EC2, ECS, EKS, Lambda, S3, etc.
- Manage infrastructure with IaC (CloudFormation, Terraform).

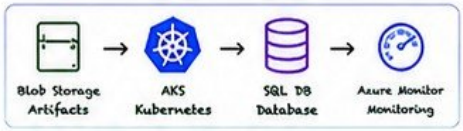


AWS DEPLOY EXAMPLE (.gitlab-ci.yml)

```yaml
deploy_aws:
  stage: deploy
  image: amazon/aws-cli:2.15.0
  id_tokens:
    AWS_OIDC_TOKEN:
      aud: https://gitlab.com
  script:
    - echo "Logging in to AWS..."
    - aws sts assume-role-with-web-identity \
        --role-arn $AWS_ROLE_ARN \
        --role-session-name gitlab-session \
        --web-identity-token $AWS_OIDC_TOKEN > /tmp/creds.json
    - export AWS_ACCESS_KEY_ID=$(jq -r '.Credentials.AccessKeyId' /tmp/creds.json)
    - export AWS_SECRET_ACCESS_KEY=$(jq -r
      export AWS_SESSION_TOKEN=$(jq -r '.Credentials.SessionToken' /tmp/creds.json)
    - aws eks update-kubeconfig --name $EKS_CLUSTER --region $AWS_REGION
    - kubectl apply -f k8s/
```

2. MICROSOFT AZURE

- Authenticate using Service Principal + Federated Credential.
- Deploy to AKS, App Service, VM, Storage, Functions.
- Use ARM Templates, Bicep, or Terraform.



AZURE DEPLOY EXAMPLE (.gitlab-ci.yml)

```yaml
deploy_azure:
  stage: deploy
  image: mcr.microsoft.com/azure-cli:2.59.0
  variables:
    AZURE_CLIENT_ID: $AZURE_CLIENT_ID
    AZURE_TENANT_ID: $AZURE_TENANT_ID
    AZURE_SUBSCRIPTION_ID: $AZURE_SUBSCRIPTION_ID
  script:
    - az login --service-principal \
        --username $AZURE_CLIENT_ID \
        --tenant $AZURE_TENANT_ID \
        --federated-token $AZURE_OIDC_TOKEN
    - az account set --subscription $AZURE_SUBSCRIPTION_ID
    - az aks get-credentials --name
        $AKS_CLUSTER --resource-group $GROUP --admin
    - kubectl apply -f k8s/
```

3. GOOGLE CLOUD PLATFORM (GCP)

- Authenticate using Workload Identity Federation.
- Deploy to GKE, Cloud Run, Compute Engine, etc.
- Use Terraform, Deployment Manager, or gcloud.

GCP DEPLOY EXAMPLE (.gitlab-ci.yml)

```yaml
deploy_gcp:
  stage: deploy
  image: google/cloud-sdk:slim
  id_tokens:
    GCP_OIDC_TOKEN:
      aud: https://gitlab.com
  script:
    - echo "Authenticating to GCP..."
    - gcloud auth login --cred-file=<(\
    - echo "{\"type\":\"external_account\",\"audience\":\"//iam.
      googleapis.com/projects/$GCP_PROJECT/locations/global/workload-
      identityPools/$POOL_ID/providers/$PROVIDER_ID\",\"subject_token_
      type\":\"urn:ietf:params:oauth:token-type:jwt\",\"token_
      url\":\"https://sts.googleapis.com/v1/token\",\"credential_
      source\":{\"file\":\"/dev/stdin\"}}")
    - gcloud container clusters get-credentials $GKE_CLUSTER
      --region $GCP_REGION --project $GCP_PROJECT
    - kubectl apply -f k8s/
```

4. AUTHENTICATION MODELS

Cloud	Recommended Auth	Method
AWS	OIDC + IAM Role	GitLab OIDC federation (assume-role-with-web-identity)
Azure	Service Principal + Federated Credential	GitLab OIDC → Azure AD → Service Principal
GCP	Workload Identity Federation	GitLab OIDC → GCP STS → Service Account

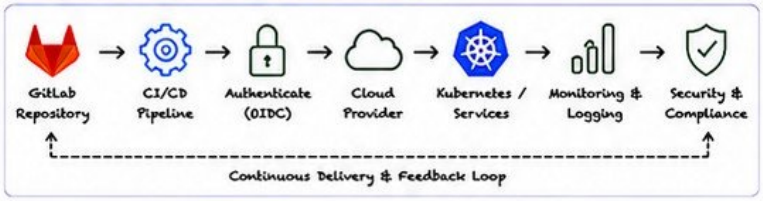
5. SECURE DEPLOYMENTS

- [x] Use least privilege IAM roles / service accounts.
- [x] Store credentials in protected variables only.
- [x] Use OIDC / Workload Identity — avoid long-lived keys.
- [x] Scan images and IaC before deployment.
- [x] Encrypt data in transit (TLS) and at rest.
- [x] Enable network policies and private endpoints.
- [x] Use protected branches and environment approvals.

6. ENTERPRISE PRACTICES

- [x] Multi-account / multi-subscription structure.
- [x] Centralized logging and monitoring.
- [x] Cost tagging and budget alerts.
- [x] Policy as Code (OPA, Sentinel, Azure Policy).
- [x] Regular access reviews and rotation.
- [x] Infrastructure versioning and documentation.
- [x] Disaster recovery and backup strategies.

7. ENTERPRISE CLOUD DEPLOYMENT ARCHITECTURE



TIP

Always automate cloud deployments with security first. Validate, monitor, and improve continuously.

8. CLOUD DEPLOYMENT CHECKLIST

- [x] Authentication is secure and least privilege.
- [x] Pipeline validates IaC and application.
- [x] Plan reviewed and approved.
- [x] Deployment executed successfully.
- [x] Health checks pass.
- [x] Monitoring and alerts configured.
- [x] Rollback strategy tested and ready.

COMMON MISTAKES

- [x] Using long-lived access keys.
- [x] Deploying without approvals to production.
- [x] Not validating IaC before applying.
- [x] Over-privileged IAM roles.
- [x] No monitoring or rollback plan.

USEFUL COMMANDS

```
aws sts get-caller-identity
az account show
gcloud auth list
kubectl get nodes -o wide
kubectl rollout status deployment/<name>
kubectl rollout undo deployment/<name>
```

TOOLS & SERVICES

- AWS: EKS, EC2, S3, IAM, CloudWatch
- Azure: AKS, App Service, Key Vault, Monitor
- GCP: GKE, Cloud Run, Cloud SQL, IAM
- Terraform / Bicep
- Helm
- GitLab CI/CD
- Prometheus / Grafana
- Argo CD / Flux (GitOps)

37. Pipeline Performance

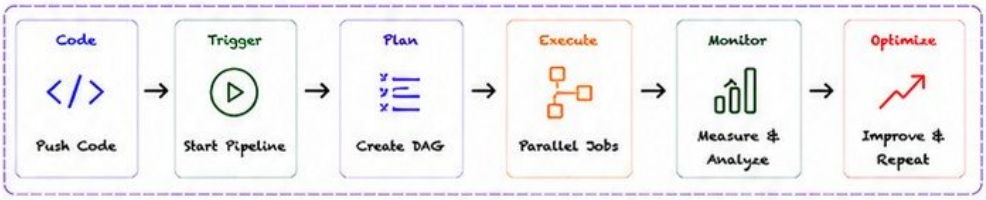
KEY TAKEAWAY

- [x] Fast pipelines deliver value faster.
- [x] Optimize for feedback speed, not just execution speed.
- [x] Parallelize where possible.
- [x] Use DAG pipelines to remove unnecessary waiting.
- [x] Monitor, measure, and eliminate bottlenecks continuously.

WHAT IS PIPELINE PERFORMANCE OPTIMIZATION?

Pipeline performance optimization is the practice of reducing pipeline execution time, improving resource usage, and increasing developer productivity. It focuses on speed, efficiency, reliability, and scalability.

PIPELINE PERFORMANCE OPTIMIZATION OVERVIEW



1. OPTIMIZATION PRINCIPLES

- [x] Reduce total pipeline duration.
- [x] Maximize parallel execution.
- [x] Eliminate unnecessary work.
- [x] Use efficient resources and caching.
- [x] Provide fast feedback to developers.

2. PARALLEL JOBS

- Run independent jobs at the same time.
- Use "needs" or stage parallelism.
- Best for tests, builds, scans, and deployments.

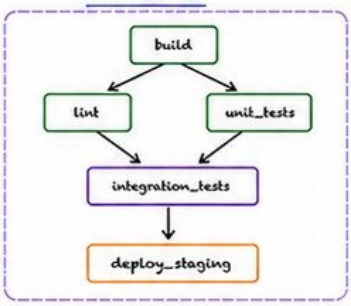
EXAMPLE: PARALLEL JOBS (.gitlab-ci.yml)

```yaml
stages:
  - build
  - test
  - deploy

build_app:
  stage: build
  script:
    - echo "Building application"

unit_tests:
  stage: test
  needs: [build_app]
  script:
    - echo "Running unit tests"

integration_tests:
  stage: test
  needs: [build_app]
  script:
    - echo "Running integration tests"

deploy_staging:
  stage: deploy
  needs: [unit_tests, integration_tests]
  script:
    - echo "Deploying to staging"
```

DAG PIPELINES (DIRECTED ACYCLIC GRAPH)

- Define explicit job dependencies.
- Jobs start as soon as their dependencies are complete.
- Reduces idle time and shortens pipelines.

DAG EXAMPLE



TIP

Use "needs:" to create DAG pipelines and remove unnecessary waiting between stages.

4. RESOURCE UTILIZATION

- Right-size your runners.
- Use autoscaling for variable loads.
- Monitor CPU, Memory, and Disk usage.
- Avoid over-provisioning and idle time.

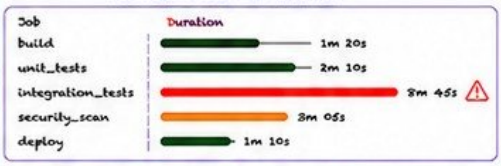
RESOURCE METRICS TO WATCH

Metric	What to Monitor
CPU	High CPU = more compute needed
Memory	High Memory = increase RAM
Disk I/O	High I/O wait = slow storage
Network	High network = optimize artifacts
Runner Queue	Long queue = need more runners

5. BOTTLENECK ANALYSIS

- Find the slowest jobs.
- Analyze wait time vs run time.
- Use pipeline analytics to identify trends.

PIPELINE ANALYTICS EXAMPLE



FINDINGS

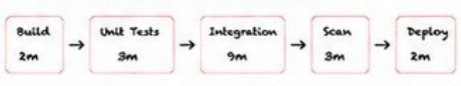
- Integration tests are the bottleneck (8m 45s).
- Focus optimization here for maximum impact.

6. OPTIMIZATION TECHNIQUES

- [x] Cache dependencies and Docker layers.
- [x] Use lightweight Docker images.
- [x] Run jobs in parallel using "needs:".
- [x] Skip unnecessary jobs with "rules:".
- [x] Use artifacts smarter (expire quickly).
- [x] Use fast package mirrors.
- [x] Test in smaller, focused scopes.
- [x] Enable runner autoscaling.
- [x] Use incremental builds and tests.

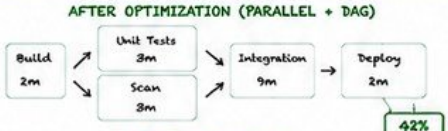
7. BEFORE vs AFTER OPTIMIZATION

BEFORE OPTIMIZATION (SEQUENTIAL)



TOTAL TIME: 19 min

AFTER OPTIMIZATION (PARALLEL + DAG)



TOTAL TIME: 11 min

8. BEST PRACTICES

- [x] Design for parallel execution.
- [x] Keep jobs small and focused.
- [x] Use DAG pipelines.
- [x] Monitor and review pipeline performance weekly.
- [x] Optimize based on real data.
- [x] Automate performance checks.
- [x] Continuously improve.

COMMON MISTAKES

- [x] Too many sequential stages.
- [x] Running jobs that don't impact the outcome.
- [x] Not using cache or artifacts properly.
- [x] Large Docker images.
- [x] Ignoring pipeline analytics.

USEFUL CLI COMMANDS

```
# List pipelines
glab pipeline list
# View pipeline details
glab pipeline view <pipeline-id>
# View job logs
glab pipeline trace <job-id>
# Retry failed jobs
glab pipeline retry --job <job-id>
```

TOOLS & FEATURES

- GitLab Pipeline Analytics
- DAG Pipelines (needs:)
- Caching & Artifacts
- Auto DevOps
- Runner Autoscaling
- Resource Groups
- CI/CD Variables
- Performance Budgets

38. Pipeline Monitoring

KEY TAKEAWAY

- ☑ Monitor pipelines continuously.
- ☑ Measure what matters.
- ☑ Use analytics to improve performance.
- ☑ Detect failures early and reduce MTTR.
- ☑ Data-driven decisions make pipelines faster and reliable.

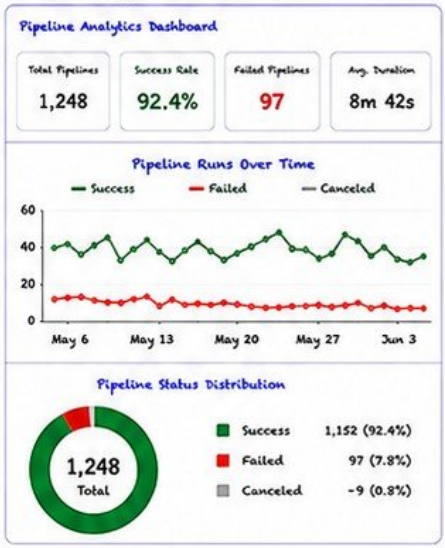
WHAT IS PIPELINE MONITORING?

Pipeline monitoring is the practice of collecting, analyzing, and visualizing pipeline data to understand performance, detect issues, and drive improvements. GitLab provides built-in analytics and integrates with powerful observability tools.

PIPELINE MONITORING OVERVIEW

1. PIPELINE ANALYTICS

- Gain insights into pipeline performance.
- Track trends, identify issues, and optimize.
- Available in GitLab → Analytics.
- Filter by project, branch, user, or time.



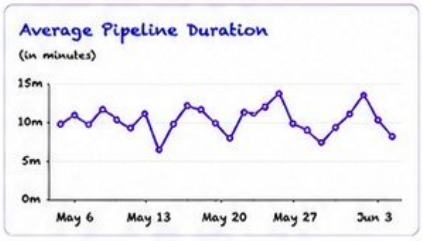
2. JOB METRICS

- Monitor individual job performance.
- Identify slow jobs and resource heavy steps.
- Drill down by job name or stage.

Job Name	Stage	Avg. Duration	Runs	Success Rate	Failures
build	build	2m 15s	1,248	98.1%	23
unit_tests	test	3m 42s	1,248	94.3%	71
integration_tests	test	6m 18s	1,248	90.2%	97
security_scan	test	4m 05s	1,248	95.6%	54
deploy_staging	deploy	1m 48s	1,248	96.4%	45
deploy_production	deploy	2m 30s	612	99.2%	5

3. DURATION ANALYSIS

- Analyze pipeline and job durations.
- Find long-running stages and optimize.
- Compare performance over time.



Top 5 Slowest Jobs (Avg. Duration)

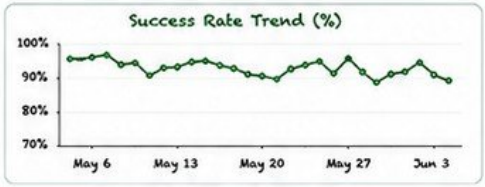
Job Name	Stage	Avg. Duration
integration_tests	test	6m 18s
security_scan	test	4m 05s
unit_tests	test	3m 42s
deploy_production	deploy	2m 30s
build	build	2m 15s

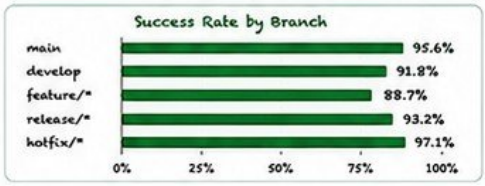
DURATION INSIGHTS

- ☑ Integration tests are the biggest time consumer.
- ☑ Optimize test execution and parallelization.
- ☑ Use caching and artifact reuse to reduce time.

4. SUCCESS RATES

- Track success rates across projects and branches.
- Measure reliability and stability.
- Set targets and improve continuously.

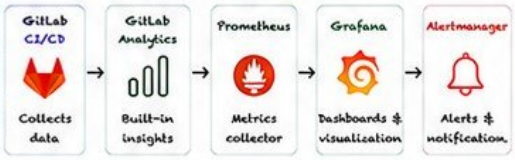




5. FAILURE TRENDS

- Identify failure patterns and root causes.
- Focus on recurring failures.
- Reduce Mean Time To Recovery (MTTR).

6. PIPELINE MONITORING STACK



BEST PRACTICES

- ☑ Define clear metrics and KPIs.
- ☑ Monitor continuously, not just when issues occur.
- ☑ Set up alerts for failures and performance degradation.
- ☑ Use dashboards to share insights with your team.
- ☑ Review trends weekly and act on the data.
- ☑ Automate remediation where possible.

KEY METRICS TO TRACK

- ★ Pipeline Success (Built-in)
- ★ Average Pipeline Duration
- ★ Job Success Rate
- ★ Job Duration (P95)
- ★ Failure Rate
- ★ Mean Time To Recovery (MTTR)
- ★ Deployment Frequency
- ★ Change Failure Rate
- ★ Resource Utilization

COMMON MISTAKES

- ✗ Not monitoring pipelines consistently.
- ✗ Ignoring slow jobs and performance trends.
- ✗ No alerts for failures or degradation.
- ✗ Not analyzing failure patterns.
- ✗ No dashboards or reporting.

USEFUL GITLAB CLI COMMANDS

```
# List pipeline runs
glab pipeline list
# View pipeline details
glab pipeline view <pipeline-id>
# View job details
glab job view <job-id>
# Retry failed job
glab job retry <job-id>
# Cancel pipeline
glab pipeline cancel <pipeline-id>
```

TOOLS & INTEGRATIONS

- GitLab Analytics (Built-in)
- Prometheus + Grafana
- ELK Stack
- Datadog
- New Relic
- Slack / Microsoft Teams
- PagerDuty / Opsgenie
- Sentry
- Loki / Tempo

39. OBSERVABILITY

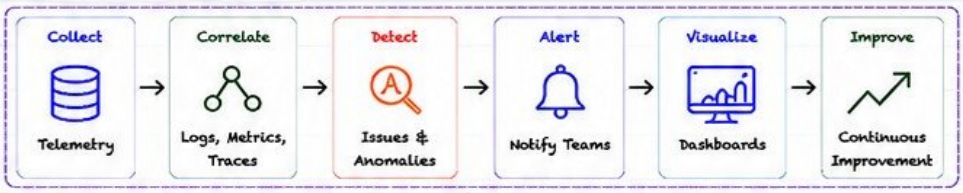
KEY TAKEAWAY

- [x] Observability makes the invisible visible.
- [x] Logs tell you WHAT happened.
- [x] Metrics tell you HOW MUCH and HOW OFTEN.
- [x] Traces tell you WHY and WHERE.
- [x] Alerts inform you BEFORE users complain.
- [x] Dashboards bring everything together.
- [x] Better visibility = Faster detection, faster response, better reliability.

WHAT IS OBSERVABILITY?

Observability is the ability to understand the internal state of your pipelines and systems by collecting and correlating telemetry data. GitLab integrates with powerful observability tools to help you see, understand, and act on what matters most.

OBSERVABILITY OVERVIEW



1. LOGS – WHAT HAPPENED?

- Capture detailed logs from jobs and systems.
- Store, search, and analyze log events.
- Correlate logs with metrics and traces.

LOG EXAMPLE (CI JOB)

```
$ gitlab-runner logs build-job
[12:24:01] Starting job on runner-01
[12:24:02] Checking out code...
[12:24:05] Installing dependencies...
[12:24:18] Running unit tests...
[12:24:42] All tests passed ✅
[12:24:43] Job succeeded in 42s
```

BEST PRACTICES

- [x] Use structured logging (JSON).
- [x] Include timestamps and correlation IDs.
- [x] Log important events, not everything.
- [x] Centralize logs (ELK, Loki, CloudWatch, etc.).

2. METRICS – HOW MUCH & HOW OFTEN?

- Measure performance, usage, and health.
- Track trends and set SLOs / SLIs.
- Power dashboards and alerts.

METRICS EXAMPLE

Metric	Value	Trend (7d)
Pipeline Duration (avg)	8m 42s	
Job Success Rate	92.4%	
Runner Utilization	78%	
CPU Usage (runners)	65%	
Queued Jobs	23	

BEST PRACTICES

- [x] Track RED metrics (Rate, Errors, Duration).
- [x] Define clear SLOs and thresholds.
- [x] Monitor both systems and pipelines.
- [x] Use labels and dimensions consistently.

3. TRACES – WHY & WHERE?

- Trace requests across services and jobs.
- Understand dependencies and latency.
- Find slow calls and root causes.

TRACE EXAMPLE (PIPELINE FLOW)

BEST PRACTICES

- [x] Propagate correlation IDs.
- [x] Instrument key services and jobs.
- [x] Visualize traces (Jaeger, Tempo, Zipkin).
- [x] Focus on latency and dependencies.

ALERTS – INFORM EARLY!

- Detect issues before they impact users.
- Alert the right people on the right channel.
- Reduce MTTR and improve reliability.

ALERT EXAMPLES

- Pipeline Failure Rate > 10% — Alert: #devops-alerts
- Job Duration > 15m — Alert: #performance-alerts
- Runner Offline > 5m — Alert: #infra-alerts

BEST PRACTICES

- [x] Alert on symptoms, not logs.
- [x] Use meaningful thresholds and baselines.
- [x] Avoid alert fatigue (be specific).
- [x] Route alerts based on severity.

5. DASHBOARD INTEGRATION

- Bring logs, metrics, and traces together.
- Monitor pipelines and infrastructure in one place.
- Share insights across teams.

SAMPLE PIPELINE DASHBOARD

BEST PRACTICES

- [x] Build role-based dashboards.
- [x] Show trends, not just numbers.
- [x] Make dashboards actionable.
- [x] Keep dashboards simple and focused.

6. INCIDENT VISIBILITY

- End-to-end visibility for faster response.
- See impact, blast radius, and timeline.
- Improve communication and resolution.

INCIDENT TIMELINE EXAMPLE

12:24	Pipeline #1254 failed
12:25	Alert triggered (Failure Rate > 10%)
12:26	Root cause identified (Test timeout)
12:28	Incident acknowledged
12:31	Fix deployed
12:35	Pipeline recovered
12:36	Incident resolved

BEST PRACTICES

- [x] Correlate logs, metrics, and traces.
- [x] Have runbooks and escalation paths.
- [x] Communicate status clearly.
- [x] Run post-incident reviews.

OBSERVABILITY STACK EXAMPLE

TIP

Good observability doesn't just show you problems. It helps you understand, prioritize, and prevent them. Instrument early. Monitor everything that matters. Visibility today, reliability tomorrow.

COMMON MISTAKES

- [x] Not monitoring pipelines consistently.
- [x] Too many alerts or noisy alerts.
- [x] Missing correlation between signals.
- [x] Ignoring trends and baselines.
- [x] No visibility into dependencies.

USEFUL GITLAB FEATURES

- Pipeline Analytics
- Value Stream Analytics
- Metrics Reports
- Job Logs & Trace Integration
- Alert Integrations (Slack, PagerDuty, Email)
- Incident Management Integration

TOOLS & INTEGRATIONS

- Prometheus + Grafana
- Loki (Logs)
- Tempo / Jaeger (Traces)
- Alertmanager
- Datadog
- New Relic
- Elastic Stack (ELK)
- CloudWatch / Azure Monitor

40. NOTIFICATIONS

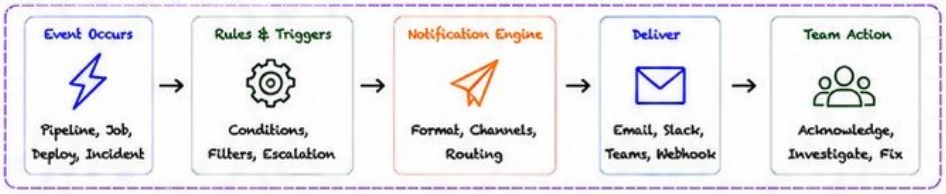
KEY TAKEAWAY

- ☑ Notifications bring the right information to the right people at the right time.
- ☑ Effective notifications reduce MTTR and improve team response.
- ☑ Use multiple channels and escalate when needed.
- ☑ ChatOps + automation = faster action and better collaboration.

WHAT ARE NOTIFICATIONS?

Notifications are automated messages sent to inform teams about pipeline events, deployments, failures, approvals, and incidents. GitLab CI/CD supports multiple channels and integrations to ensure nothing important is missed.

NOTIFICATION FLOW OVERVIEW



EMAIL NOTIFICATIONS

- Send email alerts for important events.
- Good for external users or escalations.
- Configure SMTP in GitLab.

.gitlab-ci.yml (Email Example)

```
notify_email:
  stage: notify
  script: echo "Sending email..."
  when: on_failure
  mail:
    to: devops-team@example.com
    subject: "Pipeline Failed: $CI_PROJECT_PATH"
    body: "Pipeline $CI_PIPELINE_ID failed on $CI_COMMIT_REF_NAME"
```

SLACK NOTIFICATIONS

- Real-time alerts in Slack channels.
- Rich messages with links and context.
- Use Incoming Webhooks.

.gitlab-ci.yml (Slack Example)

```
notify_slack:
  stage: notify
  script:
    - curl -X POST -H 'Content-type:
      application/json' --data "{
      \"text\": \"Pipeline $CI_JOB_STATUS
      in $CI_PROJECT_PATH: <$CI_PIPELINE_URL|View>\"
      }" $SLACK_WEBHOOK_URL
    when: always
  when: always
variables:
  SLACK_WEBHOOK_URL: $SLACK_WEBHOOK_URL
```

MICROSOFT TEAMS

- Post alerts to Teams channels.
- Adaptive cards for rich formatting.
- Use Incoming Webhook URL.

.gitlab-ci.yml (Teams Example)

```
notify_teams:
  stage: notify
  script:
    - curl -H "Content-Type:
      application/json" -d "{
      \"text\": \"Pipeline $CI_JOB_STATUS
      in $CI_PROJECT_PATH",
      \"sections\": [{\"facts\": [
      {\"name\": \"Pipeline\", \"value\":
      \"$CI_PIPELINE_ID\"},
      ]}]}" $TEAMS_WEBHOOK_URL
  when: always
```

WEBHOOKS

- Send HTTP POST requests.
- Integrate with external tools.
- Full control over payload.

.gitlab-ci.yml (Webhook Example)

```
notify_webhook:
  stage: notify
  script:
    - curl -X POST -H 'Content-Type:
      application/json' -d "{
      "event": "pipeline",
      "status": "$CI_JOB_STATUS",
      "project": "$CI_PROJECT_PATH",
      "pipeline_id": "$CI_PIPELINE_ID",
      "url": "$CI_PIPELINE_URL"
      }" $WEBHOOK_URL
  when: always
```

CHATOPS

- Interact with pipelines from chat.
- Trigger, approve, cancel directly.
- Slash commands + bots.

ChatOps Commands (Examples)

```
/gitlab pipeline list
/gitlab pipeline run
/gitlab deploy prod
/gitlab approve
/gitlab cancel 12345
```

Powered by: GitLab Bot / Slash Commands / API

6 INCIDENT NOTIFICATIONS WORKFLOW

WORKFLOW EXAMPLE

1. Pipeline fails in production deploy
2. Slack + Email + Teams alerts sent
3. On-call receives and acknowledges
4. Incident created in PagerDuty/Jira
5. Team investigates and fixes issue
6. Resolution message sent automatically
7. Post-incident report generated

Sample Notification Message (Slack)

```
✅ VERIQTA Pipeline Alert
Project: veriqta-platform
Pipeline: #1254 (FAILED)
Branch: main
Job: deploy_production
Duration: 4m 23s
Commit: a1b2c3d
Author: annfelix
View: https://gitlab.com/veriqta/veriqta-platform/-/pipelines/1254
Time: 2026-07-30 10:45:23 UTC
```

7 BEST PRACTICES

- ☑ Notify only when it matters.
- ☑ Use different channels for different severity.
- ☑ Include actionable context in every message.
- ☑ Avoid notification storms.
- ☑ Escalate if no response.
- ☑ Use templates for consistency.
- ☑ Test notifications regularly.
- ☑ Respect team time and focus.
- ☑ Log and audit all notification events.

8 NOTIFICATION RULE EXAMPLES (.gitlab-ci.yml)

EVENT	WHEN TO NOTIFY	CHANNELS
Pipeline Failed	on_failure	Slack, Email, Teams
Deploy to Production	on_success	Slack, Teams
Job Timeout	always	Slack, Email
Manual Approval	when: manual	Teams, Email
Security Scan Failure	on_failure	Slack, Teams
Infrastructure Alert	always	Webhook, Slack

9 ESCALATION STRATEGY

LEVEL	TRIGGER	ACTION	CHANNEL
L1 - Info	Pipeline Success	Inform team	Slack
L2 - Warning	Job Failed / Flaky	Notify + Monitor	Slack, Email
L3 - Critical	Prod Failed / Outage	Page On-call	Slack, Email, Teams
L4 - No Response	No Ack > 10 min	Escalate	Phone, PagerDuty

10 NOTIFICATION CHECKLIST

- ☑ Correct channels configured
- ☑ Filters and rules defined
- ☑ Message templates updated
- ☑ Escalation paths tested
- ☑ On-call schedule synced
- ☑ Webhook endpoints secured
- ☑ Audit logging enabled

COMMON MISTAKES

- ✗ Sending too many notifications.
- ✗ Not including useful context.
- ✗ No escalation path.
- ✗ Ignoring test notifications.
- ✗ Hardcoding secrets in configs.

USEFUL GITLAB VARIABLES

- $CI_PROJECT_PATH - Project path
- $CI_PIPELINE_ID - Pipeline ID
- $CI_JOB_NAME - Job name
- $CI_JOB_STATUS - Job status
- $CI_PIPELINE_URL - Pipeline URL
- $CI_COMMIT_SHA - Commit SHA
- $GITLAB_USER_LOGIN - User who triggered

TOOLS & INTEGRATIONS

- Slack
- Microsoft Teams
- Email (SMTP)
- PagerDuty
- Opsgenie
- Webhook Integrations
- Jira
- ServiceNow
- VictorOps
- Datadog / New Relic

41. Release Management

KEY TAKEAWAY

- ☑ Manage software releases with consistency and confidence.
- ☑ Use tags and changelogs for clear version tracking.
- ☑ Automate and control release processes.
- ☑ Keep teams and users informed.

WHAT IS RELEASE MANAGEMENT?

Release management is the process of planning, building, testing, and delivering software updates to users. It ensures new features, fixes, and improvements are released safely, reliably, and with full traceability.

RELEASE MANAGEMENT LIFECYCLE

1 RELEASES

- ☑ A release is a set of changes delivered to production.
- ☑ Contains features, fixes, and improvements.
- ☑ Must be tested and approved.
- ☑ Should be small, frequent, and reliable.

RELEASE TYPES

- **Major Release** – large changes, new functionality.
- **Minor Release** – feature updates, improvements.
- **Patch Release** – bug fixes, security fixes.

2 TAGS

- ☑ Tags mark specific versions of code.
- ☑ Commonly used in Git for releases.
- ☑ Tags should be immutable.
- ☑ Use semantic versioning.

SEMANTIC VERSIONING

MAJOR.MINOR.PATCH

- **MAJOR** – incompatible changes
- **MINOR** – new features (backward compatible)
- **PATCH** – bug fixes

3 CHANGELOGS

- ☑ A changelog lists all changes in each release.
- ☑ Helps users and teams understand what changed.
- ☑ Improves transparency and trust.
- ☑ Keeps history of software evolution.

CHANGELOG FORMAT

```
## Version 2.4.0 (2024-06-01)
### ✨ New Features
- Added GitOps support
- Improved pipeline performance
### 🐞 Bug Fixes
- Fixed memory leak
- Resolved login issue
### ⚙ Improvements
- Better error handling
```

4 RELEASE AUTOMATION

- ☑ Automates building, testing, and deployment of releases.
- ☑ Reduces human error.
- ☑ Ensures consistency across environments.
- ☑ Speeds up delivery.

AUTOMATION TOOLS

- Jenkins
- GitHub Actions
- GitLab CI/CD
- Azure DevOps
- Argo CD
- Flux
- CircleCI
- Tekton

5 VERSION MANAGEMENT

- ☑ Tracks and controls all software versions.
- ☑ Uses tags, branches, and changelogs.
- ☑ Avoids version conflicts.
- ☑ Supports rollback and hotfixes.

VERSION MANAGEMENT STRATEGY

1. Use semantic versioning.
2. Tag every release.
3. Maintain a clear changelog.
4. Automate the release pipeline.
5. Monitor and validate after release.

6 BEST PRACTICES

- ☑ Keep releases small and focused.
- ☑ Automate the entire release process.
- ☑ Use semantic versioning consistently.
- ☑ Maintain clear and detailed changelogs.
- ☑ Test before release.
- ☑ Monitor after deployment.
- ☑ Be ready for quick rollback.

REAL WORLD LESSON

Great software isn't just built — it's delivered safely, tracked clearly, and improved continuously.

TIP: Automate everything — build, test, release, and monitor. Release management is the bridge between development and reliable production.

42. GITLAB API

KEY TAKEAWAY

- [x] GitLab API lets you automate almost everything.
- [x] REST API is powerful, versioned, and well documented.
- [x] Use Personal Access Tokens (PAT) or OAuth for authentication.
- [x] Automate pipelines, projects, users, releases, and infrastructure.
- [x] APIs + scripts = consistent, scalable, and reliable DevOps.

WHAT IS GITLAB API?

GitLab API is a RESTful API that allows you to interact with GitLab programatically. You can automate projects, pipelines, users, groups, merge requests, releases, variables, and more.

Base URL (SaaS): https://gitlab.com/api/v4

Base URL (Self-Managed): https://gitlab.example.com/api/v4

API INTERACTION FLOW

Authenticate (Token / OAuth) → Send Request (GET / POST / PUT / DELETE) → GitLab API (Process Request) → JSON Response (Data Returned) → Your Script / Tool (Use Response)

1. REST API BASICS

- REST (Representational State Transfer)
- Uses standard HTTP methods.
- Request and response in JSON.
- All endpoints under /api/v4/

METHOD	PURPOSE	EXAMPLE
GET	Read resources	GET /projects
POST	Create resources	POST /projects
PUT	Update resources	PUT /projects/:id
DELETE	Delete resources	DELETE /projects/:id

2. AUTHENTICATION

- Use Personal Access Token (PAT) for API access.
- Scopes control what the token can do.
- Pass token in header: PRIVATE-TOKEN <token>

```
# Example: Using PAT in curl
curl --header "PRIVATE-TOKEN: $TOKEN" \
  "https://gitlab.com/api/v4/user"
```

COMMON PAT SCOPES

SCOPE	DESCRIPTION	USE CASE
api	Full API access	Automation
read_api	Read-only API access	Monitoring
write_repository	Push code	CI/CD automation
read_repository	Read code	Reporting

3. PIPELINE AUTOMATION

- Trigger pipelines, get status, cancel,
- or retry pipelines.
- Perfect for ChatOps and integrations.

```
# Trigger a pipeline
curl --request POST \
  --header "PRIVATE-TOKEN: $TOKEN" \
  --form ref=main \
  "https://gitlab.com/api/v4/projects/1234/trigger/pipeline"
```

PIPELINE ENDPOINTS

METHOD	ENDPOINT	DESCRIPTION
POST	/projects/:id/trigger/pipeline	Trigger pipeline
GET	/projects/:id/pipelines	List pipelines
GET	/projects/:id/pipelines/:id	Get pipeline
POST	/projects/:id/pipelines/:id/cancel	Cancel pipeline

4. PROJECT AUTOMATION

- Create, update, delete, or manage projects.
- Automate visibility, branches, variables, and more.

```
# Create a new project
curl --request POST \
  --header "PRIVATE-TOKEN: $TOKEN" \
  --form "name=my-project" \
  --form "visibility=private" \
  "https://gitlab.com/api/v4/projects"
```

PROJECT ENDPOINTS

METHOD	ENDPOINT	DESCRIPTION
GET	/projects	List all projects
POST	/projects	Create project
GET	/projects/:id	Get project details
PUT	/projects/:id	Update project
DELETE	/projects/:id	Delete project

5. USER & GROUP AUTOMATION

- Manage users, groups, memberships.
- Automate onboarding and access control.

```
# List users
curl --header "PRIVATE-TOKEN: $TOKEN" \
  "https://gitlab.com/api/v4/users"
# Add user to group
curl --request POST \
  --header "PRIVATE-TOKEN: $TOKEN" \
  --form user_id=45
  "https://gitlab.com/api/v4/groups/67/members"
```

USER & GROUP ENDPOINTS

METHOD	ENDPOINT	DESCRIPTION
GET	/users	List users
GET	/groups	List groups
POST	/groups/:id/members	Add user to group
DELETE	/groups/:id/members/:user_id	Remove user

6. VARIABLES AUTOMATION

- Manage CI/CD variables securely.
- Use for secrets, tokens, configuration.

```
# Create CI/CD variable
curl --request POST \
  --header "PRIVATE-TOKEN: $TOKEN" \
  --form key=API_URL \
  --form value="https://api.example.com" \
  --form protected=true
  "https://gitlab.com/api/v4/projects/1234/variables"
```

VARIABLE ENDPOINTS

METHOD	ENDPOINT	DESCRIPTION
GET	/projects/:id/variables	List variables
POST	/projects/:id/variables	Create variable
PUT	/projects/:id/variables/:key	Update variable
DELETE	/projects/:id/variables/:key	Delete variable

7. RELEASE AUTOMATION

- Create releases and manage release assets.
- Automate versioning and changelogs.

```
# Create a release
curl --request POST \
  --header "PRIVATE-TOKEN: $TOKEN" \
  --form tag_name="v1.2.3" \
  --form name="Release v1.2.3" \
  --form description="Bug fixes and improvements"
  "https://gitlab.com/api/v4/projects/1234/releases"
```

RELEASE ENDPOINTS

METHOD	ENDPOINT	DESCRIPTION
GET	/projects/:id/releases	List releases
POST	/projects/:id/releases	Create release
GET	/projects/:id/releases/:tag_name	Get release
DELETE	/projects/:id/releases/:tag_name	Delete release

8. API SCRIPTING EXAMPLE (BASH)

- Simple script to trigger pipeline and get status.

```
#!/bin/bash
TOKEN="YOUR_TOKEN_HERE"
PROJECT_ID="1234"
REF="main"

# Trigger pipeline
echo "Triggering pipeline..."
curl --silent --request POST \
    --header "PRIVATE-TOKEN: $TOKEN" \
    --form ref="$REF" \
    "https://gitlab.com/api/v4/projects/$PROJECT_ID/trigger/pipeline"

# Get latest pipeline ID
PIPELINE_ID=$(curl --silent --header "PRIVATE-TOKEN: $TOKEN" \
   "https://gitlab.com/api/v4/projects/$PROJECT_ID/pipelines?per_page=1" \
    | jq '.[0].id')
echo "Latest Pipeline ID: $PIPELINE_ID"

# Get pipeline status
curl --silent --header "PRIVATE-TOKEN: $TOKEN" \
   "https://gitlab.com/api/v4/projects/$PROJECT_ID/pipelines/$PIPELINE_ID" \
    | jq '.status, .web_url'
#requires: jq (JSON parser)
```

9. BEST PRACTICES

- [x] Use Personal Access Tokens with minimal scopes.
- [x] Store tokens in secure secrets managers.
- [x] Use API version: /api/v4/
- [x] Handle errors and rate limits gracefully.
- [x] Log API actions for audit and traceability.
- [x] Use pagination for large lists.
- [x] Keep scripts idempotent and safe.
- [x] Test API calls in a sandbox project first.

RATE LIMITING (GITLAB.COM)

GitLab.com API is rate limited.

- 600 requests per minute per user
- 429 Too Many Requests if exceeded
- Check response headers:
  - RateLimit-Remaining
  - RateLimit-Reset

TIP

APIs are force multipliers.
Automate the boring, focus on building.
Everything in the UI can be automated.
Think API-first, automate everything!

COMMON ERRORS

- ✗ 401 Unauthorized → Invalid or missing token
- ✗ 403 Forbidden → Token lacks required scope
- ✗ 404 Not Found → Wrong endpoint or ID
- ✗ 429 Rate Limit → Slow down requests
- ✗ 500 Server Error → GitLab server issue

USEFUL TOOLS

- curl – Make HTTP requests
- jq – Parse JSON responses
- httpie – Friendly API client
- Postman – Test APIs
- GitLab API Docs – https://docs.gitlab.com/ee/api/

TOOLS & INTEGRATIONS

- GitLab CI/CD
- Slack / Microsoft Teams
- Jenkins / Ansible / Terraform
- Datadog / Prometheus
- ChatOps / Webhooks

43. Pipeline Templates

KEY TAKEAWAY

- Pipeline templates make CI/CD consistent, reusable, and easy to maintain.
- Define once, use everywhere.
- Includes all modular pipeline building blocks.
- Components package the logic for reuse and versioning.
- Shared libraries + templates enforce enterprises standards at scale.

WHAT ARE PIPELINE TEMPLATES?

Pipeline templates are reusable CI/CD definitions that provide a standard structure for jobs, stages, variables, and configurations. They promote consistency, reduce duplication, and enforce enterprise best practices.

TEMPLATE BENEFITS

Consistency	Reusability	Maintainability	Governance	Speed
Same standards everywhere	Build once, use many times	Update in one place	Enforce policies and compliance	Faster pipeline creation

1. REUSABLE TEMPLATES

- Define common pipeline patterns.
- Store in a central repository.
- Reference using include.
- Keep DRY (Don't Repeat Yourself).

Example: Include a template

```
include:
  - project: 'veriqta/ci-templates'
    file: '/templates/node-app.yml'
    ref: 'v1.2.0'
```

2. INCLUDES

- Include external YAML files.
- Combine multiple templates.
- Supports local, project, and remote includes.

Include Types

```
include:
  - local: 'ci/common.yml'
  - project: 'group/templates'
    file: '/security/sast.yml'
    ref: 'v1.0.0'
  - remote: 'https://gitlab.com/org/pipeline-base/-/raw/main/base.yml'
  - template: 'Jobs/Code-Quality.gitlab-ci.yml'
```

3. COMPONENTS

- Components package jobs, scripts, and logic.
- Versioned and reusable across projects.
- Share via includes or component registry.

Component Example

```
component: deploy-aws
version: 1.1.0
inputs:
  environment:
    type: string
  region:
    type: string
  image:
    type: string
```

deploy-aws

Build
Tests
Push
Deploy

4. SHARED LIBRARIES

- Store scripts, functions, and utilities.
- Use version control for updates.
- Promotes reuse and standardization.

Example Library Structure

```
ci-Libs/
├── bash/
│   ├── utils.sh
│   └── aws.sh
├── python/
│   └── helpers.py
└── templates/
    ├── security.yml
    └── deploy.yml
```

Usage in Pipeline

```
before_script:
- source ci-libs/bash/utils.sh
- setup_environment
- install_dependencies
```

5. ENTERPRISE STANDARDIZATION

- Enforce organization-wide standards.
- Define templates for security, quality, build, test, deploy.
- Ensure compliance and governance.

Standard Templates Examples

- Security Scan Template
- Code Quality Template
- Build & Test Template
- Deploy Template
- Release Template

6. TEMPLATE VARIABLES & INPUTS

- Make templates flexible.
- Accept parameters and variables.
- Use inputs for components.

Example: Variables in Template

```
variables:
  NODE_VERSION: '20'
  ENVIRONMENT: 'production'
```

Example: Inputs in Component

```
inputs:
  environment:
    type: string
    default: 'dev'
  image:
    type: string
```

7. TEMPLATE VERSIONING

- Version templates to control updates.
- Use semantic versioning (SemVer).
- Pin versions for stability.

Best Practices

- ☑ Always version your templates
- ☑ Pin to a specific version
- ☑ Test before upgrading
- ☑ Document changes
- ☑ Use backward-compatible updates

v1.2.0

TEMPLATE USAGE WORKFLOW

Create Template → Store in Repo → Include in Pipeline → Run Pipeline → Consistent Results

9. BEST PRACTICES

- ☑ Keep templates small and focused.
- ☑ Use meaningful names and documentation.
- ☑ Validate templates with CI before publishing.
- ☑ Secure templates and control access.
- ☑ Monitor usage and gather feedback.
- ☑ Automate updates and deprecations.

COMMON MISTAKES

- ✗ Duplicate logic instead of using templates
- ✗ Hardcoding values in templates
- ✗ Not versioning templates
- ✗ Overly complex templates
- ✗ Ignoring backward compatibility

USEFUL COMMANDS

```
# Lint a template
curl --header "PRIVATE-TOKEN: $TOKEN" \
  https://gitlab.com/api/v4/ci/lint \
  --form "content=$(cat .gitlab-ci.yml)"
# Get project templates
curl --header "PRIVATE-TOKEN: $TOKEN" \
  https://gitlab.com/api/v4/projects/:id/ci/configs
```

TOOLS & INTEGRATIONS

- GitLab CI/CD
- GitLab Component Registry
- Terraform / Ansible
- Security Scanning Tools
- Code Quality Tools

44. Enterprise Pipeline Design

KEY TAKEAWAY

- ☑ Design pipelines as modular components, not monoliths.
- ☑ Build for reuse, not one-off projects.
- ☑ Make pipelines easy to maintain and easy to adapt.
- ☑ Separate responsibilities clearly across stages.
- ☑ Document design decisions and architecture patterns.

WHAT IS ENTERPRISE PIPELINE DESIGN?

Enterprise pipeline design is the process of architecting CI/CD pipelines with a focus on scalability, reusability, maintainability, and clear separation of responsibilities. It ensures consistency, reduces duplication, and supports long-term evolution.

ENTERPRISE PIPELINE DESIGN PRINCIPLES

Modular	Reusable	Maintainable	Separation	Decisions
Break pipelines into small, independent modules	Design once, reuse across projects	Easy to update, debug, and extend	Each stage has one clear responsibility	Document architecture and standards

1 MODULAR PIPELINES

- Break large pipelines into independent modules.
- Each module has a single responsibility.
- Modules communicate via artifacts and variables.
- Easy to test, reuse, and maintain.

Example: Modular Pipeline Structure

Build Module → Test Module → Scan Module → Deploy Module

2 REUSABILITY

- Use templates and includes to avoid duplication.
- Centralize common logic.
- Share components via includes or libraries.
- Enforce standards across all projects.

Example: Reusable Template

```
# .gitlab-ci.yml
include:
  - project: 'ci/pipeline-templates'
    file: '/templates/build.yml'
    ref: 'v2.1.0'
variables:
  ENVIRONMENT: 'production'
  VERSION: '$CI_COMMIT_TAG'
```

MAINTAINABILITY

- Keep pipelines simple and consistent.
- Use meaningful names and comments.
- Avoid hardcoding values.
- Make updates in one place and propagate.

Maintainability Best Practices

- ☑ Use variables for configuration
- ☑ Keep scripts small and focused
- ☑ Add clear comments
- ☑ Use versioned templates
- ☑ Monitor and refactor regularly

SEPARATION OF RESPONSIBILITIES

- Each stage or job should do one thing.
- Build ≠ Test ≠ Scan ≠ Deploy.
- Use stages to define logical boundaries.
- Improves reliability and troubleshooting.

Example: Responsibilities by Stage

STAGE	RESPONSIBILITY
Build	Compile code, build artifacts
Test	Run unit, integration, and e2e tests
Scan	Security scan, code quality scan
Package	Package and publish artifacts
Deploy	Deploy to target environment
Notify	Send notifications and reports

ARCHITECTURE DECISIONS

- Document important pipeline decisions.
- Capture the why, not just the what.
- Helps onboarding and future evolution.
- Review decisions during changes.

Example: Architecture Decision Record (ADR)

ADR-001: Use Modular Pipelines

Status: Accepted
Context: Pipelines were hard to maintain and reuse.
Decision: Break pipelines into modular components using includes.
Consequences: Easier maintenance, reusability, and scalability.

ENTERPRISE PIPELINE ARCHITECTURE

Global Layer	Company-wide templates, variables, and standards
Project Layer	Project-specific configs and customizations
Pipeline Layer	Reusable includes, components, and templates
Execution Layer	Runners, environments, and execution nodes
Artifact Layer	Artifacts, containers, reports, and packages

7 PIPELINE DESIGN PATTERNS

Monorepo Pattern
Single repo, multiple projects or services. Use directory-based rules and components.

Microservices Pattern
Each service has its own pipeline using shared templates and components.

Environment Pattern
Separate environments (dev, staging, prod) with environment-specific rules.

Compliance Pattern
Enforce scanning, approvals, and policies at every stage for compliance requirements.

ENTERPRISE DESIGN CHECKLIST

- ☑ Pipeline is modular and reusable
- ☑ Uses templates and includes
- ☑ Stages have clear responsibilities
- ☑ Variables are centralized
- ☑ Naming is consistent and meaningful
- ☑ Documentation is available
- ☑ Security and compliance are enforced
- ☑ Architecture decisions are recorded
- ☑ Easily scalable and maintainable

TIP

Great pipeline design is invisible when it works and invaluable when it scales.

COMMON MISTAKES

- ✗ Monolithic pipelines with too many jobs
- ✗ Hardcoded values and duplicated scripts
- ✗ No documentation of decisions
- ✗ Mixing responsibilities in one job
- ✗ Ignoring reusability and standards

USEFUL COMMANDS

```
# Lint pipeline configuration
curl --header "PRIVATE-TOKEN: $TOKEN" \
  https://gitlab.com/api/v4/ci/lint
# Get pipeline configuration
curl --header "PRIVATE-TOKEN: $TOKEN" \
  https://gitlab.com/api/v4/projects/:id/pipeline/:id
```

TOOLS & INTEGRATIONS

- GitLab CI/CD
- GitLab Component Registry
- Terraform / Ansible
- Security Scanning Tools
- Code Quality Tools

45. MONOREPO CI/CD

KEY TAKEAWAY

- ☑ Monorepos simplify code management at scale.
- ☑ Run only what changed (selective builds).
- ☑ Understand dependencies to avoid unnecessary work.
- ☑ Optimize pipelines for speed, cost, and reliability.

WHAT IS MONOREPO CI/CD?

Monorepo CI/CD is the practice of building, testing, and deploying multiple projects from a single repository. It uses smart detection, dependency analysis, and pipeline optimization to run only what changed and deliver faster with lower cost.

MONOREPO CI/CD BENEFITS

Unified Codebase — One repo for all projects → **Faster CI/CD** — Build only what changed → **Better Consistency** — Shared standards and tooling → **Lower Cost** — Fewer pipelines and resources → **Easier Scaling** — Add projects with ease

1 REPOSITORY ORGANIZATION

- Organize by domain, product, or capability.
- Keep clear boundaries between projects.
- Share common code in libraries.
- Use consistent structure across teams.

EXAMPLE STRUCTURE

```
/ (root)
├── apps/
│   ├── user-service/
│   ├── order-service/
│   └── payment-service/
├── libs/
│   ├── auth-lib/
│   ├── common-utils/
│   └── db-client/
├── infra/
├── .gitlab/
└── README.md
```

2 SELECTIVE BUILDS

- Detect changes and run only affected jobs.
- Save time and compute resources.
- Use path filters and dependency graphs.
- Support incremental and parallel builds.

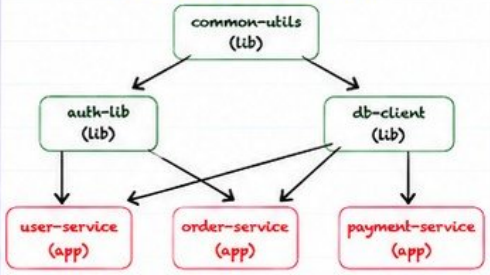
EXAMPLE: PATH FILTER

```
build:user-service:
  rules:
    - changes:
        - apps/user-service/**
        - libs/auth-lib/**

build:order-service:
  rules:
    - changes:
        - apps/order-service/**
        - libs/**
```

3 DEPENDENCY GRAPHS

- Map project dependencies automatically.
- Understand impact of changes.
- Build in the correct order.
- Prevent broken or incomplete builds.

EXAMPLE DEPENDENCY GRAPH



PIPELINE OPTIMIZATION

- Use caching to speed up builds.
- Run jobs in parallel where possible.
- Avoid redundant work.
- Reuse build artifacts across jobs.

OPTIMIZATION TECHNIQUES

- ☑ Enable cache for dependencies
- ☑ Use parallel matrix builds
- ☑ Reuse artifacts across stages
- ☑ Limit runners and concurrency
- ☑ Skip unchanged dependencies

5 CI/CD WORKFLOW EXAMPLE

Detect Changes — Identify modified files & projects → **Analyze Impact** — Build dependency graph → **Run Pipelines** — Execute only affected jobs → **Test & Quality** — Run tests, scans, and validations → **Deploy** — Deploy changed services

EXAMPLE: DETECT CHANGES SCRIPT

```bash
#!/bin/bash
# Get changed files
CHANGED_FILES=$(git diff --name-only $CI_MERGE_REQUEST_TARGET_BRANCH_NAME...)

# Check if user-service changed
if echo "$CHANGED_FILES" | grep -q "apps/user-service/"; then
    echo "user-service changed - running pipeline"
    export RUN_USER_SERVICE=true
else
    echo "user-service not changed - skipping"
    export RUN_USER_SERVICE=false
fi
```

EXAMPLE: CACHE CONFIG

```yaml
cache:
  key: "$CI_COMMIT_REF_SLUG"
  paths:
    - node_modules/
    - .m2/repository/
    - .gradle/
  policy: pull-push
```

BEST PRACTICES

- ☑ Keep repository structure clean and documented.
- ☑ Define clear boundaries between projects.
- ☑ Automate change detection and impact analysis.
- ☑ Use shared pipelines and templates.
- ☑ Monitor pipeline performance continuously.
- ☑ Review dependency graph regularly.
- ☑ Keep pipelines fast, reliable, and deterministic.

COMMON PITFALLS

- ✗ Running all jobs for every change
- ✗ Ignoring dependency relationships
- ✗ Poor repository structure
- ✗ Large, slow pipelines
- ✗ No caching or artifact reuse
- ✗ Over-complicated pipeline logic

USEFUL TOOLS

- GitLab CI/CD
- Nx
- Turborepo
- Lerna
- Changesets
- Bazel
- Rush
- GraphQL (dependency APIs)
- SonarQube
- Snyk
- Dependabot

TIP

A great monorepo pipeline runs only what matters, when it matters, and gives you fast, reliable delivery every time.

46. MICROSERVICES CI/CD

KEY TAKEAWAY

- ☑ Each service has its own pipeline and lifecycle.
- ☑ Services are built, tested, and deployed independently.
- ☑ Share infrastructure, not code or deployments.
- ☑ Orchestrate services safely in production.

WHAT IS MICROSERVICES CI/CD?

Microservices CI/CD is the practice of building, testing, and deploying each service independently while integrating with other services and shared infrastructure. It enables fast delivery, high resilience, and scalable operations.

MICROSERVICES CI/CD BENEFITS

Service Isolation — Changes are contained → **Independent Deployments** — Deploy services independently → **Loose Coupling** — Reduce impact between services → **Lower Risk** — Smaller blast radius → **Faster Delivery** — Ship features with speed

1 SERVICE ISOLATION

- Each microservice has its own repository.
- Build, test, and deploy independentty.
- Isolation reduces risk and dependencies.
- Failures in one service don't block others.

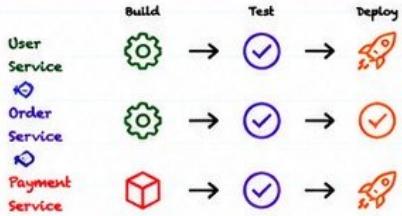
EXAMPLE STRUCTURE

```
/microservices
  ├── user-service/
  │   ├── src/
  │   ├── Dockerfile
  │   └── .gitlab-ci.yml
  ├── order-service/
  │   ├── src/
  │   ├── Dockerfile
  │   └── .gitlab-ci.yml
  ├── payment-service/
  │   ├── src/
  │   ├── Dockerfile
  │   └── .gitlab-ci.yml
  ├── shared-libs/
      ├── common/
```

2 INDEPENDENT DEPLOYMENTS

- Each service has its own pipeline.
- Deploy without waiting for other services.
- Use semantic versioning for each service.
- Rollback only the affected service.

INDEPENDENT DEPLOYMENT FLOW



3 SHARED INFRASTRUCTURE

- Share clusters, networks, databases, and tools.
- Use namespaces, IAM, and policies for isolation.
- Shared observability, logging, and security.
- Reduce cost while maintaining isolation.

SHARED INFRASTRUCTURE LAYERS

- **APPLICATION LAYER** — Microservices (Containers / Pods)
- **PLATFORM LAYER** — Kubernetes, Service Mesh, Ingress
- **DATA LAYER** — Databases, Caches, Message Brokers
- **OPERATIONS LAYER** — Monitoring, Logging, Security, CI/CD

4 SERVICE ORCHESTRATION

- Orchestrate multiple services together.
- Manage dependencies and startup order.
- Use health checks and readiness probes.
- Implement retries, timeouts, and circuit breakers.

ORCHESTRATION PATTERNS

- **API Gateway** — Route requests to appropriate services
- **Service Mesh (Istio / Linkerd)** — Traffic management, security, observability
- **Event Driven** — Use events for loose coupling
- **Saga Pattern** — Manage distributed transactions

5 MICROSERVICES CI/CD PIPELINE EXAMPLE

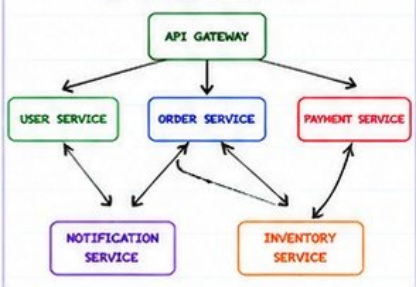
EXAMPLE: .GITLAB-CI.YML FOR A SERVICE

```yaml
stages:
  - build
  - test
  - deploy
variables:
  IMAGE: $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA

build:
  stage: build
  script:
    - docker build -t $IMAGE .
    - docker push $IMAGE
test:
  stage: test
  script:
    - docker run --rm $IMAGE npm test
deploy:
  stage: deploy
  script:
    - kubectl set image deployment/user-service user-service=$IMAGE -n production
    - kubectl rollout status deployment/user-service -n production
  environment:
    name: production
    url: https://user-service.yourdomain.com
```

6 SERVICE DEPENDENCIES MANAGEMENT

- Define and document service dependencies.
- Use contracts and APIs for communication.
- Version APIs and maintain compatibility.
- Automate integration tests between services.

EXAMPLE: DEPENDENCY MAP



7 BEST PRACTICES

- ☑ One service, one repository, one pipeline.
- ☑ Keep services small and focused.
- ☑ Automate testing at every stage.
- ☑ Use infrastructure as code.
- ☑ Monitor each service independently.
- ☑ Implement proper logging and tracing.
- ☑ Use blue-green or canary deployments.
- ☑ Continuously optimize for speed and reliability.

COMMON PITFALLS

- ✗ Tight coupling between services
- ✗ Shared databases between services
- ✗ Big monolithic pipelines
- ✗ Poor API versioning
- ✗ Ignoring service dependencies
- ✗ No integration testing
- ✗ Manual deployments
- ✗ No observability or tracing

USEFUL COMMANDS

```
# Build and push image
docker build -t $IMAGE .
docker push $IMAGE

# Deploy to Kubernetes
kubectl apply -f k8s/
kubectl rollout status deployment/<service>

# Check service
kubectl get svc -n production
kubectl logs deployment/<service> -n production
```

TIP

Design for independence, but test for integration. Great microservices CI/CD delivers speed without sacrificing stability.

47. GitOps with GitLab

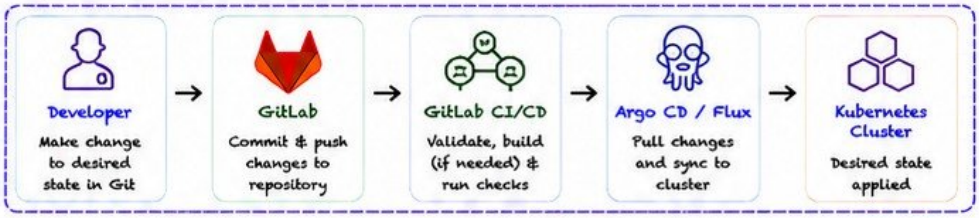
KEY TAKEAWAY

- [x] Git is the single source of truth.
- [x] Declarative configs define the desired state.
- [x] Automate deployments safely.
- [x] Detect drift and self-heal.
- [x] Keep production reliable and predictable.

WHAT IS GITOPS WITH GITLAB?

GitOps is an operational model where Git is the single source of truth for infrastructure and application configurations. GitLab provides version control, CI/CD, and automation to deliver continuous, declarative, and reliable deployments through tools like Argo CD or Flux.

GITOPS WORKFLOW OVERVIEW



1 GITOPS PRINCIPLES

- [x] Declarative: Define desired state in Git.
- [x] Versioned: All changes tracked and auditable.
- [x] Automated: Changes applied automatically.
- [x] Observable: Full visibility into state and health.
- [x] Secure: Git access controls everything.

2 GITOPS DESIRED STATE

- Desired state is stored as code in Git.
- Includes Kubernetes manifests, Helm charts, Kustomize overlays, policies, and secrets.
- The cluster state should always match what is in Git.

EXAMPLE: DESIRED STATE

```
gitops-repo/
├── clusters/
│   ├── production/
│   │   ├── namespace.yaml
│   │   ├── deployment.yaml
│   │   ├── service.yaml
│   │   ├── ingress.yaml
│   │   └── kustomization.yaml
├── apps/
│   └── payment-service/
│       └── values.yaml
```

3 DRIFT DETECTION

- GitOps tools continuously compare live state with desired state.
- Any difference is detected as drift.
- Drift is reported, alerted, and corrected automatically.

DRIFT DETECTION EXAMPLE

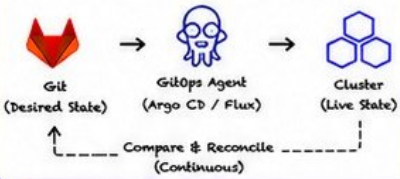
DESIRED STATE (IN GIT)		LIVE STATE (IN CLUSTER)
replicas: 3	≠	replicas: 2
image: app:v1.2.0		image: app:v1.1.0
resources:		resources:
cpu: 500m		cpu: 500m
memory: 256Mi		memory: 256Mi

STATUS: DRIFT DETECTED

4 CONTINUOUS RECONCILIATION

- GitOps agents continuously reconcile the cluster to the desired state.
- If drift is found, it is automatically corrected.
- Ensures self-healing, reliability, and consistency.

RECONCILIATION LOOP



5 GITOPS TOOLS WITH GITLAB

TOOL	DESCRIPTION
Argo CD	Declarative CD tool. Pulls from Git and syncs to Kubernetes.
Flux CD	Kubernetes-native GitOps toolset with controllers.
GitLab CI/CD	Runs tests, validates configs, and triggers GitOps sync.
Helm	Package manager for Kubernetes applications.
Kustomize	Overlay and customize manifests per environment.

6 GITOPS REPOSITORY LAYOUT

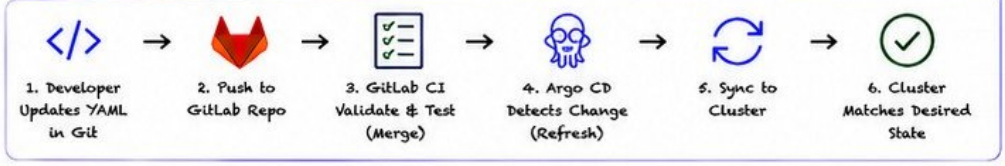
```
gitops-repo/
├── clusters/
│   ├── dev/
│   ├── staging/
│   └── production/
├── apps/
│   ├── user-service/
│   ├── order-service/
│   └── payment-service/
├── base/              # base manifests
├── overlays/          # env overlays
└── policies/          # OPA, Kyverno, etc.
```

SINGLE SOURCE OF TRUTH

7 GITOPS BEST PRACTICES

- [x] Keep Git repository clean and modular.
- [x] Use environments: dev, staging, prod.
- [x] Protect main branch and use MR approvals.
- [x] Validate manifests in CI before merge.
- [x] Use RBAC and least privilege access.
- [x] Enable notifications and dashboards.
- [x] Document processes and runbooks.
- [x] Continuously monitor and audit changes.

8 EXAMPLE: END-TO-END GITOPS FLOW



COMMON PITFALLS

- ✗ Manual changes in cluster (causes drift)
- ✗ Long-lived branches
- ✗ Skipping validation in CI
- ✗ Secrets in plain text
- ✗ Overly complex monorepos

USEFUL COMMANDS (ARGO CD)

```
argocd app list
argocd app get <app-name>
argocd app sync <app-name>
argocd app diff <app-name>
argocd app history <app-name>
```

USEFUL COMMANDS (FLUX)

```
flux get all
flux reconcile source git <name>
flux reconcile kustomization <name>
flux diff kustomization <name>
flux suspend kustomization <name>
```

TIP

GitOps works best when Git is treated as the source of truth and the cluster is treated as automatically disposable.

48. Production Troubleshooting

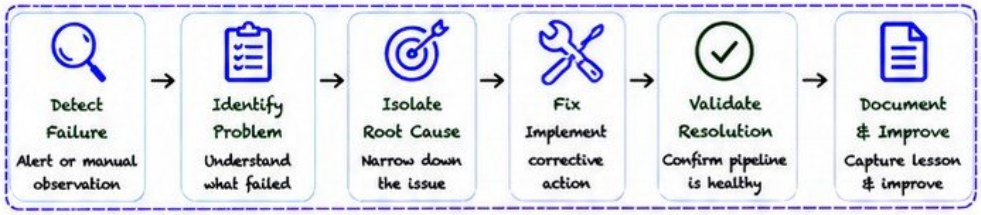
KEY TAKEAWAY

- ☑ Troubleshooting is faster when you follow a process.
- ☑ Identify → Isolate → Investigate → Fix → Validate.
- ☑ Logs, metrics, and context are your best friends.
- ☑ Document the root cause and improve the pipeline.

WHAT IS PRODUCTION TROUBLESHOOTING?

Production troubleshooting in GitLab CI/CD is the systematic process of identifying, isolating, and resolving issues that cause pipeline failures, deployment problems, or degraded delivery. The goal is to restore reliability quickly and prevent recurrence.

TROUBLESHOOTING PROCESS OVERVIEW



FAILED PIPELINES

- Jobs fail due to script errors, compile errors, test failures, or misconfigurations.
- Always check job logs first.
- Review recent changes and merge requests.

COMMON CAUSES

- Script errors
- Tests failing
- Missing dependencies
- Wrong environment variables
- Artifact not found

QUICK CHECKS

- Review job logs
- Check .gitlab-ci.yml changes
- Validate variables & secrets
- Re-run pipeline

RUNNER FAILURES

- Runner cannot pick up or execute jobs.
- Causes delays or complete pipeline stops.
- Check runner health and configuration.

COMMON CAUSES

- Runner not online
- Runner system overload
- Outdated runner version
- Wrong tags or configuration
- Disk space or permission issues

QUICK CHECKS

- Check runner status
- Review runner logs
- Validate tags & config
- Check system resources

NETWORK ISSUES

- Network problems break connectivity to GitLab, registries, or cloud services.
- Causes timeouts, download failures, or unreachable services.

COMMON CAUSES

- DNS resolution failure
- Firewall blocking
- Proxy misconfiguration
- Intermittent connectivity
- TLS/SSL handshake issues

QUICK CHECKS

- Ping & traceroute
- DNS lookup (nslookup)
- Check firewall rules
- Test external endpoints

REGISTRY FAILURES

- Cannot push or pull images from Docker registry.
- Leads to build or deploy failures.
- Authentication or permission issues are common.

COMMON CAUSES

- Invalid credentials
- Token expired
- Insufficient permissions
- Registry outage
- Image not found

QUICK CHECKS

- docker login test
- Check credentials / token
- Verify image name & tag
- Check registry status page

DEBUGGING WORKFLOW

- Follow a structured workflow to reduce MTTR.
- Collect evidence, analyze, and resolve systematically.
- Validate fix and prevent recurrence.

DEBUGGING FLOW

Reproduce Issue
↓
Collect Logs & Evidence
↓
Analyze Root Cause
↓
Implement Fix
↓
Validate & Monitor
↓
Document & Improve

USEFUL LOG LOCATIONS

Job Logs	: Job → View raw log
Pipeline Logs	Pipeline → Jobs → Logs
Runner Logs	: /var/log/gitlab-runner/
Server Logs	/var/log/gitlab/*.log
Container Logs	docker logs <container_id>

EXAMPLE: DEBUGGING FAILED JOB

```
# 1. Check job logs
$ echo "Check logs in GitLab UI"
# 2. Reproduce locally (if possible)
$ git checkout <commit>
$ docker run --rm -v $PWD:/build <image> bash
# 3. Check environment variables
env | sort
# 4. Validate dependencies
$ npm install --verbose
# 5. Run script manually
$ npm run build
# 6. Fix issue and commit
$ git commit -m "fix: resolve build issue"
$ git push
$
```

COMMON CLI COMMANDS

- Check runner status
  `$ gitlab-runner list`
- Check runner config
  `$ gitlab-runner verify`
- Test connectivity
  `$ curl -I https://gitlab.com`
- Check registry auth
  `$ docker login registry.gitlab.com`
- Check pipeline via API
  `$ curl --header "PRIVATE-TOKEN: <token>"\ "https://gitlab.com/api/v4/projects/<id>/pipelines"`

BEST PRACTICES

- ☑ Always read logs carefully.
- ☑ Reproduce issue locally when possible.
- ☑ Change one thing at a time.
- ☑ Document root cause and resolution.
- ☑ Improve pipeline after every incident.
- ☑ Monitor runners and infrastructure.

COMMON PITFALLS

- ✕ Ignoring pipeline logs
- ✕ Assuming it's a GitLab issue
- ✕ Not reproducing locally
- ✕ Hardcoding values in scripts
- ✕ Not monitoring runner health
- ✕ Not documenting incidents

USEFUL TOOLS

- GitLab CI/CD
- GitLab Runner
- Docker / BuildKit
- kubectl
- curl / wget
- jq (JSON parser)
- nc / telnet
- dig / nslookup
- traceroute
- htop / df / free

TIP

A calm, repeatable process beats panic. Good troubleshooting skills save hours and protect trust.

49. Common Production Failures

KEY TAKEAWAY

- ☑ Failures are normal in production.
- ☑ The goal is fast detection, correct diagnosis, and safe recovery.
- ☑ Understand patterns to prevent recurrence.
- ☑ Root cause, not quick fix.
- ☑ Document, improve, automate.

WHAT ARE COMMON PRODUCTION FAILURES?

Production failures in GitLab CI/CD pipelines can happen at any stage of the delivery process. Knowing the common failure patterns helps you respond faster, fix the real issue, and prevent it from happening again.

COMMON FAILURE CATEGORIES OVERVIEW

Stuck Jobs — Jobs don't start or don't finish → **Artifact Failures** — Artifacts missing, expired, or corrupted → **Cache Corruption** — Cache invalid or corrupted causing build issues → **Permission Issues** — Access denied or insufficient permissions → **Deployment Failures** — Deployment fails or rolls back → **Root Cause Analysis** — Find true cause & prevent recurrence

1 STUCK JOBS

- Job stays in pending or running state for too long.
- Runner unavailable or overloaded.
- Resource limits too low.
- External service slow or down.

COMMON CAUSES

- No available runners
- Runner offline
- Job timeout too high
- Hanging scripts
- Deadlocks in processes

QUICK CHECKS

- Check runner status
- Review job logs
- Check job timeout
- Look for hanging commands
- Verify resource usage

2 ARTIFACT FAILURES

- Artifacts missing or not found.
- Upload failed or expired.
- Wrong paths or permissions.
- Storage limits exceeded.

COMMON CAUSES

- Wrong artifact path
- Job failed before upload
- Artifact expiration
- Storage full
- Network interruption

QUICK CHECKS

- Check artifact path
- Review job logs
- Verify expiration settings
- Check storage usage
- Confirm artifact upload

3 CACHE CORRUPTION

- Cache restored but corrupted.
- Build fails due to invalid cache.
- Incompatible cache versions.
- Partial or interrupted cache uploads.

COMMON CAUSES

- Cache key not unique
- Partial upload
- Incompatible dependencies
- Shared cache conflicts
- Storage corruption

QUICK CHECKS

- Clear cache and retry
- Review cache key
- Check dependency versions
- Verify runner disk health
- Use fallback cache key

4 PERMISSION ISSUES

- Access denied errors.
- Cannot read/write files.
- Insufficient token scopes.
- Protected resources blocked.

COMMON CAUSES

- Wrong file permissions
- Protected branch rules
- Expired or invalid tokens
- Missing role access
- Runner permission limits

QUICK CHECKS

- Verify token scopes
- Check file permissions
- Review protected rules
- Validate user access
- Check runner permissions

5 DEPLOYMENT FAILURES

- Deployment job fails.
- Service not available.
- Health check failures.
- Rollback triggered.

COMMON CAUSES

- Invalid configuration
- Service unhealthy
- Kubernetes failure
- Image pull failure
- Environment mismatch

QUICK CHECKS

- Review deployment logs
- Check service health
- Verify manifests
- Check image availability
- Validate environment vars

6 ROOT CAUSE ANALYSIS (RCA)

A structured approach to finding the REAL cause.

1. **Collect** — Gather all evidence and logs
2. **Identify** — Identify what actually failed
3. **Analyze** — Find why it happened
4. **Root Cause** — Find the real underlying cause
5. **Correct** — Fix the issue permanently
6. **Document** — Document and share lessons

RCA QUESTIONS TO ASK

- ☑ What was the expected outcome?
- ☑ What actually happened?
- ☑ When did it start?
- ☑ What changed recently?
- ☑ What are the contributing factors?
- ☑ What is the root cause?
- ☑ How do we fix it permanently?
- ☑ How do we prevent recurrence?

COMMON PREVENTIVE STRATEGIES

- ☑ Monitor pipeline health continuously.
- ☑ Automate alerts and notifications.
- ☑ Use retries and fallbacks.
- ☑ Keep runners updated and scalable.
- ☑ Validate configurations before deployment.
- ☑ Test in staging before production.
- ☑ Document issues and solutions.
- ☑ Improve pipeline reliability over time.

EXAMPLE: FAILED JOB LOG (SIMPLIFIED)

```
$ Running with gitlab-runner 16.3.0
$ Preparing the "docker" executor
$ Using Docker executor with image node:18
$ Pulling docker image node:18 ...
$ ERROR: manifest for node:18 not found
$ ERROR: Job failed: exit code 1

Root Cause: Image tag does not exist in registry.
Fix: Use valid image tag or update registry.
Action: Updated image to node:18-alpine and re-run.
```

USEFUL COMMANDS

```
# Check runner status
gitlab-runner list

# Check pipeline status
curl --header "PRIVATE-TOKEN: <token>" \
https://gitlab.com/api/v4/pipelines

# Get job logs
curl --header "PRIVATE-TOKEN: <token>" \
https://gitlab.com/api/v4/jobs/<id>/trace

# Retry a failed job
curl --request POST --header "PRIVATE-TOKEN: <token>" \
https://gitlab.com/api/v4/jobs/<id>/retry
```

COMMON PITFALLS

- ✕ Ignoring warnings and small errors
- ✕ Not reading job logs
- ✕ Assuming it's a GitLab issue
- ✕ Using hardcoded values
- ✕ Not testing changes
- ✕ Not documenting fixes

TOOLS TO DEBUG

- GitLab CI/CD Logs
- GitLab Runner Logs
- Docker Logs
- Kubernetes Logs
- System Logs
- Monitoring & Alerts

KEY PRINCIPLE

Fast recovery is good.
Correct diagnosis is better,
Preventing recurrence
is the best.

TIP

Don't just fix the problem. Understand the root cause, improve the system, and make the pipeline stronger for the future.

50. Pipeline Debugging

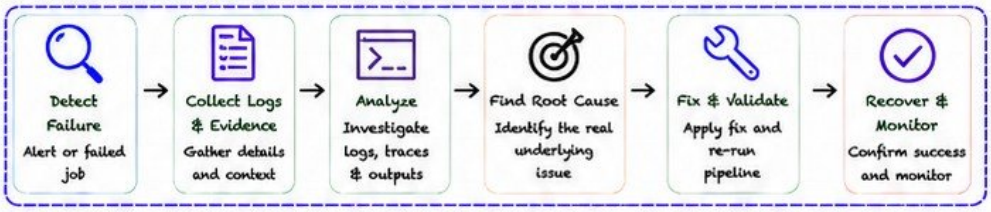
KEY TAKEAWAY

- ☑ Debugging is about visibility into what the pipeline is doing.
- ☑ Use logs, traces, and tools to find the exact failure point.
- ☑ Reproduce, understand, and fix the root cause.
- ☑ Recover safely and prevent recurrence.

WHAT IS PIPELINE DEBUGGING?

Pipeline debugging is the process of identifying, isolating, and resolving issues in GitLab CI/CD pipelines. It involves collecting evidence, analyzing logs and traces, reproducing failures, and applying fixes to restore successful delivery.

PIPELINE DEBUGGING WORKFLOW OVERVIEW



1 DEBUG LOGGING

- Enable detailed logs for jobs.
- Increase verbosity for tools.
- Capture all relevant output.
- Store logs as job artifacts.

COMMON CAUSES

- Insufficient log level
- Missing log output
- Filtered or masked logs
- Short log retention

QUICK CHECKS

- Enable debug mode
- Check job log settings
- Review script output
- Download full logs

2 TRACE OUTPUT

- Generate trace logs.
- Follow execution step-by-step.
- Capture API calls and responses.
- Use traces to understand flow.

COMMON CAUSES

- Missing trace configuration
- Network calls not traced
- Short trace retention
- Tools not instrumented

QUICK CHECKS

- Enable trace mode
- Check service endpoints
- Inspect request/response
- Review trace timeline

3 INTERACTIVE TROUBLESHOOTING

- Re-run job in debug mode.
- Use SSH or shell access.
- Inspect environment directly.
- Test commands manually.

COMMON CAUSES

- Environment differences
- Non-reproducible locally
- Missing dependencies
- Access or permission limits

QUICK CHECKS

- Run job with `when: manual`
- Use SSH into runner/container
- Reproduce failing command
- Check environment variables

4 FAILURE INVESTIGATION

- Identify failing step.
- Analyze error messages.
- Correlate logs, traces, metrics.
- Ask the 5 Whys.

COMMON CAUSES

- Code or script errors
- Configuration mismatch
- Dependency failures
- External service issues

QUICK CHECKS

- Check error logs
- Verify configs & secrets
- Validate dependencies
- Check service health

5 RECOVERY WORKFLOW

- Apply fix or rollback.
- Re-run pipeline safely.
- Validate deployment.
- Monitor and confirm stability.

COMMON CAUSES

- Partial fixes
- Rollback not tested
- Unverified recovery
- Missing validation steps

QUICK CHECKS

- Re-run pipeline
- Verify deployment
- Monitor metrics
- Confirm end-to-end

DEBUG LOGGING EXAMPLE (.GITLAB-CI.YML)

```yaml
build:
  script:
    - export CI_DEBUG_TRACE="true"
    - export CI_DEBUG_SERVICES="true"
    - echo "Starting build with debug logs"
    - make build
  variables:
    GIT_STRATEGY: fetch
    FF_SCRIPT_SECTIONS: "true"
  artifacts:
    when: always
    paths:
      - debug.log
      - build.log
```

INTERACTIVE DEBUGGING EXAMPLE

```yaml
# Run job manually with SSH access
job:
  stage: debug
  script:
    - echo "Opening shell for debugging..."
    - apk add --no-cache bash
    - bash
  when: manual
  allow_failure: true
```

TIP: Use manual jobs to open a shell and investigate the runner environment.

USEFUL CLI COMMANDS

```
# View pipeline details
  glab pipeline view <pipeline-id> --log
# View job trace
  glab api v4/projects/:id/jobs/:job_id/trace
# Retry failed job
  glab pipeline retry <pipeline-id> --job <job-name>
# Get job artifacts
  glab api v4/projects/:id/jobs/:job_id/artifacts
# Check runner status
  glab api v4/runners/all
# List pipelines
  glab pipeline list --limit 10
```

BEST PRACTICES

- ☑ Always reproduce the issue.
- ☑ Enable debug only when needed.
- ☑ Keep logs clear and actionable.
- ☑ Use structured logging.
- ☑ Document fixes and lessons learned.
- ☑ Automate validations and tests.
- ☑ Monitor pipelines and alert early.

COMMON PITFALLS

- ✗ Fixing symptoms, not root cause
- ✗ Not enough logs or context
- ✗ Ignoring intermittent failures
- ✗ Overlooking environment vars
- ✗ Skipping validation steps
- ✗ Uncontrolled manual changes
- ✗ Not learning from incidents

USEFUL TOOLS

- GitLab CI Lint
- CI/CD Pipeline Editor
- Job Artifacts
- Trace Viewer
- Runner Logs
- Docker Logs
- Kubernetes Logs
- Prometheus / Grafana
- Sentry / Error Tracking
- ELK / Loki / Fluentd

TIP

Great engineers dont just fix pipelines – they build systems that are observable, reliable, and self-healing.

51. High Availability GitLab

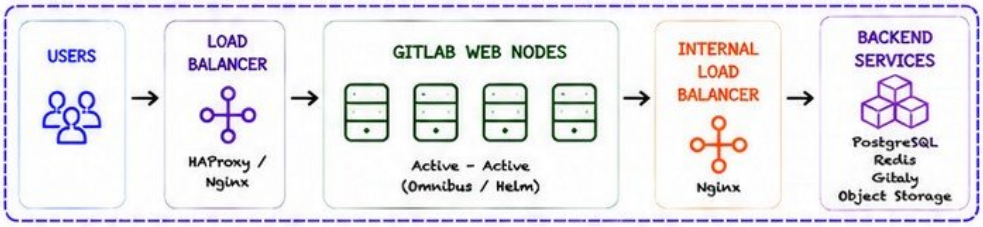
KEY TAKEAWAY

- [x] High availability ensures GitLab stays online even when failures happen.
- [x] Design for redundancy at every layer.
- [x] Data, services, and infrastructure must be resilient.
- [x] Test failover regularly.
- [x] Plan for disaster, not if, but when.

WHAT IS HIGH AVAILABILITY GITLAB?

High Availability GitLab ensures the platform remains accessible, reliable, and performant even during hardware, network, or software failures. It uses redundancy, load balancing, clustering, and automated failover to deliver continuous service.

HA ARCHITECTURE OVERVIEW



1 HA ARCHITECTURE COMPONENTS

- Redundant GitLab web nodes behind load balancer.
- PostgreSQL primary with standby replicas.
- Redis in sentinel or cluster mode.
- Gitaly clustered for repository storage.
- Shared storage for uploads, artifacts, and LFS.
- External object storage for high durability.

POSTGRESQL HA

- Primary + multiple read replicas.
- Streaming replication enabled.
- Automatic failover with Patroni.
- Regular backups to object storage.
- Test restore and failover routinely.

BEST PRACTICE
Use at least 1 primary + 2 replicas across different availability zones.

REDIS HA

- Use Redis Sentinel or Redis Cluster.
- Automatic leader election.
- Data replication and failover.
- Monitor memory, CPU, connections.
- Enable persistence (AOF + RDB).

BEST PRACTICE
Redis is critical for sessions, cache, queues, and CI/CD. Keep it highly available.

GITALY HA

- Gitaly handles all Git operations.
- Run multiple Gitaly nodes.
- Use Praefect for smart load balancing.
- Data stored on shared storage or MinIO.
- Replicates repositories across nodes.

BEST PRACTICE
Never run a single Gitaly node in production. Use at least 3 nodes.

LOAD BALANCING

- Distribute traffic across web nodes.
- Health checks to remove failed nodes.
- Use sticky sessions only if required.
- SSL termination at load balancer.
- Rate limiting and DDoS protection.

BEST PRACTICE
Use HAProxy or Nginx with active health checks and automatic failover.

SHARED & OBJECT STORAGE

- Use NFS / Ceph / EFS for shared storage.
- Store uploads, artifacts, and LFS here.
- Use S3 / MinIO for object storage.
- Enable versioning and lifecycle rules.
- Ensure cross-region replication.

BEST PRACTICE
Object storage should be durable, versioned, and replicated across regions.

DISASTER RESILIENCE

- Multi-AZ deployment for all components.
- Cross-region backup of PostgreSQL.
- Replicate object storage across regions.
- Document RTO (Recovery Time Objective).
- Regular DR drills and failover tests.

BEST PRACTICE
Assume a region will fail. Prepare to recover in another region quickly.

MONITORING & ALERTING

- Monitor all services and dependencies.
- Use Prometheus + Grafana for metrics.
- Alert on performance and failure events.
- Log aggregation with ELK / Loki.
- Dashboard your HA status clearly.

BEST PRACTICE
You can't fix what you don't monitor. Dashboards save your uptime.

HA CHECKLIST

- [x] Multiple GitLab web nodes behind LB
- [x] PostgreSQL primary + replicas with failover
- [x] Redis Sentinel / Cluster configured
- [x] Gitaly Cluster with Praefect
- [x] Shared storage and object storage setup
- [x] Backups automated and tested
- [x] Monitoring and alerting active
- [x] Disaster recovery plan documented
- [x] Regular HA and DR testing

COMMON PITFALLS

- ✗ Single point of failure anywhere
- ✗ No failover testing
- ✗ Insufficient monitoring
- ✗ Backups not tested
- ✗ Ignoring capacity and scaling
- ✗ Not separating components
- ✗ Poor network design

USEFUL COMMANDS

```
# Check PostgreSQL replication status
sudo -u gitlab-psql psql -c "SELECT * FROM pg_stat_replication;"

# Check Redis Sentinel status
redis-cli -p 26379 sentinel masters

# Check Gitaly nodes
sudo gitlab-rake gitlab:gitaly:check

# Check GitLab health
curl -I https://gitlab.example.com/-/readiness
```

TIP

High availability is not a feature, it's an architecture mindset. Build it, test it, monitor it, and trust it.

52. Disaster Recovery

KEY TAKEAWAY

- [x] Plan for disaster before it happens.
- [x] Backups are useless if you can't restore.
- [x] Test your recovery regularly.
- [x] Document every step.
- [x] Recover fast. Recover right.
- [x] Trust is built through preparation.

WHAT IS DISASTER RECOVERY?

Disaster Recovery (DR) is the ability to restore GitLab and its data in another environment after a major failure such as region outage, hardware loss, corruption, or human error. A strong DR plan ensures minimal data loss and maximum uptime.

DR STRATEGY OVERVIEW

PREPARE → Design DR plan & backups

BACKUP → Backup data & configuration regularly

REPLICATE → Store backups offsite / in another region

RESTORE → Restore systems & validate integrity

TEST → Test DR plan regularly

RECOVER → Failover & resume operations

1 BACKUP STRATEGY

- Backup GitLab database (PostgreSQL).
- Backup repositories (Gitaly storage).
- Backup Redis data (AOF or RDB).
- Backup configuration files.
- Backup CI/CD artifacts and uploads.
- Store backups offsite / in another region.

BEST PRACTICE
Follow the 3-2-1 rule: 3 copies, 2 media types, 1 offsite copy.

2 RESTORE PROCESS

- Prepare target environment.
- Restore database first.
- Restore Redis data.
- Restore Gitaly repositories.
- Restore uploads and artifacts.
- Update configuration.
- Validate services.

BEST PRACTICE
Document every restore step with commands and checks.

3 REPOSITORY RECOVERY

- Repositories are stored on Gitaly.
- Restore Gitaly storage from backup.
- Verify repository integrity.
- Run gitaly check.
- Rebuild repository cache if needed.
- Verify access and permissions.

BEST PRACTICE
Use Gitaly geo-replication for faster repository recovery.

4 DATABASE RECOVERY

- Restore PostgreSQL from backup.
- Ensure same version compatbility.
- Run gitlab-rake db:migrate.
- Check database connections.
- Verify background jobs.
- Validate all GitLab services.

BEST PRACTICE
Use PITR (Point-in-Time Recovery) for minimal data loss.

5 RUNNER RECOVERY

- Re-register GitLab Runners.
- Restore runner configuration.
- Ensure runner tokens are valid.
- Check runner tags and permissions.
- Validate runner connectivity.

BEST PRACTICE
Reep runner automation scripts and tokens securely backed up.

6 DISASTER RECOVERY TESTING

- Test DR plan regularly (quarterly).
- Simulate real-world failure scenarios.
- Measure RTO (Recovery Time Objective).
- Measure RPO (Recovery Point Objective).
- Document lessons learned.
- Improve DR plan after each test.

BEST PRACTICE
A DR plan not tested is a DR plan that will fail.

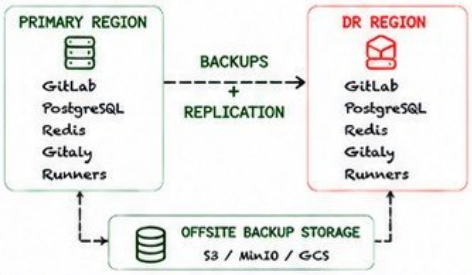
7 RTO & RPO GUIDELINES

METRIC		DESCRIPTION
RTO	Recovery Time Objective	Maximum acceptable time to restore systems.
RPO	Recovery Point Objective	Maximum acceptable data loss in time.
EXAMPLE	RTO: 1 hour RPO: 15 minutes	System must be up within 1 hour, data loss < 15 min.

GITLAB BACKUP COMPONENTS

COMPONENT	WHAT TO BACK UP	RECOMMENDED FREQUENCY
PostgreSQL	Database dumps	Daily + PITR
Gitaly Repositories	Repository storage	Daily
Redis	RDB or AOF files	Daily
Uploads / Artifacts	Object storage	Daily
CI/CD Variables	Configuration backup	Daily
GitLab Config	/etc/gitlab	Daily
Secrets & Tokens	Runner tokens, deploy keys	Weekly
Monitoring Data	Prometheus, Grafana	Weekly

DR ARCHITECTURE EXAMPLE



RECOVERY CHECKLIST

- [x] Target environment ready
- [x] Database restored
- [x] Redis restored
- [x] Gitaly repositories restored
- [x] Uploads & artifacts restored
- [x] Configuration files restored
- [x] Services restarted
- [x] Health checks passed
- [x] CI/CD pipelines tested
- [x] User access verified
- [x] Monitoring & alerts active

COMMON PITFALLS

- ✗ No backups or incomplete backups
- ✗ Backups stored in same location
- ✗ Restore process not documented
- ✗ DR plan never tested
- ✗ Ignoring RPO and RTO

USEFUL COMMANDS

```
# Backup GitLab database
sudo gitlab-rake gitlab:backup:create

# Restore GitLab
sudo gitlab-rake gitlab:backup:restore BACKUP=backup_file.tar

# Check GitLab status
sudo gitlab-ctl status

# Check Gitaly repositories
sudo gitlab-rake gitaly:check

# Reconfigure GitLab
sudo gitlab-ctl reconfigure
```

TIP

Recovery is not just about restoring data. It's about restoring trust, uptime, and user confidence.

53. Scaling GitLab CI/CD

KEY TAKEAWAY

- ☑ Scaling is required when load increases beyond current capacity.
- ☑ Scale GitLab components independently.
- ☑ Autoscale runners based on demand.
- ☑ Monitor, tune, and plan capacity continuously.
- ☑ Design for performance, not just to handle traffic.

WHAT IS SCALING GITLAB CI/CD?

Scaling GitLab CI/CD means increasing the capacity of GitLab and its runners to handle more users, projects, pipelines, and jobs efficiently. It involves scaling GitLab components, runners, infrastructure, and storage while maintaining performance and reliability.

SCALING OVERVIEW

INCREASE LOAD → More users, projects, pipelines, jobs

SCALE COMPONENTS → GitLab, Runners, Database, Storage

AUTOSCALE RUNNERS → Scale on demand

OPTIMIZE PERFORMANCE → Tune & monitor

RELIABLE & FAST → High availability & performance

1 HORIZONTAL SCALING

- Scale GitLab web nodes behind load balancer.
- Add more Sidekiq nodes for background jobs.
- Scale Gitaly nodes for Git operations.
- Use read replicas for PostgreSQL.
- Scale Redis (Sentinel/Cluster).

BEST PRACTICE
Scale each component independently based on bottlenecks.

2 RUNNER AUTOSCALING

- Use autoscaling with Kubernetes or cloud auto scaling groups.
- Scale runners based on job queue length or CPU utilization.
- Use ephemeral runners for elastic scale.
- Set maximum and minimum runner limits.

BEST PRACTICE
Autoscaling runners reduces wait time and saves cost.

3 LARGE ENTERPRISE DEPLOYMENTS

- Use multi-node GitLab EE.
- Separate database, Redis, and object storage.
- Use multiple runner fleets for different workloads.
- Implement Geo for global distribution.
- Use feature flags to manage load.

BEST PRACTICE
Isolate workloads and scale in layers.

4 PERFORMANCE TUNING

- Optimize PostgreSQL (indexes, vacuum, connection pooling).
- Tune Redis memory and policies.
- Optimize Sidekiq concurrency and queues.
- Use SSD storage for databases and repositories.
- Enable caching (dependencies, Docker layers).

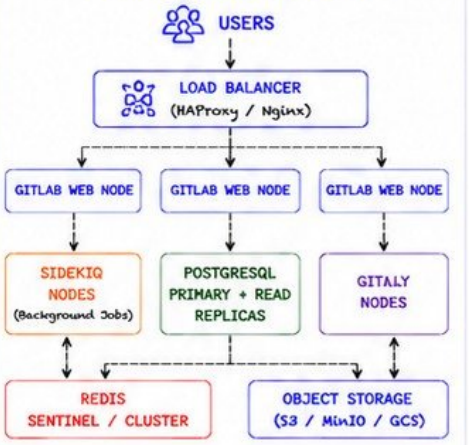
BEST PRACTICE
Small tuning changes can improve performance by up to 10x.

5 CAPACITY PLANNING

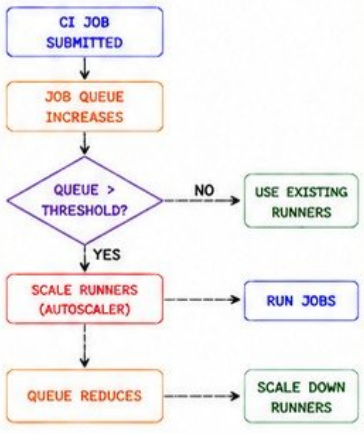
- Analyze current usage trends.
- Forecast growth in projects, users, and pipelines.
- Plan for peak usage (business hours, releases).
- Define resource limits and scaling triggers.
- Review and adjust capacity regularly.

BEST PRACTICE
Plan for 30–50% more capacity than current peak usage.

TYPICAL SCALING ARCHITECTURE



RUNNER AUTOSCALING FLOW



SCALING COMPONENT BREAKDOWN

COMPONENT	SCALE METHOD	TRIGGERS
GitLab Web Nodes	Add nodes behind LB	High web traffic
Sidekiq Nodes	Add Sidekiq nodes	High job queue
Gitaly Nodes	Add Gitaly nodes	High Git operations
PostgreSQL	Read replicas	High DB load
Redis	Sentinel / Cluster	High memory / CPU
Runners	Autoscaling	Job queue length
Object Storage	Scale buckets	Storage growth

KEY METRICS TO MONITOR

- ☑ CI job queue length
- ☑ Pipeline duration
- ☑ Runner utilization (CPU / Memory)
- ☑ PostgreSQL CPU, connections, slow queries
- ☑ Redis memory usage
- ☑ Gitaly RPC latency
- ☑ Disk I/O and storage usage
- ☑ Error rate and failure rate

COMMON PITFALLS

- ✗ Scaling only runners, not core components
- ✗ Ignoring database performance
- ✗ Not monitoring job queue length
- ✗ Over-provisioning without analysis
- ✗ Not testing scaling before peak load

USEFUL COMMANDS

```
# GitLab status
sudo gitlab-rake gitlab:env:info

# Check Sidekiq queue
sudo gitlab-rake gitlab:sidekiq:status

# Runner list
sudo gitlab-runner list

# Monitor GitLab
watch -n 5 sudo gitlab-ctl status

# Gitaly node info
sudo gitlab-rake gitlab:gitaly:check
```

TOOLS FOR SCALING

- GitLab Built-in Monitoring
- Prometheus + Grafana
- k6 / JMeter (Load Testing)
- Datadog / New Relic
- Kubernetes HPA
- Auto Scaling Groups (AWS)
- Redis Sentinel / Cluster

TIP

Scale before users complain, not after.
A well-scaled pipeline is invisible — it just works.

54. Enterprise Security Operations

KEY TAKEAWAY

- ☑ Security is everyone's responsibility, but access must be controlled.
- ☑ RBAC ensures the right access at the right level.
- ☑ Audit everything. Visibility drives accountability.
- ☑ Integrate identity, enforce Zero Trust, and continuously monitor.
- ☑ Security is not a feature, it's the foundation.

WHAT IS ENTERPRISE SECURITY OPERATIONS?

Enterprise Security Operations in GitLab ensures the platform, data, and workloads are protected through strong identity, access control, monitoring, compliance, and Zero Trust principles. It's about preventing breaches, detecting threats, and maintaining continuous compliance.

SECURITY OPERATIONS OVERVIEW

IDENTITY & ACCESS — Verify users and devices → **ACCESS CONTROL** — Enforce least privilege → **AUDIT & MONITORING** — Log and detect all activity → **COMPLIANCE REPORTING** — Meet internal and external standards → **THREAT DETECTION** — Detect and respond fast → **ZERO TRUST PRINCIPLES** — Never trust, always verify

1 ACCESS CONTROL

- Enforce least privilege access.
- Manage users, groups, and permissions.
- Restrict access to projects, runners, and environments.
- Use protected branches and environments.

BEST PRACTICE
Give minimum access required. Review access regularly.

2 RBAC (ROLE-BASED ACCESS CONTROL)

- Use predefined roles: Owner, Maintainer, Developer, Reporter, Guest.
- Custom roles for fine-grained control.
- Group inheritance simplifies management.

BEST PRACTICE
Map roles to responsibilities. Avoid over-privileged roles.

3 AUDIT LOGS

- Track every user action.
- Logins, changes, API calls, pipeline runs, approvals.
- Logs are immutable and tamper-proof.
- Stream logs to SIEM for analysis.

BEST PRACTICE
Enable audit logs and retain for compliance period.

4 COMPLIANCE REPORTING

- Generate reports for SOC 2, ISO 27001, GDPR, HIPAA, PCI DSS.
- Use Compliance Center to monitor controls.
- Schedule automated compliance reports.

BEST PRACTICE
Automate reporting and review exceptions regularly.

5 IDENTITY INTEGRATION

- Integrate with SSO (SAML, OAuth, OIDC).
- Use SCIM for user provisioning.
- Enforce MFA for all users.
- Centralize identity management.

BEST PRACTICE
Use SSO + MFA for all users and admins.

6 ZERO TRUST PRINCIPLES

- Never trust by default.
- Verify every request.
- Enforce MFA, IP allowlists, and device trust.
- Micro-segment access to resources.

BEST PRACTICE
Assume breach. Verify explicitly. Limit blast radius.

RBAC ROLE MATRIX (EXAMPLE)

ROLE	DESCRIPTION	TYPICAL USE
Owner	Full administrative access	Platform admins
Maintainer	Manage projects and settings	Team leads
Developer	Code, pipelines, and deployments	Engineers
Reporter	Read-only access	Auditors, viewers
Guest	Limited external access	External users

AUDIT LOG FLOW

USER ACTION (Login / Push / Change Settings) → GITLAB AUDIT LOG → LOG STORAGE (Elasticsearch / SIEM) → ANALYZE & ALERT (Monitoring / SIEM Alerts)

BEST PRACTICE
Centralize logs, set up alerts, and review security events daily.

IDENTITY INTEGRATION FLOW

USER → SSO / IDP (SAML / OIDC) → GITLAB ACCESS → MFA VERIFICATION

BEST PRACTICE
Use SSO + MFA + SCIM for seamless and secure access management.

SECURITY CONTROLS CHECKLIST

- ☑ MFA enabled for all users
- ☑ SSO integrated and enforced
- ☑ RBAC roles reviewed regularly
- ☑ Protected branches and environments
- ☑ Audit logs enabled and monitored
- ☑ Compliance reports generated
- ☑ Secrets managed and rotated
- ☑ Runners isolated and secured
- ☑ Regular security scans enabled
- ☑ Vulnerability management in place

COMMON PITFALLS

- ✕ Over-permissioned users
- ✕ Disabled or ignored audit logs
- ✕ No MFA or weak passwords
- ✕ Public projects with sensitive data
- ✕ Unreviewed access over time
- ✕ No compliance monitoring
- ✕ Shared accounts
- ✕ Ignoring security alerts

USEFUL COMMANDS (API / CLI)

```
# List audit events
curl --header "PRIVATE-TOKEN: <token>" \
https://gitlab.com/api/v4/audit_events

# Get group members
curl --header "PRIVATE-TOKEN: <token>" \
https://gitlab.com/api/v4/groups/:id/members

# Get compliance frameworks
curl --header "PRIVATE-TOKEN: <token>" \
https://gitlab.com/api/v4/compliance/frameworks

# List protected branches
curl --header "PRIVATE-TOKEN: <token>" \
https://gitlab.com/api/v4/projects/:id/protected_branches
```

TIP

Security is a journey, not a destination. Continuously assess, automate, and improve. A secure pipeline builds a secure future.

ZERO TRUST PRINCIPLES APPLIED TO GITLAB

- **VERIFY** — Every user, device, and request
- **LIMIT ACCESS** — Least privilege access to resources
- **ASSUME BREACH** — Monitor and detect abnormal behavior

CORE SECURITY PILLARS

- **CONFIDENTIALITY** — Protect data and secrets
- **INTEGRITY** — Ensure data accuracy and consistency
- **AVAILABILITY** — Keep systems available and resilient

55. Production Architecture Case Study

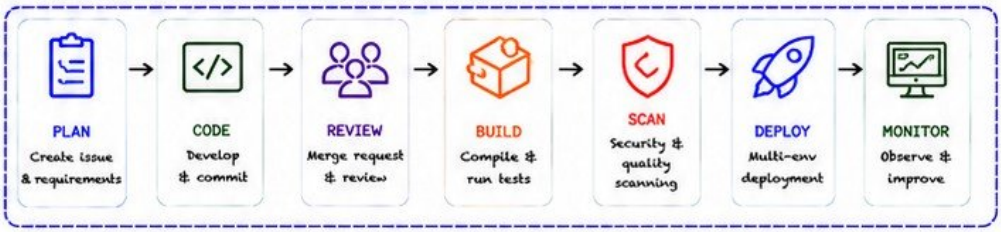
KEY TAKEAWAY

- ☑ Good architecture solves real business problems.
- ☑ Security, reliability, and operability are non-negotiable.
- ☑ Automate everything. Document everything.
- ☑ Design for failure. Test for failure. Recover fast.
- ☑ Architecture is a living system, not a diagram.

WHAT THIS CASE STUDY COVERS

This case study demonstrates an enterprise-grade DevOps platform using GitLab CI/CD deployed across multiple environments with strong security, high availability, and full observability.

ENTERPRISE DEVELOPMENT WORKFLOW OVERVIEW



1 ENTERPRISE DEVELOPMENT WORKFLOW

- Developers work in feature branches.
- Merge requests enforce reviews and quality gates.
- Protected branches for mainline.
- Trunk-based development.
- Automated testing on every commit.

BEST PRACTICE
Small, frequent commits. Automate reviews and tests.

2 MULTI-ENVIRONMENT PIPELINE

- Environments: Dev → Test → Staging → Prod
- Environment isolation with separate kubernetes namespaces/accounts.
- Manual approval for Staging/Prod.
- Artifact promotion, not rebuild.
- Environment-specific variables and configuration.

BEST PRACTICE
Promote the same artifact across environments. No drift.

3 SECURITY INTEGRATION

- SAST, DAST, Dependency, Container scanning in pipeline.
- Secrets managed with Vault / CI variables (masked & protected).
- Infrastructure as Code scanning.
- Policy as Code with OPA / Conftest.
- Image signing and verification.

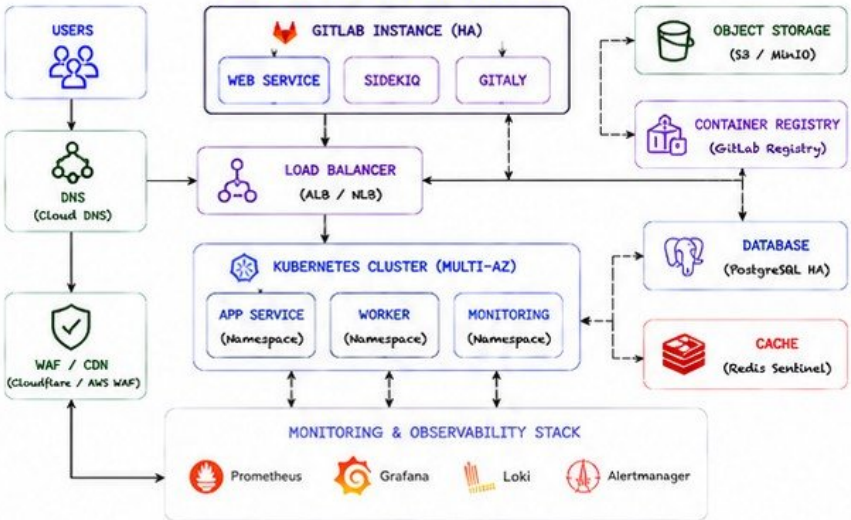
BEST PRACTICE
Fail the pipeline if critical vulnerabilities are found.

4 DEPLOYMENT FLOW

- CI builds and tests the code.
- CD deploys to target environment.
- Blue/Green or Canary for Prod.
- Health checks and automated rollback.
- Post-deployment smoke tests.

BEST PRACTICE
Deploy small, verify fast, rollback automatically.

REFERENCE ARCHITECTURE DIAGRAM



5 OPERATIONAL CONSIDERATIONS

- High Availability across AZs.
- Backups for DB, Registry, and GitLab.
- Monitoring, alerting, and dashboards.
- Centralized logging and audit trails.
- Capacity planning and autoscaling.
- Disaster recovery and regular DR tests.

BEST PRACTICE Automate backups, test restores, and review alerts daily.

TECHNOLOGY SUMMARY

COMPONENT	TECHNOLOGY CHOICES
Source Control	GitLab
CI/CD	GitLab CI/CD
Runners	Kubernetes Autoscaled Runners
Containers	Docker
Orchestration	Kubernetes
Registry	GitLab Container Registry
Storage	S3 / MinIO
Database	PostgreSQL (HA)
Cache	Redis (Sentinel)
Monitoring	Prometheus, Grafana, Alertmanager
Logging	Loki, Promtail
Security	SAST, DAST, Trivy, Vault, OPA

ENVIRONMENT STRATEGY

ENVIRONMENT	PURPOSE
DEVELOPMENT	Active development & unit testing
TEST	Integration & system testing
STAGING	Pre-production validation
PRODUCTION	Live traffic & real users

PIPELINE STAGES (EXAMPLE)

1. Lint & Code Quality
2. Unit Tests
3. Build & Package
4. Security Scans
5. Integration Tests
6. Deploy to Test
7. Deploy to Staging (Manual)
8. Deploy to Production (Manual)
9. Post-Deploy Tests

COMMON PITFALLS

- ✗ Direct commits to main branch
- ✗ Skipping code reviews
- ✗ No environment isolation
- ✗ Hardcoded secrets
- ✗ Manual deployments
- ✗ No rollback strategy
- ✗ Ignoring monitoring alerts

SUCCESS PRINCIPLES

- ☑ Automate everything
- ☑ Secure by design
- ☑ Observe everything
- ☑ Design for failure
- ☑ Document everything
- ☑ Continuously improve

TIP

Architecture is not about using many tools. It's about using the right tools in the right way.

56. Senior Engineer Best Practices

KEY TAKEAWAY

- ☑ Great pipelines are simple, fast, secure, reliable, and maintainable.
- ☑ Design for change, scale, and failure.
- ☑ Automate with purpose, not just to save clicks.
- ☑ Measure everything. Improve continuously.
- ☑ Document clearly. Share knowledge.
- ☑ Strong teams build strong pipelines.

WHAT MAKES A SENIOR ENGINEER?

Senior engineers don't just build pipelines that work. They build pipelines that are reliable, scalable, observable, secure, cost-effective, and easy for the entire team to understand and evolve.

PIPELINE DESIGN PRINCIPLES

SIMPLE	FAST	RELIABLE	SECURE	COST-EFFECTIVE	MAINTAINABLE
Keep pipelines as simple as possible.	Optimize for speed without sacrificing quality.	Build for failure. Pipelines must recover and retry smartly.	Security is everyone's responsibility.	Optimize resources, runners, storage, and usage.	Easy to read, change, extend, and debug.

1 MAINTAINABILITY

- Use modular includes and templates.
- Keep jobs small and focused.
- Avoid copy-paste. Reuse components.
- Use meaningful names and clear variables.
- Version your pipeline libraries.

BEST PRACTICE
A maintainable pipeline saves hours of debugging and speeds up onboarding.

2 COST OPTIMIZATION

- Use autoscaling runners.
- Right-size runner instance types.
- Use spot/preemptible instances.
- Clean up environments and artifacts.
- Optimize caching and storage.

BEST PRACTICE
Monitor usage and spend. Optimize continuously.

3 RELIABILITY

- Design for idempotency.
- Use retry with backoff.
- Handle failures gracefully.
- Add timeouts to all jobs.
- Validate critical steps.

BEST PRACTICE
Reliable pipelines reduce rework and increase confidence.

4 TEAM COLLABORATION

- Follow code review for pipeline changes.
- Use merge requests for .gitlab-ci.yml.
- Share pipeline libraries across teams.
- Document decisions and trade-offs.
- Encourage feedback and improvements.

BEST PRACTICE
Collabboration and transparency build better pipelines and stronger teams.

5 DOCUMENTATION STANDARDS

- Document pipeline purpose and flow.
- Comment complex logic.
- Maintain README in repo.
- Document variables and secrets usage.
- Keep runbooks for incidents.

BEST PRACTICE
Good documentation reduces onboarding time and prevents mistakes.

6 OBSERVABILITY & FEEDBACK

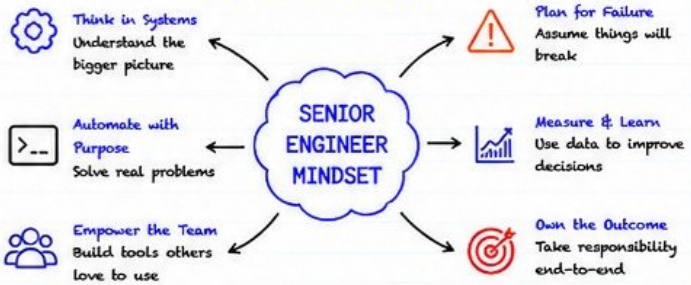
- Enable detailed logging.
- Use metrics, dashboards, and alerts.
- Track DORA metrics.
- Provide fast feedback to developers.
- Make failures easy to diagnose.

BEST PRACTICE
You can't improve what you don't measure.

PIPELINE QUALITY CHECKLIST

- ☑ Pipeline is simple and easy to understand
- ☑ Jobs are modular and reusable
- ☑ Security scans are integrated
- ☑ Tests are reliable and fast
- ☑ Artifacts are managed and retained properly
- ☑ Caching is optimized
- ☑ Runners are autoscaled
- ☑ Pipeline has timeouts and retries
- ☑ Logging and monitoring are configured
- ☑ Documentation is up to date
- ☑ Cost is monitored and optimized

SENIOR ENGINEER MINDSET



TIP

The best pipelines are boring.
They just work, every time.
Your users don't see the pipeline.
They feel the results.

COMMON PITFALLS

- ✗ Overly complex pipelines
- ✗ Hardcoded values
- ✗ Ignoring security
- ✗ Long feedback cycles
- ✗ No monitoring or alerts
- ✗ Poor documentation

USEFUL COMMANDS

```
# Trigger pipeline
curl --request POST --header "PRIVATE-TOKEN: <token>" \
https://gitlab.com/api/v4/projects/:id/pipeline

# Get pipeline status
curl --header "PRIVATE-TOKEN: <token>" \
https://gitlab.com/api/v4/projects/:id/pipelines

# Get job logs
curl --header "PRIVATE-TOKEN: <token>" \
https://gitlab.com/api/v4/projects/:id/jobs/:job_id/trace

# Get runner list
gitlab-runner list
```

QUALITY METRICS (DORA)

- **Deployment Frequency** — How often you deploy
- **Lead Time for Changes** — Commit to production
- **Change Failure Rate** — % of deployments causing failures
- **Mean Time to Restore** — Time to recover from failure

57. Enterprise CI/CD Checklist

KEY TAKEAWAY

- ☑ A great pipeline is more than automation.
- ☑ Every layer must be secure, reliable, observable, and scalable.
- ☑ Check everything before you deploy. Trust but verify.
- ☑ Standardization reduces risk and increases speed.
- ☑ Document, automate, monitor, and continuously improve.

WHAT IS THIS CHECKLIST?

This checklist ensures your CI/CD pipeline and delivery process are enterprise-ready, secure, reliable, scalable, observable, and operationally excellent from code commit to production.

CI/CD LIFECYCLE OVERVIEW

1 REPOSITORY READINESS

CHECKPOINT	STATUS
Code is stored in Git with branch strategy	☐
README.md is updated and clear	☐
.gitignore is comprehensive	☐
Code follows style and quality standards	☐
Pre-commit hooks are configured	☐
Secrets are not committed	☐
Large files are managed with LFS	☐
Repository access is restricted	☐

BEST PRACTICE
A clean repository reduces pipeline failures and security risks.

2 PIPELINE READINESS

CHECKPOINT	STATUS
Pipeline is version controlled (.gitlab-ci.yml)	☐
Pipeline is modular and reusable	☐
Stages are logically ordered	☐
Jobs have clear responsibilities	☐
Artifacts are defined and retained	☐
Caching is configured appropriately	☐
Pipeline variables are managed securely	☐
Timeouts and retries are configured	☐

BEST PRACTICE
A well-designed pipeline is reliable, maintainable, and easy to debug.

3 SECURITY VALIDATION

CHECKPOINT	STATUS
SAST scanning is enabled	☐
Dependency scanning is enabled	☐
Container image scanning is enabled	☐
Secrets detection is enabled	☐
IaC scanning is enabled (if applicable)	☐
License compliance checks are enabled	☐
Security policies are enforced	☐
Vulnerabilities are reviewed and fixed	☐

BEST PRACTICE
Security must be integrated, not an afterthought.

4 DEPLOYMENT VALIDATION

CHECKPOINT	STATUS
Deployment to staging is automated	☐
Integration tests pass	☐
Smoke tests pass	☐
Approval gates are configured	☐
Deployment to production is controlled	☐
Rollback strategy is defined and tested	☐
Blue/Green or Canary strategy (if used)	☐
Post-deployment verification is in place	☐

BEST PRACTICE
Validate every deployment before exposing it to users.

5 MONITORING VALIDATION

CHECKPOINT	STATUS
Application metrics are collected	☐
Pipeline metrics are collected	☐
Logs are centralized and searchable	☐
Alerts are configured and tested	☐
Dashboards are available	☐
Error tracking is enabled	☐
Synthetic checks are configured	☐
Performance monitoring is enabled	☐

BEST PRACTICE
You can't improve what you don't measure.

6 OPERATIONAL READINESS

CHECKPOINT	STATUS
Runbooks are documented	☐
On-call rotation is defined	☐
Incident response process is defined	☐
Backup and recovery are tested	☐
DR plan is documented and tested	☐
Access and permissions are reviewed	☐
Cost monitoring and budgets are set	☐
Team communication channel is active	☐

BEST PRACTICE
Operational readiness ensures stability, continuity, and fast recovery.

FINAL VERIQTA PRINCIPLE

A production-ready pipeline is built on security, quality, reliability, observability, and operational excellence.

Automate with confidence. Deliver with consistency. Operate with excellence.

QUICK HEALTH SCORECARD

Repository Readiness	/ 8
Pipeline Readiness	/ 8
Security Validation	/ 8
Deployment Validation	/ 8
Monitoring Validation	/ 8
Operational Readiness	/ 8
TOTAL SCORE	/ 48

COMMON PITFALLS

- ✕ Hardcoded secrets in repositories
- ✕ Skipping tests to "save time"
- ✕ No rollback plan
- ✕ Ignoring pipeline failures
- ✕ Lack of monitoring and alerts
- ✕ No documentation

TIP

Review this checklist regularly. Pipelines age, teams change, and weaknesses creep in. Continuous reviews prevent future outages.

58. Production Readiness Assessment

KEY TAKEAWAY

- ☑ Production readiness is a continuous journey, not a one-time event.
- ☑ Maturity across people, process, tools, and technology matters.
- ☑ Higher maturity = fewer failures, faster delivery, and happier users.
- ☑ Assess, improve, and evolve consistently.

WHAT IS THIS ASSESSMENT?

This Production Readiness Assessment evaluates your DevOps platform, CI/CD pipeline, security posture, and operational capabilities across six key maturity dimensions to help you understand where you stand and how to level up.

MATURITY LEVELS GUIDE

1	2	3	4	5
INITIAL	DEVELOPING	DEFINED	MANAGED	OPTIMIZED
Ad-hoc, manual, inconsistent. High risk.	Basic processes exist but not consistent.	Standardized processes and repeatable.	Measured, monitored, and continuously improving.	Fully optimized, automated, and innovative.

CI MATURITY

CRITERIA	MATURITY SCALE (1-5) 1	2	3	4	5
• Source control workflow	○	○	○	○	○
• Automated build process	○	○	○	○	○
• Automated tests (unit/integration)	○	○	○	○	○
• Code quality checks	○	○	○	○	○
• Pipeline reliability	○	○	○	○	○
• Build reproducibility	○	○	○	○	○
• Feedback speed	○	○	○	○	○
• Developer experience	○	○	○	○	○

CURRENT LEVEL: _____

FOCUS AREA: _____

CD MATURITY

CRITERIA	MATURITY SCALE (1-5) 1	2	3	4	5
• Automated deployments	○	○	○	○	○
• Environment consistency	○	○	○	○	○
• Deployment approvals	○	○	○	○	○
• Rollback strategy	○	○	○	○	○
• Blue/Green or Canary (if used)	○	○	○	○	○
• Infrastructure as Code	○	○	○	○	○
• Release frequency	○	○	○	○	○
• Change failure rate	○	○	○	○	○

CURRENT LEVEL: _____

FOCUS AREA: _____

SECURITY MATURITY

CRITERIA	MATURITY SCALE (1-5) 1	2	3	4	5
• SAST enabled	○	○	○	○	○
• Dependency scanning	○	○	○	○	○
• Container image scanning	○	○	○	○	○
• Secret detection	○	○	○	○	○
• IaC scanning	○	○	○	○	○
• License compliance	○	○	○	○	○
• Access control & RBAC	○	○	○	○	○
• Vulnerability management	○	○	○	○	○

CURRENT LEVEL: _____

FOCUS AREA: _____

PLATFORM MATURITY

CRITERIA	MATURITY SCALE (1-5) 1	2	3	4	5
• High availability architecture	○	○	○	○	○
• Scalable runners / autoscaling	○	○	○	○	○
• Multi-environment management	○	○	○	○	○
• Self-service capabilities	○	○	○	○	○
• Reusable pipeline templates	○	○	○	○	○
• Observability integration	○	○	○	○	○
• Cost optimization	○	○	○	○	○
• Platform reliability	○	○	○	○	○

CURRENT LEVEL: _____

FOCUS AREA: _____

OPERATIONAL MATURITY

CRITERIA	MATURITY SCALE (1-5) 1	2	3	4	5
• Monitoring & alerting	○	○	○	○	○
• Incident response process	○	○	○	○	○
• Runbooks & documentation	○	○	○	○	○
• Backup & disaster recovery	○	○	○	○	○
• Performance monitoring	○	○	○	○	○
• SLA / SLO defined	○	○	○	○	○
• Capacity planning	○	○	○	○	○
• Post-incident reviews	○	○	○	○	○

CURRENT LEVEL: _____

FOCUS AREA: _____

PEOPLE & PROCESS MATURITY

CRITERIA	MATURITY SCALE (1-5) 1	2	3	4	5
• Clear ownership & roles	○	○	○	○	○
• DevOps culture	○	○	○	○	○
• Collaboration & communication	○	○	○	○	○
• Training & knowledge sharing	○	○	○	○	○
• Change management	○	○	○	○	○
• Documentation standards	○	○	○	○	○
• Continuous improvement	○	○	○	○	○
• Metrics-driven decisions	○	○	○	○	○

CURRENT LEVEL: _____

FOCUS AREA: _____

OVERALL MATURITY SCORE

Add up your scores (max 30)

TOTAL SCORE = ____ / 30

MATURITY LEVEL

0 - 10 → INITIAL
11 - 18 → DEVELOPING
19 - 24 → DEFINED
25 - 27 → MANAGED
28 - 30 → OPTIMIZED

IMPROVEMENT ROADMAP

CURRENT STATE Where you are now	→	NEXT PRIORITIES What to improve first	→	ACTIONS Key actions to take	→	TARGET STATE Where you want to be	→	SUCCESS METRICS How you will measure
_____		1. _____		• _____		_____		• _____
_____		2. _____		• _____		_____		• _____
_____		3. _____		• _____		_____		• _____
_____								

Review this assessment quarterly. Track progress. Celebrate wins. Keep improving.

TIP

Maturity is not about being perfect. It's about being better every iteration.

BEST PRACTICE

- ☑ Assess honestly, not emotionally.
- ☑ Focus on one maturity dimension at a time.
- ☑ Automate what can be automated.
- ☑ Measure everything that matters.

COMMON PITFALLS

- ✗ Over-automating without process.
- ✗ Skipping security in the pipeline.
- ✗ No rollback or recovery plan.
- ✗ Ignoring monitoring and alerts.
- ✗ No documentation or runbooks.

SUCCESS PRINCIPLES

- ☑ People + Process + Platform = Success
- ☑ Secure by design, not by chance.
- ☑ Automate with purpose.
- ☑ Observe everything.
- ☑ Improve continuously.

59. Enterprise CI/CD Architecture Review

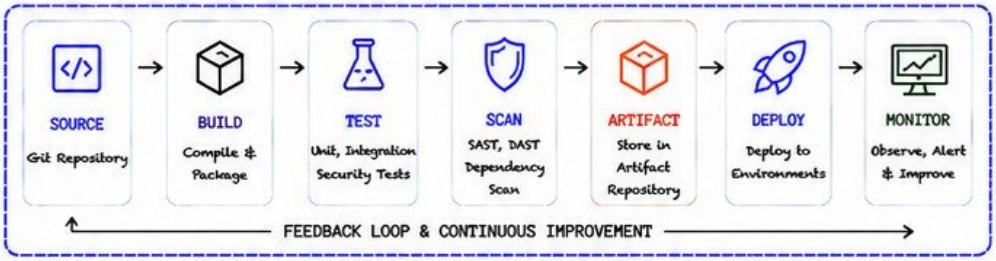
KEY TAKEAWAY

- ☑ A strong CI/CD architecture is secure, reliable, scalable, and observable.
- ☑ Design for failure, scale for growth, and automate everything.
- ☑ End-to-end visibility ensures speed with confidence.
- ☑ Security, quality, and reliability are built in, not bolted on.

WHAT IS THIS REVIEW?

This architecture review ensures your CI/CD platform is designed for enterprise scale, security, reliability, and performance from source code to production.

END-TO-END PIPELINE ARCHITECTURE



1 SOURCE TO PRODUCTION WORKFLOW

- Developer commits code
- Code pushed to repository
- Pipeline triggered automatically
- Build and tests executed
- Security scans and quality gates
- Artifact published to registry
- Deployment to target environment
- Monitoring and feedback

BEST PRACTICE
Automate every step.
Make the pipeline the single path to production.

2 SECURITY CONTROLS

- ☑ SAST scanning on every MR
- ☑ DAST scanning before production
- ☑ Dependency scanning & SBOM
- ☑ Container image scanning
- ☑ Secrets detection and scanning
- ☑ Code quality and policy enforcement
- ☑ Least privilege access for runners
- ☑ Signed artifacts and provenance
- ☑ Protected environments and approvals
- ☑ Audit logs and immutable records

BEST PRACTICE
Shift security left.
Fail fast on security issues.

3 RELIABILITY REVIEW

- ☑ Pipeline is idempotent and repeatable
- ☑ Jobs have retry and timeout policies
- ☑ Artifacts are immutable and versioned
- ☑ Environments have health checks
- ☑ Rollback strategy is automated
- ☑ Infra as Code for all environments
- ☑ Backups for critical services
- ☑ Disaster recovery plan in place
- ☑ Alerting and incident response defined
- ☑ Post-deployment verification enabled

BEST PRACTICE
Design for failure.
Automate recovery and rollback.

4 SCALABILITY REVIEW

- ☑ Runners are horizontally scalable
- ☑ Auto-scaling runners (Kubernetes / VM)
- ☑ Pipelines are optimized for parallelism
- ☑ Caching configured for speed
- ☑ Artifact storage scales automatically
- ☑ Database and services are highly available
- ☑ Load balancer distributes traffic
- ☑ Monitoring for performance metrics
- ☑ Capacity planning and stress tested
- ☑ Multi-region or multi-AZ deployment

BEST PRACTICE
Scale out, not up. Measure and plan capacity before you need it.

5 DESIGN VALIDATION

- ☑ Meets business and compliance requirements
- ☑ Supports multi-team and multi-project
- ☑ Environment strategy is well-defined
- ☑ Role-based access control validated
- ☑ Policies enforced across pipeline
- ☑ Observability and logging validated
- ☑ Cost efficient and optimized
- ☑ Documentation is complete and current
- ☑ Design reviewed and approved
- ☑ Future-ready and extensible

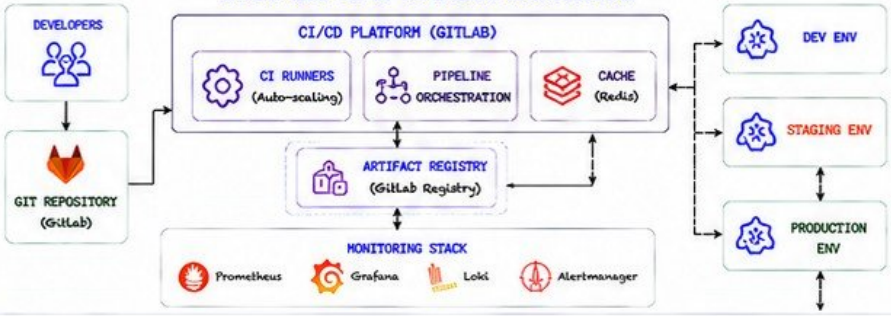
BEST PRACTICE
Validate design regularly.
Evolve based on real-world usage.

6 ARCHITECTURE REVIEW SCORECARD

DIMENSION	SCORE (1-5)
Source to Production Workflow	○ ○ ○ ○ ○
Security Controls	○ ○ ○ ○ ○
Reliability	○ ○ ○ ○ ○
Scalability	○ ○ ○ ○ ○
Design Validation	○ ○ ○ ○ ○
Observability & Monitoring	○ ○ ○ ○ ○
Operational Readiness	○ ○ ○ ○ ○

OVERALL SCORE: _____ / 35

ENTERPRISE CI/CD REFERENCE ARCHITECTURE



SECURITY & GOVERNANCE

- IAM & RBAC
- SECRET MANAGEMENT
- POLICY AS CODE
- AUDIT LOGS
- COMPLIANCE REPORTS

TIP

A great CI/CD architecture is not just about speed. It is about secure delivery, reliability, and continuous customer value.

60. Final Production Review

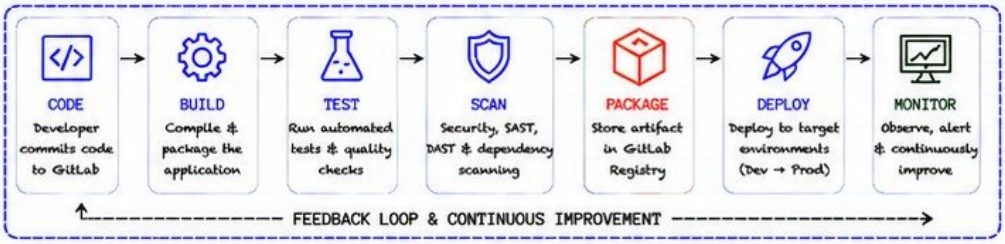
KEY TAKEAWAY

- ☑ A production-ready CI/CD pipeline is secure, reliable, observable, and continuously improving.
- ☑ End-to-end visibility ensures speed with confidence.
- ☑ Automation + quality + security = predictable delivery.
- ☑ Senior engineers design for failure, scale, and continuous value.
- ☑ Great pipelines empower teams and drive business outcomes.

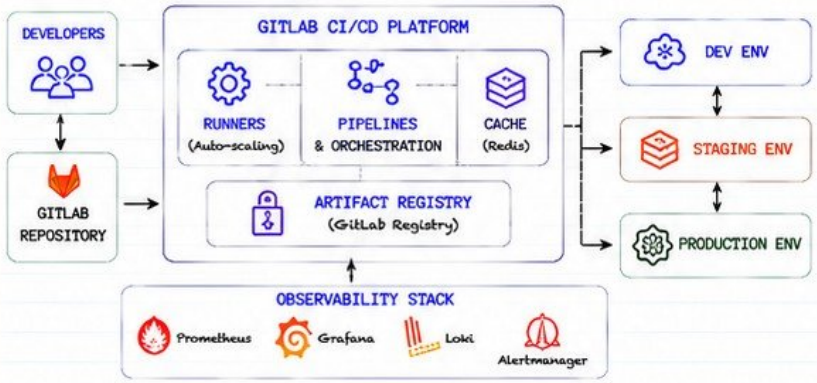
WHAT IS THIS FINAL REVIEW?

This final review brings everything together. We validate the complete GitLab CI/CD workflow, architecture, security, reliability, and operational readiness to ensure your pipeline delivers consistent value in production.

COMPLETE GITLAB CI/CD WORKFLOW (END-TO-END)



END-TO-END ARCHITECTURE SUMMARY



PRODUCTION DEPLOYMENT CHECKLIST

- ☑ Code is reviewed, tested, and approved
- ☑ All automated tests pass
- ☑ Security scans are clean
- ☑ Artifacts are signed and verified
- ☑ Dependencies are up-to-date
- ☑ Secrets are stored securely
- ☑ Infrastructure is validated
- ☑ Database migrations are tested
- ☑ Backup strategy is in place
- ☑ Rollback plan is defined and tested
- ☑ Monitoring, logging, and alerts configured
- ☑ Access control and approvals validated
- ☑ Capacity and performance validated
- ☑ Disaster recovery plan is tested
- ☑ Documentation is updated
- ☑ Stakeholders are informed

COMMON ENTERPRISE FAILURE PATTERNS

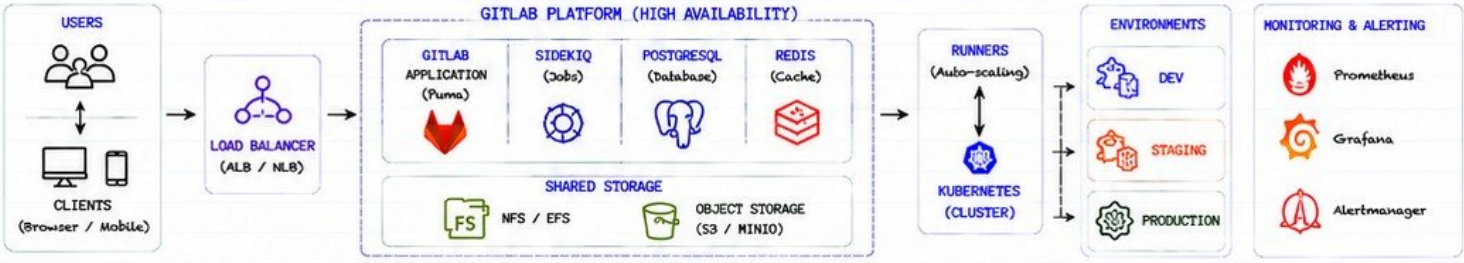
- ✗ Skipping security scans to save time
- ✗ Hardcoded secrets and credentials
- ✗ No rollback plan or untested rollback
- ✗ Pipelines that are too complex and brittle
- ✗ No environment parity (works on Dev, fails on Prod)
- ✗ Manual deployments in critical steps
- ✗ Ignoring alerts and monitoring
- ✗ Runners overloaded or not scaled
- ✗ Artifacts not versioned or not immutable
- ✗ Poor documentation and knowledge loss

SENIOR ENGINEER PRINCIPLES

- ☆ Design for failure, monitor for signals.
- ☆ Automate everything that can be automated.
- ☆ Security and quality are built-in, not bolted-on.
- ☆ Keep pipelines simple, modular, and reusable.
- ☆ Observability is not optional.
- ☆ Optimize for feedback speed.
- ☆ Standardize, document, and share knowledge.
- ☆ Continuously improve based on data.
- ☆ Empower teams with self-service platforms.
- ☆ Business value is the ultimate metric.

CONTINUOUS IMPROVEMENT ROADMAP

1. Assess current state and measure performance
2. Identify bottlenecks and improvement areas
3. Prioritize quick wins and high-impact changes
4. Implement, automate, and standardize
5. Monitor results and gather feedback
6. Iterate, optimize, and scale continuously



SECURITY & GOVERNANCE LAYER (APPLIES ACROSS EVERYTHING)

IAM & RBAC · GROUP & PROJECT PERMISSIONS · SECRET MANAGEMENT (VAULT / K8S SECRETS) · AUDIT LOGS · POLICY AS CODE (OPA / CONFTST) · COMPLIANCE REPORTING · DATA ENCRYPTION (IN TRANSIT & AT REST)

TIP

A great pipeline is never "finished". It evolves with your systems, your team, and your customers.

BEST PRACTICE

- ☑ Review and refactor pipelines regularly.
- ☑ Measure what matters and act on it.
- ☑ Automate, standardize, and improve continuously.

FINAL VERIQTA PRINCIPLE

Reliable pipelines build trust.
Trusted systems deliver value.
Value drives the business forward.